

1 Algorithm

1.1 LIS

```
1 // Longest increasing subsequence
2 int LIS(const vector<int>& s) {
3     // use binary search to update the smallest
4     // element
5     // that ends in a subsequence of length d
6     vector<int> v{};
7     v.push_back(s[0]);
8     for (int i{1}; i < s.size(); ++i)
9         if (s[i] > v.back()) v.push_back(s[i]);
10        else *lower_bound(v.begin(), v.end(),
11                          , s[i]) = s[i];
12    return v.size();
13 }
```

1.2 LCS

```
1 // Longest common subsequence
2 int LCS(const vector<int>& a, const vector<
3         int>& b) {
4     vector<vector<int>> dp(a.size() + 1,
5                           vector<int>(b.size() + 1));
6     for (int i{1}; i <= int(a.size()); ++i)
7         for (int j{1}; j <= int(b.size()); ++j) {
8             dp[i][j] = max(dp[i][j], max(dp[i - 1][j], dp[i][j - 1]));
9             if (a[i - 1] == b[j - 1]) dp[i][j] = max(dp[i][j], dp[i - 1][j - 1] + 1);
10        }
11    return dp[a.size()][b.size()];
12 }
```

2 Basic

2.1 mod helper function

```
1 int add(int i, int j) {
2     if ((i += j) >= MOD)
3         i -= MOD;
4     return i;
5 }
6
7 int sub(int i, int j) {
8     if ((i -= j) < 0)
9         i += MOD;
10    return i;
11 }
```

2.2 self-defined-pq-operator

```
1 auto cmp = [](int a, int b) {
2     return a > b;
3 };
4 priority_queue<int, vector<int>, decltype(
5     cmp)> pq(cmp);
```

2.3 generating all subsets

```
1 for (int b = 0; b < (1<<n); b++) {
2     vector<int> subset;
3     for (int i = 0; i < n; i++) {
4         if (b&(1<<i)) subset.push_back(vc[i]);
5     }
6 }
```

2.4 tuple

```
1 int main() {
2     tuple<int, double, string> myTuple(100,
3     3.14, "hello world");
4     tuple<int, double, string> anotherTuple
5     = make_tuple(100, 18, "Tom");
6     cout << get<0>(myTuple) << "\n"; // 取得
7     // 第一個元素
8     auto [a, b, c] = myTuple;
9     get<1>(myTuple) = 5.43; // 修改第二個元
10    // 素的值
11
12     int a;
13     double b;
14     string c;
15     tie(a, b, c) = myTuple;
16 }
```

2.5 memset

```
1 memset(a, 0, sizeof(a)); // 0
2 memset(a, 0xf3f3f3f, sizeof(a)); // INF
```

2.6 submask enumeration

```
1 // O(3^n)
2
3 int m = 0b10110; // binary: 10110, decimal:
4 22
5 cout << "Mask: " << bitset<5>(m) << " (" <<
6     m << ")\\n";
```

```
7 // Enumerate all submasks
8 for (int s = m; s; s = (s - 1) & m) {
9     cout << bitset<5>(s) << " (" << s << "
10     \\n";
11 }
12 // Optionally include the empty submask
13 cout << bitset<5>(0) << " (0)\\n";
```

2.7 custom-hash

```
1 struct custom_hash {
2     static uint64_t splitmix64(uint64_t x) {
3         // <http://xorshift.di.unimi.it/
4         // splitmix64.c>
5         x += 0x9e3779b97f4a7c15;
6         x = (x ^ (x >> 30)) * 0
7             xbf58476d1ce4e5b9;
8         x = (x ^ (x >> 27)) * 0
9             x94d0499bb133111eb;
10        return x ^ (x >> 31);
11 }
12
13 size_t operator()(uint64_t x) const {
14     static const uint64_t FIXED_RANDOM =
15         chrono::steady_clock::now().
16         time_since_epoch().count();
17     return splitmix64(x + FIXED_RANDOM);
18 }
19
20 unordered_map<long long, int, custom_hash>
21 safe_map;
22 gp_hash_table<long long, int, custom_hash>
23 safe_hash_table;
```

2.8 stringstream split by comma

```
1 while (std::getline(ss, segment, ',')) {
2     segments.push_back(segment);
3 }
```

3 DP

3.1 deque

```
1 /*
2 遊戲 DP - O(N^2)
3 A 與 B 將進行以下的遊戲。
4
5 最初，他們會得到一個序列 a = (a1, a2, ...,
6     aN)。在 a 尚未為空時，兩位玩家輪流進行以
7     下操作，從 A 開始：
```

```
7 從 a 的開頭或結尾移除一個元素。玩家會獲得 x
8 分，其中 x 為被移除的元素。
9 設 X 與 Y 分別為遊戲結束時 A 與 B 的總得分。
10 A 會嘗試最大化 X-Y，而 B 會嘗試最小化 X-Y。
11
12 假設兩位玩家都採取最優策略，請求出最後的 X-Y
13 值。
14
15 定義 dp[i][j] 為在區間 [i, j] 上，對於 B 來
16 說的最優分數 (X-Y)。
17
18 */
19 void solve() {
20     int n;
21     cin >> n;
22     vector<int> a(n);
23     vector<vector<int>> dp(n + 1, vector<int>
24         (n + 1, 0));
25
26     for (int i = 0; i < n; ++i) {
27         cin >> a[i];
28         dp[i][i] = a[i];
29     }
30
31     for (int i = n - 1; i >= 0; --i) {
32         for (int j = i + 1; j < n; ++j) {
33             dp[i][j] = max(a[i] - dp[i + 1][j],
34                             dp[i][j] - a[j]);
35         }
36     }
37     cout << dp[0][n - 1] << "\\n";
38 }
```

3.2 walk

```
1 /*
2 DP on graphs - O(N^3 Log K)
3 給定一個簡單的有向圖 G，具有 N 個頂點，編號
4 為 1, 2, ..., N。
5
6 對於任意 i, j (1 ≤ i, j ≤ N)，給定整數 a_{i,j}，
7 表示是否存在從頂點 i 指向頂點 j 的有
8 向邊。若 a_{i,j} = 1，則存在邊；若 a_{i,j} = 0，
9 則不存在。
10
11 求圖中長度為 K 的不同有向路徑數目，對 10^9+7
12 取模。路徑可重複通過相同邊（即允許重複
13 邊）。
14
15 注意：當我們將鄰接矩陣 m 與 m 相乘時，得到的
16 是長度為 2 的路徑數；若取 m 的 p 次方 m^p，
17 則其 (i, j) 元素表示從 i 到 j 的長度
18 為 p 的路徑數。
19
20 */
21 void solve() {
22     int n, k;
23     cin >> n >> k;
```

```

15     vector<vector<int>> m(n, vector<int>(n))
16         ;
17     for (int i = 0; i < n; ++i) {
18         for (int j = 0; j < n; ++j) {
19             cin >> m[i][j];
20         }
21     }
22
23     Matrix<int> mat(m);
24     mat = power(mat, k);
25     int ans = 0;
26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             ans += mat.mat[i][j];
29             ans %= MOD;
30         }
31     }
32     cout << ans << "\n";
33 }

```

3.3 grouping

```

1  /*
2  狀態  $DP - O(3^N * 2^N * N^2)$ 
3  有  $N$  隻兔子，編號為  $1, 2, \dots, N$ 。
4
5  對於每一對  $i, j$  ( $1 \leq i, j \leq N$ )，兔子  $i$  與  $j$  的相容
6  度由整數  $a_{i,j}$  描述。這裡  $a_{i,i} = 0$  對於
7  每個  $i$  ( $1 \leq i \leq N$ )，且  $a_{i,j} = a_{j,i}$  對於任
8  意  $i$  與  $j$  ( $1 \leq i, j \leq N$ )。
9
10 A 將  $N$  隻兔子分成若干個群組。每隻兔子必須且
11 僅屬於一個群組。分群後，對於每一對  $i$  與
12  $j$  ( $1 \leq i < j \leq N$ )，若兔子  $i$  與  $j$  屬於同一群
13 組，A 即可獲得  $a_{i,j}$  分。
14
15 求 A 能獲得的最大總分。
16
17 令  $cost[S]$  表示將集合  $S$  中的所有兔子放在同一
18 群組時所得到的分數。此值可在  $O(2^N * N^2)$  時間內計算。
19
20 接著我們計算  $dp[S]$ ，表示對集合  $S$  中的兔子進
21 行分群時所能得到的最大分數。
22
23 */
24 void solve() {
25     int n;
26     cin >> n;
27     vector<vector<int>> a(n, vector<int>(n))

```

```

28     // backtrack all subset
29     for (int b = 0; b < (1<<n); ++b) {
30         vector<int> subset;
31         for (int i = 0; i < n; ++i) {
32             if (b & (1<<i)) {
33                 for (const int& j : subset)
34                     cost[b] += a[i][j];
35             }
36             subset.pb(i);
37         }
38     }
39
40     // dp
41     for (int i = 0; i < (1<<n); ++i) {
42         int j = ((1<<n) - 1) ^ i;
43         for (int s = j; s != 0; s = (s - 1)
44             & j) {
45             dp[i ^ s] = max(dp[i ^ s], dp[i]
46                 + cost[s]);
47         }
48     }
49     cout << dp[(1<<n) - 1] << "\n";
50 }

```

3.4 matching

```

1  /*
2  Bitmask  $DP - O(N * 2^N)$ 
3  有  $N$  個男人和  $N$  個女人，分別編號為  $1, 2, \dots, N$ 。
4
5  對於每個  $i, j$  ( $1 \leq i, j \leq N$ )，男人  $i$  和女人
6   $j$  的相容性由整數  $a[i][j]$  給出。
7  如果  $a[i][j] = 1$ ，則男人  $i$  和女人  $j$  是相容
8  的；
9  如果  $a[i][j] = 0$ ，則不是。
10
11 A 正在嘗試組成  $N$  對，每對由一個相容的男人和
12 女人組成。在這裡，每個男人和每個女人必須
13 恰好屬於一對。
14
15 求 A 可以組成  $N$  對的方法數，結果對  $10^9 + 7$ 
16 取模。
17
18 定義  $dp[S]$  為將集合  $S$  中的女性與前  $|S|$  個男
19 性配對的方法數。
20
21 */
22 const int maxn = 21;
23 const int MOD = 1e9 + 7;
24 int n;
25 int grid[maxn][maxn];
26 int dp[1<<maxn];
27
28 void solve() {
29     cin >> n;
30     memset(dp, 0, sizeof(dp));
31 }

```

```

26     for (int i = 0; i < n; ++i) {
27         for (int j = 0; j < n; ++j) {
28             cin >> grid[i][j];
29         }
30     }
31
32     dp[0] = 1;
33     for (int s = 0; s < (1<<n); ++s) {
34         int ps = __builtin_popcount(s);
35         for (int w = 0; w < n; ++w) {
36             if ((s & (1<<w)) || !grid[ps][w]) {
37                 continue;
38             }
39
40             dp[s | (1<<w)] += dp[s];
41             dp[s | (1<<w)] %= MOD;
42         }
43     }
44     cout << dp[(1<<n) - 1] << "\n";
45 }

```

3.5 projects

```

1  /*
2  LIS  $DP - O(N \log N)$ 
3  有  $n$  個你可以參加的專案。對於每個專案，你知
4  道其開始與結束天數以及可獲得的報酬金額。
5  在同一天你最多只能參加一個專案。
6  問：你最多可以賺到多少金額？
7
8   $dp[i]$  = 在第  $i$  天之前我們可以賺到的最大金
9  額。
10
11 */
12 void solve() {
13     int n;
14     cin >> n;
15     vector<array<int, 3>> vc(n);
16     map<int, int> days;
17
18     for (int i = 0; i < n; ++i) {
19         int a, b, p;
20         cin >> a >> b >> p;
21         days[a] = days[b] = 1;
22         vc[i] = {a, b, p};
23     }
24     int idx = 1;
25     for (auto& x : days) {
26         x.second = idx++;
27     }
28     vector<int> dp(idx, 0);
29
30     sort(vc.begin(), vc.end(), [](const
31         array<int, 3>& va, const array<int,
32         3>& vb) {
33         if (va[1] != vb[1]) return va[1] <
34             vb[1];
35         if (va[0] != vb[0]) return va[0] <
36             vb[0];
37         return va[2] > vb[2];
38     });

```

```

33     int i = 0;
34     for (int d = 1; d < idx; ++d) {
35         dp[d] = dp[d - 1];
36         while (i < n && days[vc[i][1]] == d)
37             {
38                 dp[d] = max(dp[d], dp[days[vc[i]
39                     ][0]] - 1] + vc[i][2]);
40                 i++;
41             }
42     }
43     cout << dp[idx - 1] << "\n";

```

3.6 stones

```

1  /*
2  遊戲  $DP - O(N^2)$ 
3  有一個集合  $A = \{a_1, a_2, \dots, a_N\}$ ，包含  $N$  個正整
4  數。太郎和次郎將進行以下的遊戲。
5
6  一開始有一堆  $K$  顆石頭。兩位玩家輪流進行以下
7  操作，從太郎開始：
8
9  選擇集合  $A$  中的一個元素  $x$ ，並從石堆中移除恰
10 好  $x$  顆石頭。
11 當某位玩家無法進行操作時即輸掉比賽。假設兩位
12 玩家都採取最優策略，請判斷誰會獲勝。
13
14 定義  $dp[i]$  表示當剩下  $i$  顆石頭時，是否有可能
15 獲勝。
16
17 */
18 void solve() {
19     int n, k;
20     cin >> n >> k;
21     vector<int> a(n);
22     vector<bool> dp(k + 1, 0);
23
24     for (int i = 0; i < n; ++i) {
25         cin >> a[i];
26     }
27
28     for (int i = 1; i <= k; ++i) {
29         for (int x : a) {
30             if (i >= x && !dp[i - x]) {
31                 dp[i] = 1;
32             }
33         }
34     }
35     cout << (dp[k] ? "First" : "Second") <<
36         "\n";
37 }

```

3.7 coins

```

1  /*
2  機率  $DP - O(N^2)$ 
3  給定一個正奇數  $N$ 

```

```

4 有  $N$  枚編號為  $1, 2, \dots, N$  的硬幣，第  $i$  枚出現正
   面的機率為  $p$ ，反面為  $1-p$ 。
5 已經拋擲所有硬幣，求正面數大於反面的機率。
6
7 定義  $dp[i][j]$  為拋完前  $i$  枚硬幣後，得到  $j$  次
   正面的機率。
8
9
10 void solve() {
11     int n;
12     cin >> n;
13     vector<double> a(n);
14     vector<vector<double>> dp(n + 1, vector<
15         double>(n + 1, 0.0));
16
17     for (int i = 0; i < n; ++i) {
18         cin >> a[i];
19     }
20
21     for (int i = 0; i <= n; ++i) {
22         dp[i][0] = 1.0;
23     }
24
25     int least = n / 2 + 1;
26
27     for (int i = 1; i <= n; ++i) {
28         for (int j = 1; j <= least; ++j) {
29             // head
30             dp[i][j] = dp[i - 1][j - 1] * a[
31                 i - 1];
32             // tail
33             dp[i][j] += dp[i - 1][j] * (1 -
34                 a[i - 1]);
35         }
36     }
37
38     cout << fixed << setprecision(10) << dp[
39         n][least] << "\n";
40 }

```

3.8 elevator rides

```

1 /*
2 狀態  $DP - O(2^N)$ 
3 有  $n$  個人想要搭電梯到樓頂，建築物只有一部電
   梯。你知道每個人的體重以及電梯的最大允許
   載重。最少需要搭乘多少次電梯？
4
5 定義  $dp[S] = \{r, w\}$ ，其中  $r$  是將集合  $S$  中的
   所有人送到樓頂所需的最少電梯次數， $w$  是最
   後一次電梯所載人的總重量。
6
7
8 void solve() {
9     int n, x;
10    cin >> n >> x;
11    vector<int> w(n);
12    vector<pii> dp(1<n, {INF, INF});
13
14    for (int i = 0; i < n; ++i) {
15        cin >> w[i];

```

```

16    }
17
18    dp[0] = {1, 0};
19    for (int b = 1; b < (1<n); ++b) {
20        for (int i = 0; i < n; ++i) {
21            if (b & (1<i)) {
22                auto [r_prev, w_prev] = dp[b
23                    ^ (1<i)];
24                pii can;
25                if (w_prev + w[i] <= x) {
26                    can = {r_prev, w_prev +
27                        w[i]};
28                }
29                else {
30                    can = {r_prev + 1, w[i
31                        ]};
32                }
33                dp[b] = min(dp[b], can);
34            }
35        }
36    }
37
38    cout << dp[(1<n) - 1].first << "\n";
39 }

```

3.9 slimes

```

1 /*
2 Range  $DP - O(N^3)$ 
3 有  $N$  個史萊姆排成一列。最初，從左邊數來第  $i$ 
   個史萊姆的大小為  $a_i$ 。
4
5 A 想要把所有史萊姆合併成一個更大的史萊姆。他
   會重複執行以下操作，直到只剩下一個史萊姆
   為止：
6
7 選擇兩個相鄰的史萊姆，將它們合併成一個新的史
   萊姆。新史萊姆的大小為  $x+y$ ，其中  $x$  和  $y$ 
   是合併前兩個史萊姆的大小。
8 這時會產生  $x+y$  的花費。合併時，史萊姆的相對
   位置不會改變。
9 請求出合併所有史萊姆所需的最小總花費。
10
11 令  $dp[i][j]$  表示將第  $i$  個到第  $j$  個史萊姆合併
   成一個史萊姆的最小花費。
12
13
14 const int maxn = 401;
15 const int INF = 1e18;
16 int dp[maxn][maxn];
17 int a[maxn];
18 int prefix[maxn + 1];
19
20 int f(int i, int j) {
21     if (i + 1 == j) {
22         return a[i] + a[j];
23     }
24     if (i == j) {
25         return 0;
26     }
27     if (dp[i][j] != INF) {
28         return dp[i][j];

```

```

29    }
30    //cerr << i << " " << j << "\n";
31
32    int ans = INF;
33    for (int k = i; k < j; ++k) {
34        ans = min(ans, f(i, k) + f(k + 1, j)
35            );
36    }
37    return dp[i][j] = ans + (prefix[j + 1] -
38        prefix[i]);
39 }

```

3.10 digit sum

```

1 /*
2 Digit  $DP - O(|K| * D)$ 
3 計算在  $1$  到  $K$  (含) 之間，滿足其十進位數字和
   為  $D$  的倍數的整數數量，答案對  $10^9+7$  取
   模。
4
5 令  $dp[i][j]$  表示在已確定前  $i$  位數字的情況
   下，構成長度為  $|K|$  的數字且目前數字和
   mod  $D$  等於  $j$  的方法數。
6
7
8 const int MOD = 1e9 + 7;
9 int dp[10001][101][2];
10
11 void solve() {
12     string K;
13     int D;
14     cin >> K >> D;
15     int len = K.size();
16     memset(dp, 0, sizeof(dp));
17     dp[0][0][1] = 1;
18
19     for (int i = 1; i <= len; ++i) {
20         int limit = K[i - 1] - '0';
21         for (int s = 0; s < D; ++s) {
22             for (int flag = 0; flag <= 1; ++
23                 flag) {
24                 int ways = dp[i - 1][s][flag
25                     ];
26                 if (ways == 0) continue;
27                 int max_d = (flag ? limit :
28                     9);
29                 for (int d = 0; d <= max_d;
30                     ++d) {
31                     int rs = (s + d) % D;
32                     int rflag = (flag && d
33                         == max_d ? 1 : 0);
34                     dp[i][rs][rflag] += ways
35                         ;
36                     dp[i][rs][rflag] %= MOD;
37                 }
38             }
39         }
40     }
41
42     int ans = (dp[len][0][0] + dp[len
43         ][0][1]) % MOD;
44     ans = (ans - 1 + MOD) % MOD;
45 }

```

```

38     cout << ans << "\n";
39 }

```

3.11 sushi

```

1 /*
2 期望值  $DP - O(N^3)$ 
3 有  $N$  盤壽司，編號從  $1$  到  $N$ 。第  $i$  盤最初有  $a_i$ 
   ( $1 \leq a_i \leq 3$ ) 塊壽司。
4
5 太郎不斷擲一顆編號  $1$  到  $N$  的骰子。如果結果是
   第  $i$  盤且該盤還有壽司，他就吃掉一塊；否
   則什麼也不做。
6
7 請求出吃完所有壽司所需擲骰子的期望次數。
8
9  $dp[x][y][z]$  代表還有
10  $x$  盤剩 1 塊壽司、
11  $y$  盤剩 2 塊壽司、
12  $z$  盤剩 3 塊壽司時的期望擲骰次數。
13
14
15 const int maxn = 301;
16 double dp[maxn][maxn][maxn];
17 int n;
18
19 double dfs(int x, int y, int z) {
20     if (x < 0 || y < 0 || z < 0) return 0;
21     if (x == 0 && y == 0 && z == 0) return
22         0;
23     if (dp[x][y][z] > 0) return dp[x][y][z];
24     double ans = n + x * dfs(x - 1, y, z)
25         + y * dfs(x + 1, y - 1, z
26             )
27         + z * dfs(x, y + 1, z -
28             1);
29     return dp[x][y][z] = ans / (x + y + z);
30 }
31
32 void solve() {
33     cin >> n;
34     vector<int> a(n);
35     memset(dp, -1, sizeof(dp));
36     vector<int> freq(4, 0);
37
38     for (int i = 0; i < n; ++i) {
39         cin >> a[i];
40         freq[a[i]]++;
41     }
42
43     cout << fixed << setprecision(10) << dfs
44         (freq[1], freq[2], freq[3]) << "\n";
45 }

```

3.12 candies

```

1 /*
2 組合  $DP - O(NK)$ 

```

```

3 有  $N$  個小孩，編號為  $1, 2, \dots, N$ 。
4
5 他們決定將  $K$  顆糖果分給自己。對於每個  $i$  ( $1 \leq i \leq N$ )，第  $i$  個小孩最多可以拿到  $a_i$  顆糖果（包含  $0$  顆）。所有糖果都必須分完，不能剩下。
6
7 請問有多少種分配糖果的方法？請將答案對  $10^9+7$  取模。若存在某個小孩分到的糖果數不同，則視為不同的分配方式。
8
9 令  $dp[i][j]$  表示將  $j$  顆糖果分給前  $i$  個小孩的方法數。
10
11
12 void solve() {
13     int n, k;
14     cin >> n >> k;
15     vector<int> a(n);
16     vector<int> dp(k + 1, 0), S(k + 1, 0);
17
18     for (int i = 0; i < n; ++i) {
19         cin >> a[i];
20     }
21
22     dp[0] = 1;
23     for (int i = 0; i < n; ++i) {
24         vector<int> new_dp(k + 1, 0);
25         S[0] = dp[0];
26         for (int j = 1; j <= k; ++j) {
27             S[j] = (S[j - 1] + dp[j]) % MOD;
28         }
29         for (int j = 0; j <= k; ++j) {
30             if (j - a[i] - 1 >= 0) {
31                 new_dp[j] = (S[j] - S[j - a[i] - 1] + MOD) % MOD;
32             }
33             else {
34                 new_dp[j] = S[j] % MOD;
35             }
36         }
37         dp = new_dp;
38     }
39     cout << dp[k] << "\n";
40 }

```

3.13 permutation

```

1  /*
2  抽象 DP -  $O(N^2)$ 
3  設  $N$  為正整數。給定一個長度為  $N-1$  的字串  $s$ ，字元僅包含 ' $<$ ' 與 ' $>$ '。
4
5  求滿足條件的排列 ( $p_1, p_2, \dots, p_N$ ) (即  $1$  到  $N$  的排列) 數量。答案對  $10^9+7$  取模：
6
7  對於每個  $i$  ( $1 \leq i \leq N-1$ )，若  $s$  的第  $i$  個字元為 ' $<$ '，則要求  $p_i < p_{i+1}$ ；若為 ' $>$ '，則要求  $p_i > p_{i+1}$ 。
8

```

```

9  令  $dp[i][j]$  表示：在考慮前  $i$  個比較符號（即構成長度為  $i+1$  的排列）且最後一個元素為  $j$  的有效排列數量。
10
11
12 void solve() {
13     int n;
14     string s;
15     cin >> n >> s;
16     vector<vector<int>> dp(n + 1, vector<int>(n + 1, 0));
17     vector<int> prefix(n + 1, 0);
18     dp[1][0] = 1;
19
20     for (int i = 2; i <= n; ++i) {
21         for (int k = 0; k < n; ++k) {
22             prefix[k + 1] = prefix[k] + dp[i - 1][k];
23         }
24         for (int j = 0; j < i; ++j) {
25             if (s[i - 2] == '>') {
26                 dp[i][j] += prefix[i - 1] - prefix[j];
27                 dp[i][j] %= MOD;
28                 /*
29                 for (int k = j; k < i - 1; ++k) {
30                     dp[i][j] += dp[i - 1][k];
31                 }
32                 */
33             }
34             else {
35                 dp[i][j] += prefix[j];
36                 dp[i][j] %= MOD;
37                 /*
38                 for (int k = 0; k < j; ++k) {
39                     dp[i][j] += dp[i - 1][k];
40                 }
41                 */
42             }
43         }
44     }
45     int ans = 0;
46     for (int j = 0; j < n; ++j) {
47         ans += dp[n][j];
48         ans %= MOD;
49     }
50     cout << ans << "\n";
51 }

```

3.14 Knasack2

```

1  // 01 背包，背包承重大 ( $1e9$ )，物品價值和較小 ( $1e5$ )
2
3  const int maxn = 101;
4  const int maxv = 100001;
5  int weight[maxn];
6  int cost[maxn];

```

```

7  int dp[maxv];
8
9  void solve() {
10     int n, w;
11     cin >> n >> w;
12
13     for (int i = 0; i < n; ++i) {
14         cin >> weight[i] >> cost[i];
15     }
16     fill(dp, dp + maxv, 1e18);
17
18     dp[0] = 0;
19     for (int i = 0; i < n; ++i) {
20         for (int j = maxv - 1; j >= 0; --j) {
21             if (dp[j] + weight[i] <= w) {
22                 dp[j + cost[i]] = min(dp[j + cost[i]], dp[j] + weight[i]);
23             }
24         }
25     }
26
27     for (int i = maxv - 1; i >= 0; --i) {
28         if (dp[i] != 1e18) {
29             cout << i << "\n";
30             return;
31         }
32     }
33 }

```

3.15 flowers

```

1  /*
2  LIS DP + Segment Tree -  $O(N \log N)$ 
3  有  $N$  朵花排成一列。對於每個  $i$  ( $1 \leq i \leq N$ )，第  $i$  朵花的高度與美麗分別為  $h_i$  與  $a_i$ 。此處  $h_1, h_2, \dots, h_N$  兩兩互異。
4
5  A 會拔掉一些花，使得剩下的花從左到右的高度為單調遞增（嚴格遞增）。
6
7  求剩下花的美麗值總和的最大可能值。
8
9  令  $dp[i]$  表示以第  $i$  朵花為結尾的遞增子序列所能取得的最大美麗值。
10
11
12 void solve() {
13     int n;
14     cin >> n;
15     SGT<int, MergeMax> tree(n + 1, 0);
16     vector<int> h(n), b(n);
17
18     for (int i = 0; i < n; ++i) {
19         cin >> h[i];
20     }
21     for (int i = 0; i < n; ++i) {
22         cin >> b[i];
23     }
24
25     for (int i = 0; i < n; ++i) {

```

```

26         int mx = tree.query(0, h[i]);
27         tree.modify(h[i], mx + b[i]);
28     }
29
30     cout << tree.query(0, n + 1) << "\n";
31 }

```

3.16 independent set

```

1  /*
2  DP on Trees -  $O(N)$ 
3  有一棵含  $N$  個頂點的樹，頂點編號為  $1, 2, \dots, N$ 。對於每個  $i$  ( $1 \leq i \leq N-1$ )，第  $i$  條邊連接頂點  $x_i$  和  $y_i$ 。
4
5  A 決定將每個頂點塗成白色或黑色，但不允許兩個相鄰的頂點同時為黑色。
6
7  求將頂點塗色的方案數，對  $10^9+7$  取模。
8
9  設  $dp[i][j]$  表示以節點  $i$  為根的子樹中，在節點  $i$  顏色為  $j$  時的塗色方案數（例如  $j=0$  表示白色， $j=1$  表示黑色）。
10
11
12 const int maxn = 100001;
13 vector<int> adj[maxn];
14 int f[maxn][2];
15 const int MOD = 1e9 + 7;
16
17 void dp(int u, int p) {
18
19     for (int v : adj[u]) {
20         if (v != p) {
21             dp(v, u);
22             f[u][0] = (f[v][0] + f[v][1]) % MOD * f[u][0] % MOD;
23             f[u][1] = f[v][0] * f[u][1] % MOD;
24         }
25     }
26 }
27
28 void solve() {
29     int n;
30     cin >> n;
31
32     for (int i = 0; i < n; ++i) {
33         f[i][0] = f[i][1] = 1;
34     }
35
36     for (int i = 0; i < n - 1; ++i) {
37         int u, v;
38         cin >> u >> v;
39         u--, v--;
40         adj[u].pb(v);
41         adj[v].pb(u);
42     }
43
44     dp(0, -1);
45 }

```

```

46     cout << (f[0][0] + f[0][1]) % MOD << "\n";
47 }

```

3.17 counting tower

```

1  /*
2  狀態機 DP - O(N)
3  你的任務是建造一座寬度為 2、高度為 n 的塔。
4  你有無限數量寬度與高度為整數的方塊。
5
6  dp[i][0] = 高度為 i 的塔中，頂層為一個寬度為
7  2 的方塊（即該層由跨越兩欄的單一方塊覆
8  蓋）的塔的數量。
9  dp[i][1] = 高度為 i 的塔中，頂層在該層有兩個
10 寬度為 1 的方塊（每欄各一個）的塔的數
11量。
12
13 */
14 void solve() {
15     long long n;
16     cin >> n;
17     dp[1][0] = 1;
18     dp[1][1] = 1;
19     for(int i = 2; i <= n; i++){
20         dp[i][0] = (4 * dp[i-1][0] + dp[i-1][1]) % mod;
21         dp[i][1] = (dp[i-1][0] + 2 * dp[i-1][1]) % mod;
22     }
23     cout << (dp[n][0] + dp[n][1]) % mod << endl;
24 }

```

3.18 counting numbers

```

1  /*
2  區間 DP - O(digits*10)
3  您的任務是計算在區間 a 到 b 之間，沒有任何相
4  鄰兩位數字相同的整數的個數。
5
6  定義 dp[pos][prev_digit][is_tight][is_started]
7  為從位置 pos 到結尾的有效整
8  數數量。
9  其中 prev_digit 是位置 pos-1 的數字。
10 is_tight 表示是否受原數字前綴的限制。
11 is_started 表示是否已開始構成有效數字（用以
12 避免計入前導零）。
13
14 */
15 // dp[pos][prev_digit][is_tight][is_started]
16 int dp[20][10][2][2];
17 string s;
18
19 int f(int pos, int prev_digit, bool is_tight, bool is_started) {
20     if (pos == (int)s.size()) {
21         return 1;
22     }
23     if (dp[pos][prev_digit][is_tight][is_started] != -1) {
24         return dp[pos][prev_digit][is_tight][is_started];
25     }
26     int ans = 0;
27     int max_d = is_tight ? (s[pos] - '0') : 9;
28     for (int d = 0; d <= max_d; ++d) {
29         if (is_started || d != 0) {
30             continue;
31         }
32         bool new_is_started = is_started || (d > 0);
33         bool new_is_tight = is_tight && (d == max_d);
34         ans += f(pos + 1, d, new_is_tight, new_is_started);
35     }
36     return dp[pos][prev_digit][is_tight][is_started] = ans;
37 }
38
39 int count(int x) {
40     if (x < 0) return 0;
41     s = to_string(x);
42     memset(dp, -1, sizeof(dp));
43     return f(0, 0, true, false);
44 }
45
46 void solve() {
47     int a, b;
48     cin >> a >> b;
49
50     int ca = count(a - 1);
51     int cb = count(b);
52     cout << cb - ca << "\n";
53 }

```

```

14     *k=v;
15 }
16 void save() { sp.PB(SZ(h)); }
17 void undo() {
18     assert(!sp.empty());
19     int last=sp.back(); sp.pop_back();
20     while (SZ(h)!=last) {
21         auto x=h.back(); h.pop_back();
22         *x.F=x.S;
23     }
24 }
25 int f(int x) {
26     while (fa[x]!=x) x=fa[x];
27     return x;
28 }
29 void uni(int x, int y) {
30     x=f(x); y=f(y);
31     if (x==y) return;
32     if (sz[x]<sz[y]) swap(x, y);
33     assign(&sz[x], sz[x]+sz[y]);
34     assign(&fa[y], x);
35 }
36 }djs;

```

```

29     t[tv] = merge(t[left(tv)], t[
30         right(tv, tl, tm)];
31     } else t[tv] = {v[tl], tl};
32 }
33 void push(int tv, int tl, int tr) { //
34     lazy propagation
35     if (!lz[tv]) return;
36     int tm{(tl + tr) / 2};
37     update(left(tv), tm - tl, lz[tv].
38         value());
39     update(right(tv, tl, tm), tr - tm,
40         lz[tv].value());
41     lz[tv].reset();
42 }
43 void set(int p, const value_t& x, int tv,
44     int tl, int tr) {
45     if (tr - tl > 1) {
46         push(tv, tl, tr);
47         int tm{(tl + tr) / 2};
48         if (p < tm) set(p, x, left(tv),
49             tl, tm);
50         else set(p, x, right(tv, tl, tm),
51             tm, tr);
52         t[tv] = merge(t[left(tv)], t[
53             right(tv, tl, tm)]);
54     } else t[tv].fi = x;
55 }
56 void rmodify(int l, int r, const value_t
57     & x, int tv, int tl, int tr) {
58     if (!(l == tl && r == tr)) {
59         push(tv, tl, tr);
60         int tm{(tl + tr) / 2};
61         if (r <= tm) rmodify(l, r, x,
62             left(tv), tl, tm);
63         else if (l >= tm) rmodify(l, r,
64             x, right(tv, tl, tm), tm, tr);
65         else rmodify(l, tm, x, left(tv),
66             tl, tm),
67             rmodify(tm, r, x, right(tv,
68                 tl, tm), tm, tr);
69         t[tv] = merge(t[left(tv)], t[
70             right(tv, tl, tm)]);
71     } else update(tv, tr - tl, x);
72 }
73 node_t rquery(int l, int r, int tv, int
74     tl, int tr) {
75     if (l == tl && r == tr) return t[tv];
76     push(tv, tl, tr);
77     int tm{(tl + tr) / 2};
78     if (r <= tm) return rquery(l, r,
79         left(tv), tl, tm);
80     else if (l >= tm) return rquery(l, r,
81         right(tv, tl, tm), tm, tr);
82     else return merge(rquery(l, tm, left
83         (tv), tl, tm),
84         rquery(tm, r, right(tv, tl, tm),
85             tm, tr));
86 }
87
88 public:
89     explicit SGT(const vector<value_t>& v) :
90         n{v.size()}, t{2 * n - 1}, lz{2 * n -
91             1} { build(v, 0, 0, n); }
92     void set(int p, const value_t& x) { set(
93         p, x, 0, 0, n); }

```

4.2 segment tree range update (lazy propagation)

```

1  // segment tree
2  // range query & range modify
3  class SGT {
4      using value_t = int;
5      using node_t = pair<value_t, int>;
6
7      int n;
8      vector<node_t> t;
9      vector<optional<value_t>> lz;
10     // [ tv+1 : tv+2*(tm-tl) ) -> Left
11     subtree
12     int left(int tv) { return tv + 1; }
13     int right(int tv, int tl, int tm) {
14         return tv + 2 * (tm - tl); }
15
16     /** differ from case to case */
17     // query is "max" and modify is "add"
18     here
19     node_t merge(const node_t& x, const
20         node_t& y) { // associative function
21         return max(x, y);
22     }
23
24     void update(int tv, int len, const
25         value_t& x) {
26         if (!lz[tv]) lz[tv] = x;
27         else lz[tv] = lz[tv].value() + x;
28         t[tv].fi = t[tv].fi + x;
29     }
30     /*****
31     void build(const vector<value_t>& v, int
32         tv, int tl, int tr) {
33         if (tr - tl > 1) {
34             int tm{(tl + tr) / 2};
35             build(v, left(tv), tl, tm);
36             build(v, right(tv, tl, tm), tm,
37                 tr);
38         }
39     }

```

4 Data Structure

4.1 undo disjoint set

```

1  struct DisjointSet {
2      // save() is like recursive
3      // undo() is like return
4      int n, fa[MXN], sz[MXN];
5      vector<pair<int*,int>> h;
6      vector<int> sp;
7      void init(int tn) {
8          n=tn;
9          for (int i=0; i<n; i++) sz[fa[i]=i]=1;
10         sp.clear(); h.clear();
11     }
12     void assign(int *k, int v) {
13         h.PB({k, *k});
14     }
15 }

```



```

71 void rmodify(int l, int r, const value_t
    & x) { rmodify(l, r, x, 0, 0, n); }
72 // [l:r)
    node_t rquery(int l, int r) { return
        rquery(l, r, 0, 0, n); } // [l:r)
73 };
74
75 int main() {
76     vector<long long> a = {1, 5, 2, 4, 3};
77     SGT st(a);
78
79     auto [val, idx] = st.rquery(0, 5);
80     cout << "Initial max: " << val << " at "
81         << idx << "\\n"; // (5, 1)
82
83     st.rmodify(1, 4, 3); // add 3 to indices
84     1..3
85     tie(val, idx) = st.rquery(0, 5);
86     cout << "After add: " << val << " at "
87         << idx << "\\n"; // (8, 1)
88
89     st.set(2, 10); // set a[2] = 10
90     tie(val, idx) = st.rquery(0, 5);
91     cout << "After set: " << val << " at "
92         << idx << "\\n"; // (10, 2)
93 }

```

4.3 lazy propagation

```

1 // Lazy Propagation Segment Tree supporting
  range add, range set, and range sum
  queries
2
3 int n;
4 int sz;
5 struct Node {
6     int sum = 0;
7     int lazy_add = 0;
8     int lazy_set = 0;
9     bool has_set = false;
10 };
11 vector<Node> tree;
12
13 void build(const vector<int>& a) {
14     tree.resize(sz << 1);
15     for (int i = 0; i < sz; ++i) {
16         tree[i + sz].sum = i < n ? a[i] : 0;
17     }
18     for (int i = sz - 1; i >= 1; --i) {
19         tree[i].sum = tree[i << 1].sum +
20             tree[i << 1 | 1].sum;
21     }
22
23 void apply_set(int idx, int l, int r, int
    val) {
24     tree[idx].sum = (r - l + 1) * val;
25     tree[idx].lazy_set = val;
26     tree[idx].lazy_add = 0;
27     tree[idx].has_set = true;
28 }
29

```

```

30 void apply_add(int idx, int l, int r, int
    val) {
31     if (tree[idx].has_set) {
32         tree[idx].lazy_set += val;
33     }
34     else {
35         tree[idx].lazy_add += val;
36     }
37     tree[idx].sum += (r - l + 1) * val;
38 }
39
40 void push(int idx, int l, int r) {
41     if (l == r) return;
42
43     int mid = l + (r - l) / 2;
44     int left = idx << 1, right = idx << 1 |
        1;
45
46     if (tree[idx].has_set) {
47         // propagate the set operation
48         apply_set(left, l, mid, tree[idx].
            lazy_set);
49         apply_set(right, mid + 1, r, tree[
            idx].lazy_set);
50         tree[idx].has_set = false;
51         tree[idx].lazy_set = 0;
52         tree[idx].lazy_add = 0;
53     }
54     else if (tree[idx].lazy_add != 0) {
55         // propagate the add operation
56         apply_add(left, l, mid, tree[idx].
            lazy_add);
57         apply_add(right, mid + 1, r, tree[
            idx].lazy_add);
58         tree[idx].lazy_add = 0;
59     }
60 }
61
62 void range_set(int idx, int l, int r, int ql
    , int qr, int val) {
63     if (qr < l || ql > r) return;
64     if (ql <= l && r <= qr) {
65         apply_set(idx, l, r, val);
66         return;
67     }
68
69     push(idx, l, r);
70     int mid = (l + r) / 2;
71     range_set(idx << 1, l, mid, ql, qr, val)
72     ;
73     range_set(idx << 1 | 1, mid + 1, r, ql,
        qr, val);
74     tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
75 }
76
77 void range_add(int idx, int l, int r, int ql
    , int qr, int val) {
78     if (ql > r || qr < l) return;
79     if (ql <= l && r <= qr) {
80         apply_add(idx, l, r, val);
81         return;
82     }
83
84     push(idx, l, r);
85     int mid = (l + r) / 2;

```

```

85     range_add(idx << 1, l, mid, ql, qr, val)
86     ;
87     range_add(idx << 1 | 1, mid + 1, r, ql,
        qr, val);
88     tree[idx].sum = tree[idx << 1].sum +
        tree[idx << 1 | 1].sum;
89 }
90
91 int range_sum(int idx, int l, int r, int ql,
    int qr) {
92     if (qr < l || ql > r) return 0;
93     if (ql <= l && r <= qr) return tree[idx
        ].sum;
94
95     push(idx, l, r);
96     int mid = (l + r) / 2;
97     return range_sum(idx << 1, l, mid, ql,
        qr) +
98         range_sum(idx << 1 | 1, mid + 1,
            r, ql, qr);
99 }
100
101 void solve() {
102     int q;
103     cin >> n >> q;
104     vector<int> a(n);
105
106     sz = 1;
107     while (sz < n) sz <= 1;
108     for (int i = 0; i < n; ++i) {
109         cin >> a[i];
110     }
111
112     build(a);
113
114     while (q--) {
115         int mode, l, r, x;
116         cin >> mode >> l >> r;
117         l--, r--;
118         if (mode == 1) {
119             range_add(1, 0, sz - 1, l, r, x)
120             ;
121         }
122         else if (mode == 2) {
123             cin >> x;
124             range_set(1, 0, sz - 1, l, r, x)
125             ;
126         }
127         else {
128             cout << range_sum(1, 0, sz - 1,
                l, r) << "\\n";
129         }
130     }
131 }

```

4.4 Lazy heap

```

1 template<typename T, typename Compare = less
    <T>>
2 struct lp {
3     priority_queue<T, vector<T>, Compare> pq
        ;

```

```

4 unordered_map<T, int> cnt;
5 size_t sz = 0;
6 void real_remove() {
7     while (!pq.empty() && cnt[pq.top()] >
        0) {
8         cnt[pq.top()]--;
9         pq.pop();
10    }
11 }
12 size_t size() {
13     return sz;
14 }
15 void remove(T n) {
16     cnt[n]++;
17     sz--;
18 }
19 T top() {
20     real_remove();
21     return pq.top();
22 }
23 void push(T n) {
24     pq.push(n);
25     sz++;
26 }
27 void pop() {
28     real_remove();
29     sz--;
30     pq.pop();
31 }
32 };

```

4.5 segment tree prefix sum lower bound

```

1 class SGT {
2     int n;
3     vector<long long> t;
4     int left(int tv) { return tv + 1; }
5     int right(int tv, int tl, int tm) {
6         return tv + 2 * (tm - tl); }
7     void modify(int p, long long x, int tv,
        int tl, int tr) {
8         if (tr - tl > 1) {
9             int tm((tl + tr) / 2);
10            if (p < tm) modify(p, x, left(tv),
                tl, tm);
11            else modify(p, x, right(tv, tl,
                tm), tm, tr);
12            t[tv] = t[left(tv)] + t[right(tv),
                tl, tm];
13        } else t[tv] = x;
14    }
15
16    long long query(int l, int r, int tv,
        int tl, int tr) {
17        if (l == tl && r == tr) return t[tv]
18        ;
19        int tm((tl + tr) / 2);
20        if (r <= tm) return query(l, r, left
            (tv), tl, tm);
21        else if (l >= tm) return query(l, r,
            right(tv, tl, tm), tm, tr);
22    }

```

```

19     else return query(1, tm, left(tv),
20         t1, tm) +
21         query(tm, r, right(tv, t1, tm),
22             tm, tr);
23 }
24 public:
25 explicit SGT(int _n) : n{_n}, t(2 * n -
26     1) {}
27 void modify(int p, long long x) { modify
28     (p, x, 0, 0, n); }
29 long long query(int l, int r) { return
30     query(l, r, 0, 0, n); }
31 int ps_lower_bound(long long ps) { //
32     prefix sum lower bound
33     if (ps > t[0]) return n;
34     int tv{0}, tl{0}, tr{n};
35     while (tr - tl > 1) {
36         int tm{(tl + tr) / 2};
37         if (t[left(tv)] >= ps) tv = left
38             (tv), tr = tm;
39         else ps -= t[left(tv)], tv =
40             right(tv, tl, tm), tl = tm;
41     }
42     return tl;
43 }
44 };

```

4.6 Fenwick tree (BIT)

```

1 struct Fenwick {
2     int n;
3     vector<int> bit, prefix;
4     Fenwick(int _n) : n(_n), bit(n + 1, 0),
5         prefix(n + 1, 0) {}
6     void update(int i, int delta) {
7         for (; i <= n; i += i & -i) {
8             bit[i] += delta;
9         }
10    }
11    int query(int i) {
12        int ans = 0;
13        for (; i >= 1; i -= i & -i) {
14            ans += bit[i];
15        }
16        return ans;
17    }
18    int range(int l, int r) {
19        return query(r) - query(l - 1);
20    }
21 };
22 int main() {
23     int n = 5;
24     vector<int> a = {0, 1, 2, 3, 4, 5}; //
25     1-indexed
26     Fenwick ft(n);
27     for (int i = 1; i <= n; ++i) ft.update(i
28         , a[i]);
29     cout << ft.query(3) << "\n"; // 6
30     cout << ft.range(2, 4) << "\n"; // 9

```

```

31     ft.update(3, 10); //
32     a[3] += 10
33     cout << ft.query(3) << "\n"; // 16
34     cout << ft.range(1, 5) << "\n"; // 25

```

4.7 BIT - range update + range prefix sum

```

1 int n, q;
2
3 void solve() {
4     cin >> n >> q;
5     Fenwick bit_values(n), bit_count(n);
6
7     while (q--) {
8         int mode, l, r, val;
9         cin >> mode;
10        if (mode == 0) {
11            cin >> l >> r >> val;
12            bit_values.update(l, val);
13            bit_count.update(l, val * (1 -
14                1));
15            bit_values.update(r + 1, -val);
16            bit_count.update(r + 1, -val * r
17                );
18        }
19        else {
20            cin >> l >> r;
21            int pref_l = bit_values.query(l
22                - 1) * (1 - 1) - bit_count.
23                query(l - 1);
24            int pref_r = bit_values.query(r)
25                * r - bit_count.query(r);
26            cout << pref_r - pref_l << "\n";
27        }
28    }
29 }

```

4.8 Order Statistics Tree

```

1 #include <ext/pb_ds/assoc_container.hpp>
2 #include <ext/pb_ds/tree_policy.hpp>
3 using namespace __gnu_pbds;
4
5 template <class T>
6 using ordered_set =
7     tree<T, null_type, less<T>, rb_tree_tag,
8         tree_order_statistics_node_update>;
9
10 int main() {
11     int n;
12     cin >> n;
13     ordered_set<int> st;
14     vector<int> p(n);
15     for (int i = 0; i < n; ++i) {
16         st.insert(i);
17         cin >> p[i];
18     }

```

```

19 for (int i = 0; i < n; ++i) {
20     int ind;
21     cin >> ind;
22     ind--;
23     int pos = *st.find_by_order(ind);
24     cout << st.order_of_key(pos) << ' ';
25     st.erase(pos);
26     cout << p[pos] << (i == n - 1 ? '\n' : '
27     ');

```

4.9 Trie (Prefix tree)

```

1 struct trie {
2     int n = 0;
3     trie *a[2];
4     trie() {
5         a[0] = a[1] = nullptr;
6     }
7
8     void insert(int k) {
9         trie *curr = this;
10        for (int i = 63; i >= 0; --i) {
11            bool bit = (k & (1LL << i)) > 0;
12            if (curr->a[bit] == nullptr) {
13                curr->a[bit] = new trie();
14            }
15            curr = curr->a[bit];
16            curr->n++;
17        }
18    }
19
20    void erase(int k) {
21        trie *curr = this;
22        for (int i = 63; i >= 0; --i) {
23            bool bit = (k & (1LL << i)) > 0;
24            curr = curr->a[bit];
25            curr->n--;
26        }
27    }
28
29    int query(int k) {
30        trie *curr = this;
31        int x = 0;
32        for (int i = 63; i >= 0; --i) {
33            x ^= (1LL << i) & k;
34            x ^= 1LL << i;
35            bool bit = (k & (1LL << i)) ==
36                0;
37            if (curr->a[bit] == nullptr ||
38                curr->a[bit]->n == 0)
39                x ^= 1LL << i, bit ^= 1;
40            curr = curr->a[bit];
41        }
42        return x;
43    }

```

4.10 segment tree

```

1 // Segment tree
2 template<typename value_t, class merge_t>
3 class SGT {
4     int n;
5     vector<value_t> t;
6     value_t defa;
7     merge_t merge;
8 public:
9     explicit SGT(int _n, value_t _defa,
10         const merge_t& _merge = merge_t{})
11         : n{_n}, t(2 * n), defa{_defa},
12             merge{_merge} {}
13
14     void modify(int p, const value_t& x) {
15         for (t[p += n] = x; p > 1; p >>= 1)
16             t[p >> 1] = merge(t[p], t[p ^
17                 1]);
18     }
19
20     value_t query(int l, int r, value_t init
21         ) { return
22         query(l, r, defa); }
23
24     value_t query(int l, int r, value_t init
25         ) {
26         for (l += n, r += n; l < r; l >>= 1,
27             r >>= 1) {
28             if (l & 1) init = merge(init, t[
29                 l++]);
30             if (r & 1) init = merge(init, t[
31                 --r]);
32         }
33         return init;
34     }
35 };
36
37 // Custom merge for range minimum + index
38 struct MergeMin {
39     pair<int, int> operator()(const pair<int
40         , int>& a,
41         const pair<int
42         , int>& b)
43         const {
44         if (a.first != b.first) return (a.
45             first < b.first) ? a : b;
46         return (a.second < b.second) ? a : b
47             ; // tie-break on index
48     }
49 };
50
51 int main() {
52     int n = 6;
53     SGT<pair<int, int>, MergeMin> tree(n, {
54         INT_MAX, -1});
55
56     vector<int> a = {5, 3, 6, 1, 4, 2};
57     for (int i = 0; i < n; ++i)
58         tree.modify(i, {a[i], i});
59
60     auto [min_val, min_idx] = tree.query(1,
61         5); // range [1, 5)
62     cout << "Min value in [1, 5): " <<
63         min_val << ", at index " << min_idx
64         << '\n';
65
66     return 0;
67 }

```

4.11 BIT range update point query

```

1 // Fenwick Tree (Binary Indexed Tree) for
  Range Updates and Point Queries
2
3 template<typename T>
4 class BIT {
5 #define ALL(x) begin(x), end(x)
6 private:
7     vector<T> arr;
8     int n;
9     inline int lowbit(int x) { return x & (-
10        x); }
11     void addInternal(int s, T v) {
12         while (s > 0) {
13             arr[s] += v;
14             s -= lowbit(s);
15         }
16 public:
17     void init(int n_) {
18         n = n_;
19         arr.resize(n + 1);
20         fill(ALL(arr), 0);
21     }
22     void add(int l, int r, T v) {
23         // add v to interval [l, r], 1-based
24         addInternal(l, -v);
25         addInternal(r, v);
26     }
27     T query(int x) {
28         // value at index x
29         T res = 0;
30         while (x <= n) {
31             res += arr[x];
32             x += lowbit(x);
33         }
34         return res;
35     }
36 #undef ALL
37 };
38
39 int main() {
40     BIT<int> bit;
41     bit.init(5);
42
43     // add +3 to indices 1..3
44     bit.add(0, 3, 3);
45
46     // add +2 to indices 3..5
47     bit.add(2, 5, 2);
48
49     cout << "Value at 1 = " << bit.query(1)
50          << "\n"; // expect 3
51     cout << "Value at 3 = " << bit.query(3)
52          << "\n"; // expect 3+2=5
53     cout << "Value at 5 = " << bit.query(5)
54          << "\n"; // expect 2
55 }

```

4.12 DSU remove node find prev next one

```

1 // previous/next one
2 class PwNx {
3     vector<int> pa, sz, mn, mx;
4     int find(int x) { // collapsing find
5         return pa[x] == -1 ? x : pa[x] =
6             find(pa[x]);
7     }
8     void unionn(int x, int y) { // weighted
9         union
10         auto rx{find(x)}, ry{find(y)};
11         if (rx == ry) return;
12         if (sz[rx] < sz[ry]) swap(rx, ry);
13         pa[ry] = rx, sz[rx] += sz[ry], mn[rx]
14             = min(mn[rx], mn[ry]), mx[rx]
15             = max(mx[rx], mx[ry]);
16     }
17 public:
18     explicit PwNx(int n) : pa(n + 1, -1), sz
19         (n + 1, 1), mn(n + 1) { iota(mn.
20         begin(), mn.end(), 0), mx = mn; }
21     void remove(int i) { unionn(i, i + 1); }
22     int prev(int i) { return mn[find(i)] -
23         1; }
24     int next(int i) {
25         int j{mx[find(i)]};
26         if (i == j) j = mx[find(j + 1)];
27         return j;
28     }
29     bool exist(int i) { return i == mx[find(
30         i)]; }
31 };

```

4.13 DSU

```

1 // fast disjoint set union
2 class DSU {
3     vector<int> pa, sz;
4 public:
5     explicit DSU(int n) : pa(n, -1), sz(n,
6         1) {}
7     int find(int x) { // collapsing find
8         return pa[x] == -1 ? x : pa[x] =
9             find(pa[x]);
10    }
11    void unite(int x, int y) { // weighted
12        union
13        auto rx{find(x)}, ry{find(y)};
14        if (rx == ry) return;
15        if (sz[rx] < sz[ry]) swap(rx, ry);
16        pa[ry] = rx, sz[rx] += sz[ry];
17    }
18 };

```

5 Geometry

5.1 cross. product

```

1 // If cross(a,b,c) > 0, c is to the left of
  line ab
2 // If cross(a,b,c) < 0, c is to the right of
  line ab
3 // If cross(a,b,c) = 0, a, b, c are
  collinear
4
5 // point struct version
6 // 2D Point structure
7 struct Point {
8     double x, y;
9 };
10
11 // Cross product (scalar in 2D)
12 double cross(const Point& a, const Point& b,
13     const Point& c) {
14     // Computes (b - a) x (c - a)
15     return (b.x - a.x) * (c.y - a.y) -
16         (b.y - a.y) * (c.x - a.x);
17 }
18
19 // std::complex version
20 typedef std::complex<double> point;
21 #define x real()
22 #define y imag()
23
24 double cross(const point &a, const point &b)
25 {
26     return (std::conj(a) * b).imag();
27 }

```

5.2 Check Point in Polygon

```

1 /*
2 We can cast a ray from the point P going in
  any fixed direction (people usually go
  to the right).
3 If the point is located on the outside of
  the polygon the ray will intersect its
  edges an even number of times.
4 If the point is on the inside of the polygon
  then it will intersect the edge an odd
  number of times.
5 */
6
7 struct Point {
8     int x, y;
9 };
10
11 int cross(const Point& a, const Point& b,
12     const Point& c) {
13     // return (c - a) x (b - a)
14     int ans = (c.x - a.x) * (b.y - a.y) -
15         (c.y - a.y) * (b.x - a.x);
16     if (ans == 0) return 0;
17     return ans > 0 ? 1 : -1;
18 }

```

```

16 }
17
18 bool intersect(const Point& a, const Point&
19     b, const Point& c, const Point& d) {
20     int c1 = cross(a, b, c);
21     int c2 = cross(a, b, d);
22     int c3 = cross(c, d, a);
23     int c4 = cross(c, d, b);
24
25     return (c1 * c2 < 0 && c3 * c4 < 0);
26 }
27
28 bool onSegment(const Point& a, const Point&
29     b, const Point& c) {
30     return (cross(a, b, c) == 0 &&
31         min(a.x, b.x) <= c.x && c.x <=
32         max(a.x, b.x) &&
33         min(a.y, b.y) <= c.y && c.y <=
34         max(a.y, b.y));
35 }
36
37 void solve() {
38     int n, m;
39     cin >> n >> m;
40     vector<Point> poly(n);
41
42     for (int i = 0; i < n; ++i) {
43         int a, b;
44         cin >> a >> b;
45         poly[i] = {a, b};
46     }
47
48     // set a point at outside point to form
49     the ray
50     Point outside = {(int)2e9 + 7, (int)2e9
51         + 13};
52     for (int i = 0; i < m; ++i) {
53         int a, b;
54         bool found = false;
55         cin >> a >> b;
56         Point p = {a, b};
57
58         int cnt = 0;
59         if (intersect(poly.back(), poly[0],
60             p, outside)) {
61             cnt++;
62         }
63         if (onSegment(poly.back(), poly[0],
64             p)) {
65             cout << "BOUNDARY\n";
66             found = true;
67         }
68         for (int j = 0; j < n - 1 && !found;
69             ++j) {
70             if (intersect(poly[j], poly[j +
71                 1], p, outside)) {
72                 cnt++;
73             }
74             if (onSegment(poly[j], poly[j +
75                 1], p)) {
76                 cout << "BOUNDARY\n";
77                 found = true;
78             }
79         }
80         if (found) continue;
81         if (cnt & 1) {

```



```

71     cout << "INSIDE\n";
72 }
73 else {
74     cout << "OUTSIDE\n";
75 }
76 }
77 }

```

5.3 Point

```

1  /**
2   * Author: Ulf Lundstrom
3   * Date: 2009-02-26
4   * License: CC0
5   * Source: My head with inspiration from
6   *         tinyKACTL
7   * Description: Class to handle points in
8   *             the plane.
9   * T can be e.g. double or long long. (
10  * Avoid int.)
11  * Status: Works fine, used a lot
12  */
13 #pragma once
14
15 template <class T> int sgn(T x) { return (x
16 > 0) - (x < 0); }
17
18 template <class T>
19 struct Point {
20     typedef Point P;
21     T x, y;
22     explicit Point(T x=0, T y=0) : x(x), y(y)
23     {}
24     bool operator<(P p) const { return tie(x,y)
25 < tie(p.x,p.y); }
26     bool operator==(P p) const { return tie(x,
27 y)==tie(p.x,p.y); }
28     P operator+(P p) const { return P(x+p.x, y
29 +p.y); }
30     P operator-(P p) const { return P(x-p.x, y
31 -p.y); }
32     P operator*(T d) const { return P(x*d, y*d
33 ); }
34     P operator/(T d) const { return P(x/d, y/d
35 ); }
36     T dot(P p) const { return x*p.x + y*p.y; }
37     T cross(P p) const { return x*p.y - y*p.x;
38 }
39     T cross(P a, P b) const { return (a-*this)
40 .cross(b-*this); }
41     T dist2() const { return x*x + y*y; }
42     double dist() const { return sqrt((double)
43 dist2()); }
44     // angle to x-axis in interval [-pi, pi]
45     double angle() const { return atan2(y, x);
46 }
47     P unit() const { return *this/dist(); } //
48     makes dist()=1
49     P perp() const { return P(-y, x); } //
50     rotates +90 degrees
51     P normal() const { return perp().unit(); }
52     // returns point rotated 'a' radians ccw
53     around the origin
54     P rotate(double a) const {

```

```

36     return P(x*cos(a)-y*sin(a),x*sin(a)+y*
37 cos(a)); }
38 friend ostream& operator<<(ostream& os, P
39 p) {
40     return os << "(" << p.x << ", " << p.y <<
41 ")"; }
42 };

```

5.4 Polygon Lattice Points

```

1  /**
2   * Pick's Theorem:
3   * For a simple polygon with integer
4   * coordinates, the area A, the number of
5   * interior lattice points a
6   * and the number of boundary lattice points b
7   * satisfy:
8   * A = a + b/2 - 1
9   */
10 #pragma once
11
12 template <class T>
13 struct Polygon {
14     typedef Polygon P;
15     T x, y;
16     explicit Polygon(T x=0, T y=0) : x(x), y(y)
17     {}
18     bool operator<(P p) const { return tie(x,y)
19 < tie(p.x,p.y); }
20     bool operator==(P p) const { return tie(x,
21 y)==tie(p.x,p.y); }
22     P operator+(P p) const { return P(x+p.x, y
23 +p.y); }
24     P operator-(P p) const { return P(x-p.x, y
25 -p.y); }
26     P operator*(T d) const { return P(x*d, y*d
27 ); }
28     P operator/(T d) const { return P(x/d, y/d
29 ); }
30     T dot(P p) const { return x*p.x + y*p.y; }
31     T cross(P p) const { return x*p.y - y*p.x;
32 }
33     T cross(P a, P b) const { return (a-*this)
34 .cross(b-*this); }
35     T dist2() const { return x*x + y*y; }
36     double dist() const { return sqrt((double)
37 dist2()); }
38     // angle to x-axis in interval [-pi, pi]
39     double angle() const { return atan2(y, x);
40 }
41     P unit() const { return *this/dist(); } //
42     makes dist()=1
43     P perp() const { return P(-y, x); } //
44     rotates +90 degrees
45     P normal() const { return perp().unit(); }
46     // returns point rotated 'a' radians ccw
47     around the origin
48     P rotate(double a) const {

```

5.5 line intersect

```

1  // 2D Point structure
2  struct Point {
3      double x, y;

```

```

4  };
5
6  // Cross product (scalar in 2D)
7  double cross(const Point& a, const Point& b,
8  const Point& c) {
9      // Computes (b - a) x (c - a)
10     return (b.x - a.x) * (c.y - a.y) -
11            (b.y - a.y) * (c.x - a.x);
12 }
13
14 // Check if value is between two others (
15 // with endpoints allowed)
16 bool between(double a, double b, double x) {
17     return min(a, b) <= x && x <= max(a, b);
18 }
19
20 // Check if point c lies on segment ab
21 bool onSegment(const Point& a, const Point&
22 b, const Point& c) {
23     return cross(a, b, c) == 0 &&
24            between(a.x, b.x, c.x) &&
25            between(a.y, b.y, c.y);
26 }
27
28 // Main intersection check
29 bool segmentsIntersect(const Point& A, const
30 Point& B, const Point& C, const
31 Point& D) {
32     double c1 = cross(A, B, C);
33     double c2 = cross(A, B, D);
34     double c3 = cross(C, D, A);
35     double c4 = cross(C, D, B);
36
37     // General case
38     if (c1 * c2 < 0 && c3 * c4 < 0) return
39 true;
40
41     // Special cases: collinear +
42     // overlapping
43     if (c1 == 0 && onSegment(A, B, C))
44         return true;
45     if (c2 == 0 && onSegment(A, B, D))
46         return true;
47     if (c3 == 0 && onSegment(C, D, A))
48         return true;
49     if (c4 == 0 && onSegment(C, D, B))
50         return true;
51
52     return false;
53 }
54
55 int main() {
56     Point A{0, 0}, B{4, 4};
57     Point C{0, 4}, D{4, 0};
58
59     if (segmentsIntersect(A, B, C, D))
60         cout << "Segments intersect\n";
61     else
62         cout << "Segments do not intersect\n";
63
64     return 0;
65 }

```

5.6 Polygon area

```

1  /**
2   * Author: Ulf Lundstrom
3   * Date: 2009-03-21
4   * License: CC0
5   * Source: tinyKACTL
6   * Description: Returns twice the signed
7   *             area of a polygon.
8   * Clockwise enumeration gives negative
9   * area. Watch out for overflow if using
10  int as T!
11  * Status: Stress-tested and tested on
12  kattis:polygonarea
13  */
14 #pragma once
15
16 #include "Point.h"
17
18 template <class T>
19 T polygonArea2(vector<Point<T>>& v) {
20     T a = v.back().cross(v[0]);
21     for (int i = 0; i < (int)v.size() - 1;
22         ++i) {
23         a += v[i].cross(v[i+1]);
24     }
25     return a;
26 }
27
28 int main() {
29     // Example: square with vertices (0,0),
30     // (0,1), (1,1), (1,0)
31     vector<Point<long long>> poly = {
32     {0,0}, {0,1}, {1,1}, {1,0}
33     };
34     long long area2 = polygonArea2(poly);
35     cout << "Twice signed area = " << area2
36     << "\n";
37     cout << "Absolute area = " << abs(area2)
38     / 2.0 << "\n";
39
40     return 0;
41 }

```

5.7 using std::complex

```

1  #include <iostream>
2  #include <complex>
3  #include <cmath>
4
5  typedef std::complex<double> point;
6  #define x real()
7  #define y imag()
8
9  constexpr double PI = std::acos(-1.0);
10 // conj(a) = a 的共軛 · reflection of a
11 // across x-axis
12
13 // Basic operations
14 double dot(const point &a, const point &b) {
15     return (std::conj(a) * b).real(); }

```

```

14 double cross(const point &a, const point &b)
15 { return (std::conj(a) * b).imag(); }
16 // Projections / reflections / geometry
17 // helpers
18 point project_onto_vector(const point &p,
19 const point &v) {
20 return v * (dot(p, v) / std::norm(v));
21 }
22 point project_onto_line(const point &p,
23 const point &a, const point &b) {
24 return a + (b - a) * (dot(p - a, b - a)
25 / std::norm(b - a));
26 }
27 point reflect_across_line(const point &p,
28 const point &a, const point &b) {
29 return a + std::conj((p - a) / (b - a))
30 * (b - a);
31 }
32 point intersection(const point &a, const
33 point &b, const point &p, const point &q
34 ) {
35 double c1 = cross(p - a, b - a), c2 =
36 cross(q - a, b - a);
37 return (c1 * q - c2 * p) / (c1 - c2); //
38 // undefined if parallel (divide by
39 // zero)
40 }
41 double angle_between(const point &a, const
42 point &b) {
43 // angle of vector b - a
44 return std::arg(b - a);
45 }
46 double angle_ABC(const point &a, const point
47 &b, const point &c) {
48 double r = std::remainder(std::arg(a - b)
49 - std::arg(c - b), 2.0 * PI);
50 return std::abs(r);
51 }
52 int main() {
53 // sample
54 point a = 2.0; // (2,0)
55 point b(3.0, 7.0); // (3,7)
56 std::cout << a << ' ' << b << '\n'; //
57 (2,0) (3,7)
58 std::cout << a + b << '\n'; //
59 (5,7)
60 // usage examples
61 point p1(3, 2), p2(2, -7);
62 std::cout << "p1 + p2 = " << p1 + p2 <<
63 '\n'; // (5,-5)
64 std::cout << "p1 - p2 = " << p1 - p2 <<
65 '\n'; // (1,9)
66 std::cout << "3.0 * p1 = " << 3.0 * p1
67 << '\n'; // (9,6)
68 std::cout << "p1 / 5.0 = " << p1 / 5.0
69 << '\n'; // (0.6,0.4)
70 // dot / cross via complex

```

```

59 std::cout << "dot(p1,p2) = " << dot(p1,
60 p2) << '\n';
61 std::cout << "cross(p1,p2) = " << cross(
62 p1, p2) << '\n';
63 // distances, angle, rotation, polar/
64 // cartesian
65 std::cout << "squared dist = " << std::
66 norm(p1 - p2) << '\n';
67 std::cout << "euclid dist = " << std::
68 abs(p1 - p2) << '\n';
69 std::cout << "angle p1->p2 = " <<
70 angle_between(p1, p2) << '\n';
71 std::cout << "angle ABC = " << angle_ABC
72 (point(1,0), point(0,0), point(0,1))
73 << '\n';
74 // project / reflect / intersect
75 // examples
76 point v(1, 1);
77 point proj = project_onto_vector(p1, v);
78 std::cout << "project p1 onto v = " <<
79 proj << '\n';
80 point lineA(0,0), lineB(2,0);
81 std::cout << "project p1 onto line = "
82 << project_onto_line(p1, lineA,
83 lineB) << '\n';
84 std::cout << "reflect p1 across line = "
85 << reflect_across_line(p1, lineA,
86 lineB) << '\n';
87 // intersection example
88 point r1a(0,0), r1b(1,1);
89 point r2a(0,1), r2b(1,0);
90 std::cout << "intersection = " <<
91 intersection(r1a, r1b, r2a, r2b) <<
92 '\n';
93 // polar/cartesian and rotation
94 point polar_pt = std::polar(5.0, PI/4);
95 std::cout << "polar(5,PI/4) = " <<
96 polar_pt << '\n';
97 point rotated = p1 * std::polar(1.0, PI
98 /2); // rotate p1 by 90 degrees
99 // about origin
100 std::cout << "rotate p1 by 90deg = " <<
101 rotated << '\n';
102 return 0;

```

5.8 point distance from a line

```

1 // calculate area using cross product and
2 // divide by base length
3 double point_line_distance(const point &p,
4 const point &a, const point &b) {
5 // area = 0.5 * |cross(b - a, p - a)|
6 // base = |b - a|
7 // height = 2 * area / base = |cross(b -
8 a, p - a)| / |b - a|

```

```

6 std::abs(cross(b - a, p - a)) / std::abs
7 (b - a);
8 }
9 // Prim's algorithm
10 vector<tuple<int, int, long long>> Prim(
11 const vector<vector<pair<int, long long
12 >>>& adj) {
13 const auto& n = adj.size();
14 vector<tuple<int, int, long long>> mst
15 {};
16 vector<bool> found(n, false);
17 using ti = tuple<long long, int, int>;
18 priority_queue<ti, vector<ti>, greater<
19 ti>> pq{};
20 found[0] = true;
21 for (auto& [v, w] : adj[0]) pq.emplace(w
22 , 0, v);
23 for (int i = 0; i < n - 1; ++i) {
24 int mn, u, v;
25 do {
26 tie(mn, u, v) = pq.top(), pq.pop()
27 ();
28 } while (found[v]);
29 found[v] = true, mst.emplace_back(u,
30 v, mn);
31 for (auto& [x, w] : adj[v]) pq.
32 emplace(w, v, x);
33 }
34 return mst;
35 }

```

6 Graph

6.1 Prim

6.2 Eulerian cycle

```

1 // Eulerian cycle in an undirected graph
2 vector<int> euler_cycle(vector<vector<pair<
3 int, int>>>& adj, int w = 0) {
4 int n{adj.size()}, m{};
5 for (int v{0}; v < n; ++v) m += adj[v].
6 size();
7 m /= 2;
8 vector<int> res{};
9 stack<pair<int, int>> stk{};
10 stk.emplace(w, -1);
11 vector<int> nxt(n);
12 vector<bool> usd(m);
13 while (!stk.empty()) {
14 auto [u, i]{stk.top()};

```

```

15 while (nxt[u] < adj[u].size() && usd
16 [adj[u][nxt[u]].second]) ++nxt[u];
17 if (nxt[u] < adj[u].size()) {
18 auto [v, j]{adj[u][nxt[u]]};
19 ++nxt[u], usd[j] = true, stk.
20 emplace(v, j);
21 } else {
22 if (i != -1) res.push_back(i);
23 stk.pop();
24 }
25 return res;
26 }
27 int main() {
28 int n = 4; // number of vertices
29 vector<vector<pair<int, int>>> adj(n);
30 // Add edges with edge IDs
31 int eid = 0;
32 auto add_edge = [&](int u, int v) {
33 adj[u].push_back({v, eid});
34 adj[v].push_back({u, eid});
35 ++eid;
36 };
37 add_edge(0, 1);
38 add_edge(1, 2);
39 add_edge(2, 3);
40 add_edge(3, 0);
41 vector<int> cycle = euler_cycle(adj, 0);
42 cout << "Euler cycle (edge IDs in order)
43 : ";
44 for (int id : cycle) cout << id << " ";
45 cout << "\n";
46 return 0;
47 }

```

6.3 maximum weight bipartite matching

```

1 // maximum weight matching in a weighted
2 // bipartite graph
3 // Hungarian Algorithm - O(V^3)
4 class bipartiteGraph {
5 int n;
6 vector<long long> lx, ly;
7 vector<bool> vx, vy;
8 queue<int> qu{}; // only X's vertices
9 vector<int> py;
10 vector<long long> dy; // smallest (lx[x]
11 + ly[y] - w[x][y])
12 vector<int> pdy; // & which x
13 void relax(int x) {
14 for (int y{0}; y < n; ++y)
15 if (lx[x] + ly[y] - w[x][y] < dy
16 [y])

```

```

14         dy[y] = lx[x] + ly[y] - w[x
15             ][y], pdy[y] = x;
16     }
17     void augment(int y) {
18         while (y != -1) {
19             int x{qu.front()}; qu.pop();
20             int x{py[y]}, yy{mx[x]};
21             mx[x] = y, my[y] = x;
22             y = yy;
23         }
24     }
25     bool bfs() {
26         while (!qu.empty()) {
27             int x{qu.front()}; qu.pop();
28             for (int y{0}; y < n; ++y) {
29                 if (!vy[y] && lx[x] + ly[y]
30                     == w[x][y]) {
31                     vy[y] = true, py[y] = x;
32                     if (my[y] == -1) return
33                         augment(y), true;
34                     int xx{my[y]};
35                     qu.push(x), vx[xx] =
36                         true, relax(xx);
37                 }
38             }
39         }
40         return false;
41     }
42     void relabel() {
43         long long d{numeric_limits<long long>
44             ::max()};
45         for (int y{0}; y < n; ++y) if (!vy[y]
46             ) d = min(d, dy[y]);
47         for (int x{0}; x < n; ++x) if (vx[x]
48             ) lx[x] -= d;
49         for (int y{0}; y < n; ++y) if (vy[y]
50             ) ly[y] += d;
51         for (int y{0}; y < n; ++y) if (!vy[y]
52             ) dy[y] -= d;
53     }
54     bool expand() { // expand one layer with
55         new equality edges
56         for (int y{0}; y < n; ++y) {
57             if (!vy[y] && dy[y] == 0) {
58                 vy[y] = true, py[y] = pdy[y]
59                 ];
60                 if (my[y] == -1) return
61                     augment(y), true;
62                 int xx{my[y]};
63                 qu.push(xx), vx[xx] = true,
64                     relax(xx);
65             }
66         }
67         return false;
68     }
69     public:
70         vector<vector<long long>> w;
71         vector<int> mx, my;
72         bipartiteGraph(int _n) : n{_n}, lx(n),
73             ly(n), vx(n), vy(n), py(n), dy(n),
74             pdy(n),
75             w(n, vector<long long>(n, 0)), mx(n,
76                 -1), my(n, -1) {}
77         long long Hungarian() {
78             for (int i{0}; i < n; ++i) {
79                 lx[i] = ly[i] = 0;
80                 for (int j{0}; j < n; ++j)
81                     lx[i] = max(lx[i], w[i][j]);
82                 mx[i] = my[i] = -1;
83             }
84             for (int x{0}; x < n; ++x) {
85                 fill(vx.begin(), vx.end(), false
86                     );
87                 fill(vy.begin(), vy.end(), false
88                     );
89                 fill(dy.begin(), dy.end(),
90                     numeric_limits<long long>::
91                         max());
92                 qu = {}, qu.push(x), vx[x] =
93                     true, relax(x);
94                 while (!bfs()) {
95                     relabel();
96                     if (expand()) break;
97                 }
98             }
99             long long weight{0};
100             for (int x{0}; x < n; ++x) weight +=
101                 w[x][mx[x]];
102             return weight;
103         }
104     };
105     // usage example:
106     int main() {
107         int n = 4;
108         bipartiteGraph g(n);
109         long long mat[4][4] = {
110             {9, 2, 7, 8},
111             {6, 4, 3, 7},
112             {5, 8, 1, 8},
113             {7, 6, 9, 4}
114         };
115         g.Hungarian();
116         std::cout << "Maximum weight: " <<
117             maxWeight << "\n";
118         for (int i = 0; i < n; ++i)
119             for (int j = 0; j < n; ++j)
120                 g.w[i][j] = mat[i][j];
121         long long maxWeight = g.Hungarian();
122         std::cout << "Maximum weight: " <<
123             maxWeight << "\n";
124         for (int i = 0; i < n; ++i)
125             for (int j = 0; j < n; ++j)
126                 g.w[i][j] = mat[i][j];
127         std::cout << "X " << i << " matched
128             with Y " << g.mx[i] << "\n";
129     }
130 }
131
132 6.4 maximum flow
133
134 // minimum cut maximum flow
135 // relabel-to-front algorithm
136 // (an implementation of generic push-relabel
137     algorithm)
138
139 // handle the capacity, c, of a vertex, v.
140 // Add a new vertex, v', and an edge v-v'
141     with capacity c.
142 template<typename T> //requires
143     signed_integral<T>
144 void handle_vertex_capacity(int u, T c, int&
145     n, vector<tuple<int, int, T>>& e) {
146     int w{n++};
147     for (auto& [_u, _v, _c] : e) if (_u == u
148         ) _u = w;
149     e.emplace_back(u, w, c);
150 }
151 template<typename T> //requires
152     signed_integral<T>
153 class flowNetwork {
154     int n, s, t;
155     vector<int> h; // height label
156     vector<T> e; // excess
157     vector<vector<int>> adj; // adjacent
158         list
159     vector<vector<T>> c, r{};
160     vector<size_t> p; // record the position
161         processed
162     list<int> lst{}; // topological sort of
163         admissible network
164     void push(int u, int v) {
165         T f{min(e[u], r[u][v])};
166         r[u][v] -= f, r[v][u] += f;
167         e[u] -= f, e[v] += f;
168     }
169     void relabel(int u) {
170         int mn{numeric_limits<int>::max()};
171         for (auto& v : adj[u])
172             if (r[u][v]) mn = min(mn, h[v]);
173         h[u] = mn + 1;
174     }
175     void init() {
176         fill(h.begin(), h.end(), 0);
177         fill(e.begin(), e.end(), 0);
178         lst.clear();
179         for (int u{0}; u < n; ++u) if (u !=
180             s && u != t) lst.push_back(u);
181         fill(p.begin(), p.end(), 0);
182         r = c;
183     }
184     void preflow() {
185         h[s] = n;
186         for (auto& v : adj[s]) {
187             e[v] += r[s][v], e[s] -= r[s][v]
188             ];
189             r[v][s] += r[s][v], r[s][v] = 0;
190         }
191     }
192     bool discharge(int u) {
193         bool flag{false};
194         while (e[u]) {
195             for (; p[u] < adj[u].size(); ++p
196                 [u]) {
197                 int v{adj[u][p[u]]};
198                 if (r[u][v] && h[u] == h[v]
199                     + 1) push(u, v);
200                 if (!e[u]) break;
201             }
202             if (e[u]) relabel(u), p[u] = 0,
203                 flag = true;
204         }
205         return flag; // return if relabel or
206             not
207     }
208     public:
209         flowNetwork(int _n) : n{_n}, s{0}, t{n -
210             1}, h(n), e(n), adj(n, vector
211                 <T>(n)), p(n) {}
212         void set_st(int _s, int _t) {
213             s = _s, t = _t;
214         }
215         void add_edge(int u, int v, T _c) {
216             if (!c[u][v] && !c[v][u]) adj[u].
217                 push_back(v), adj[v].push_back(u)
218                 );
219             c[u][v] += _c;
220         }
221         T max_flow() {
222             assert(s != t);
223             init();
224             preflow();
225             auto it{lst.begin()};
226             while (it != lst.end()) {
227                 if (discharge(*it)) { // relabel
228                     -to-front
229                     lst.push_front(*it), lst.
230                         erase(it);
231                     it = lst.begin();
232                 }
233                 it = next(it);
234             }
235             return e[t];
236         }
237     };
238     int main() {
239         using T = int;
240         // Build an example graph with a vertex
241             capacity on node 2.
242         int n = 4; // nodes: 0 (source), 1, 2, 3
243             (sink)
244         vector<tuple<int, int, T>> edges = {
245             {0, 1, 3},
246             {0, 2, 2},
247             {1, 2, 5},
248             {1, 3, 2},
249             {2, 3, 3}
250         };
251         // Enforce vertex capacity of 2 on node
252             2.
253         handle_vertex_capacity<T>(2, 2, n, edges
254             );
255         // Build the network and compute max
256             flow.
257         flowNetwork<T> net(n);
258         net.set_st(0, 3);
259         for (auto [u, v, cap] : edges) net.
260             add_edge(u, v, cap);
261         cout << "Max flow = " << net.max_flow()
262             << '\n';
263     }

```

6.5 maximum cardinality bipartite matching

```

1 class BipartiteMatching {
2     vector<bool> vis;
3     vector<int> mx{}, my{};
4     bool dfs(const vector<vector<int>>& adj,
5             int x) {
6         for (int y : adj[x]) {
7             if (vis[y]) continue;
8             vis[y] = true;
9             if (my[y] == -1 || dfs(adj, my[y])) {
10                 mx[x] = y; my[y] = x;
11                 return true;
12             }
13         }
14         return false;
15     }
16 public:
17     pair<vector<int>, vector<int>> operator
18     ()(const vector<vector<int>>& adj,
19        int ny) {
20         vis.assign(ny, false);
21         mx.assign((int)adj.size(), -1);
22         my.assign(ny, -1);
23         for (int x = 0; x < (int)adj.size();
24             ++x) {
25             fill(vis.begin(), vis.end(),
26                  false);
27             dfs(adj, x);
28         }
29         return {mx, my};
30     }
31 };
32
33 // Usage example:
34 int main() {
35     // Left partition has 4 nodes (0..3),
36     // right partition has 4 nodes (0..3).
37     vector<vector<int>> adj(4);
38     adj[0] = {0, 1};
39     adj[1] = {1, 2};
40     adj[2] = {2};
41     adj[3] = {2, 3};
42
43     BipartiteMatching bm;
44     auto res = bm(adj, 4);
45     const auto& mx = res.first;
46
47     cout << "Matched pairs (left -> right):\n";
48     for (int i = 0; i < (int)mx.size(); ++i)
49         if (mx[i] != -1)
50             cout << i << " -> " << mx[i] <<
51             "\n";
52 }

```

6.6 Floyd-Warshall

```
1 // Floyd-Warshall algorithm
```

```

2 template<typename T>
3 vector<vector<optional<T>>> Floyd_Warshall(
4     const vector<vector<optional<T>>>& adj)
5 {
6     const auto& n{adj.size()};
7     auto d{adj};
8
9     for (int i{0}; i < n; ++i) d[i][i] = 0;
10    for (int k{0}; k < n; ++k)
11        for (int i{0}; i < n; ++i)
12            for (int j{0}; j < n; ++j) {
13                if (!d[i][k] || !d[k][j])
14                    continue; // no value
15                if (!d[i][j] || d[i][j] > d[
16                    i][k].value() + d[k][j].
17                    value())
18                    d[i][j] = d[i][k].value
19                    () + d[k][j].value()
20                    ;
21            }
22    }
23    return d;
24 }

```

6.7 MST

```

1 // Kruskal' s algorithm
2 vector<tuple<int, int, long long>> Kruskal(
3     int n, vector<tuple<long long, int, int>
4     >>& edges) {
5     vector<tuple<int, int, long long>> mst
6     {};
7     DSU dsu{n};
8
9     sort(edges.begin(), edges.end());
10    for (auto& [w, u, v] : edges)
11        if (dsu.find(u) != dsu.find(v))
12            dsu.unionn(u, v), mst.
13            emplace_back(u, v, w);
14    return mst;
15 }

```

6.8 Hopcroft-Karp

```

1 // maximum cardinality bipartite matching in
2 // unweighted bipartite graph
3 // Hopcroft-Karp algorithm - O(EV^2)
4 class bipartiteGraph {
5     int nx, ny;
6     vector<int> dx, dy;
7     bool dfs(int y) {
8         for (auto& x : adjy[y]) {
9             if (dx[x] + 1 != dy[y]) continue
10             ;
11             if (mx[x] == -1 || dfs(mx[x])) {
12                 my[y] = x, mx[x] = y;
13                 dy[y] = -1; // augmented -
14                 remove from BFS forest
15                 return true;
16             }
17         }
18     }
19 }

```

```

19 }
20 dy[y] = -1; // no augmenting path -
21 // remove from BFS forest
22 return false;
23 }
24 bool bfs() {
25     fill(dx.begin(), dx.end(), -1);
26     fill(dy.begin(), dy.end(), -1);
27     queue<int> qu1, qu2{};
28     for (int x{0}; x < nx; ++x) // use
29         all unmatched x's as roots
30         if (mx[x] == -1) qu1.push(x), dx[
31             x] = 0;
32
33     bool found{false};
34     while (!qu1.empty() && !found) { //
35         stop at the level found
36         while (!qu2.empty()) {
37             auto x{qu1.front()}; qu1.pop()
38             ;
39             for (auto& y : adjx[x])
40                 if (dy[y] == -1 && my[y]
41                     != x) {
42                     dy[y] = dx[x] + 1;
43                     if (my[y] == -1)
44                         found = true;
45                     else qu2.push(my[y])
46                         , dx[my[y]] = dy
47                         [y] + 1;
48                 }
49             qu.swap(qu2);
50         }
51         return found;
52     }
53     vector<vector<int>> adjx, adjy;
54 public:
55     vector<int> mx, my;
56     bipartiteGraph(int _nx, int _ny) : nx{
57         _nx}, ny{_ny}, dx(nx), dy(ny)
58         , adjx(nx), adjy(ny), mx(nx, -1), my
59         (ny, -1) {}
60     void addEdge(int x, int y) {
61         adjx[x].push_back(y), adjy[y].
62         push_back(x);
63     }
64     int Hopcroft_Karp() {
65         int c{0};
66         while (bfs()) {
67             for (int y{0}; y < ny; ++y)
68                 if (my[y] == -1 && dy[y] !=
69                     -1) c += dfs(y);
70         }
71         return c;
72     }
73 };
74
75 // Usage example:
76 int main() {
77     // Left partition has 4 nodes (0..3),
78     // right partition has 4 nodes (0..3).
79     bipartiteGraph bg(4, 4);
80     bg.addEdge(0, 0);
81     bg.addEdge(0, 1);
82     bg.addEdge(1, 1);
83     bg.addEdge(1, 2);
84 }

```

```

66     bg.addEdge(2, 2);
67     bg.addEdge(3, 2);
68     bg.addEdge(3, 3);
69
70     int max_matching = bg.Hopcroft_Karp();
71     cout << "Maximum matching size: " <<
72     max_matching << "\n";
73     cout << "Matched pairs (left -> right):\n";
74     for (int i = 0; i < (int)bg.mx.size();
75         ++i)
76         if (bg.mx[i] != -1)
77             cout << i << " -> " << bg.mx[i]
78             << "\n";
79 }

```

6.9 Dijkstra

```

1 // Dijkstra algorithm
2 template<typename T>
3 vector<optional<T>> Dijkstra(const vector<
4     vector<pair<int, T>>& adj, int s) {
5     const auto& n{adj.size()};
6     vector<optional<T>> d(n, nullopt);
7     d[s] = 0;
8
9     vector<bool> found(n, false);
10    using pq_t = pair<T, int>;
11    priority_queue<pq_t, vector<pq_t>,
12        greater<pq_t>> pq{};
13    pq.emplace(0, s);
14    while (!pq.empty()) {
15        auto [_, u]{pq.top()}; pq.pop();
16        if (found[u]) continue;
17        found[u] = true;
18        for (auto& [v, w] : adj[u])
19            if (!d[v] || d[v] > d[u].value()
20                + w) {
21                d[v] = d[u].value() + w;
22                pq.emplace(d[v].value(), v);
23            }
24    }
25    return d;
26 }

```

6.10 Dinic

```

1 int n, s, t, cnt, inf = 1e17;
2 vector<vector<int>> adj;
3 int level[100000], ptr[100000], rev[100000],
4     to[100000], cap[100000];
5 void add(int a, int b, int c) {
6     int id = cnt++, rid = cnt++;
7     to[id] = b;
8     to[rid] = a;
9     cap[id] = c;
10    cap[rid] = 0;
11    rev[id] = rid;

```

```

12 rev[rid] = id;
13 adj[a].push_back(id);
14 adj[b].push_back(rid);
15 }
16 bool bfs() {
17     memset(level, -1, sizeof(level));
18     queue<int> q;
19     q.push(s);
20     level[s] = 0;
21
22     while(!q.empty()) {
23         int curr = q.front(); q.pop();
24         for(int id: adj[curr]) {
25             int next = to[id];
26             if(cap[id] > 0 && level[next] ==
27                -1) {
28                 level[next] = level[curr] +
29                     1;
30                 q.push(next);
31             }
32         }
33     }
34     return level[t] != -1;
35 }
36 int dfs(int curr, int flow) {
37     if(curr == t || flow == 0) return flow;
38     for(; ptr[curr] < adj[curr].size(); ptr[
39         curr]++) {
40         int id = adj[curr][ptr[curr]];
41         int next = to[id];
42         if(cap[id] > 0 && level[next] ==
43            level[curr] + 1) {
44             int temp = dfs(next, min(flow,
45                 cap[id]));
46             if(temp == 0) continue;
47             int rid = rev[id];
48             cap[id] -= temp;
49             cap[rid] += temp;
50             return temp;
51         }
52     }
53     return 0;
54 }
55 int maxflow() {
56     int flow = 0;
57     while(bfs()) {
58         memset(ptr, 0, sizeof(ptr));
59         while(true) {
60             int temp = dfs(s, inf);
61             if(temp == 0) break;
62             flow += temp;
63         }
64     }
65     return flow;
66 }

```

6.11 maximum cardinality matching

```

1 // maximum matching
2 // Edmonds' Blossom algorithm

```

```

3 // O(VEa(E,V))
4 class graph {
5     int n;
6     vector<int> p, d, a, c1, c2;
7     // (alternating tree), (unvisited: -1,
8     // even: 0, odd: 1), (auxiliary for lca
9     //), (cross edge)
10    int label{0};
11    /* DSU */
12    vector<int> dsu_p, dsu_sz, dsu_b;
13    void dsu_reset() {
14        fill(dsu_p.begin(), dsu_p.end(), -1)
15        ;
16        fill(dsu_sz.begin(), dsu_sz.end(),
17            1);
18        iota(dsu_b.begin(), dsu_b.end(), 0);
19    }
20    int find(int x) { // collapsing find
21        return dsu_p[x] == -1 ? x : dsu_p[x]
22            = find(dsu_p[x]);
23    }
24    int base(int x) { return dsu_b[find(x)];
25    }
26    void contract(int x, int y) { //
27        weighted union
28        auto rx{find(x)}, ry{find(y)};
29        if (rx == ry) return;
30        auto b{dsu_b[rx]}; // treat x's base
31        // as new base
32        if (dsu_sz[rx] < dsu_sz[ry]) swap(rx
33            , ry);
34        dsu_p[ry] = rx, dsu_sz[rx] += dsu_sz
35            [ry], dsu_b[rx] = b;
36    }
37    //*****
38    queue<int> qu{}; // only even vertices
39    void handle_one_side(int x, int y, int b
40        ) {
41        for (int v{base(x)}; v != b; v =
42            base(p[v])) {
43            c1[v] = x, c2[v] = y, contract(b
44                , v);
45            if (d[v] == 1) qu.push(v); //
46                odd vertex -> even vertex
47        }
48    }
49    int lca(int u, int v) {
50        ++label; // use next number in order
51        // not to clear a
52        while (true) {
53            if (a[u] == label) return u;
54            a[u] = label;
55            if (p[u] != -1) u = base(p[u]);
56            swap(u, v);
57        }
58    }
59    void augment(int r, int y) {
60        if (r == y) return;
61        if (d[y] == 0) { // even vertex ->
62            odd vertex
63            // check d[y] == 0 instead of d[
64                p[y]] == 1, so (m[y], y) is
65                in the blossom
66            augment(m[y], m[c1[y]]);
67            augment(r, m[c2[y]]);
68        }
69    }
70 }

```

```

71 m[c1[y]] = c2[y], m[c2[y]] = c1[
72     y];
73 } else {
74     int x{p[y]};
75     augment(r, m[x]);
76     m[x] = y, m[y] = x;
77 }
78 }
79 bool bfs(int r) {
80     dsu_reset();
81     fill(d.begin(), d.end(), -1);
82
83     qu = {}, qu.push(r), d[r] = 0;
84     while (!qu.empty()) {
85         int x{qu.front()}; qu.pop();
86         for (auto& y : adj[x]) {
87             if (base(x) == base(y))
88                 continue;
89             if (d[y] == -1) {
90                 p[y] = x, d[y] = 1;
91                 if (m[y] == -1) { //
92                     augmenting path
93                     augment(-1, y);
94                     return true;
95                 } else {
96                     p[m[y]] = y, d[m[y]]
97                         = 0;
98                     qu.push(m[y]);
99                 }
100             } else if (d[base(y)] == 0) {
101                 // blossom
102                 int b{lca(base(x), base(
103                     y))};
104                 handle_one_side(x, y, b)
105                 ;
106                 handle_one_side(y, x, b)
107                 ;
108             }
109         }
110     }
111     return false;
112 }
113 public:
114     vector<vector<int>>> adj;
115     vector<int> m;
116     explicit graph(int _n) : n{_n}, p(n, -1)
117         , d(n), a(n), c1(n), c2(n), dsu_p(n)
118         , dsu_sz(n), dsu_b(n), adj(n), m(n,
119             -1) {}
120     int Edmonds() {
121         int c{0};
122         for (int v{0}; v < n; ++v)
123             if (m[v] == -1 && bfs(v)) ++c;
124         return c;
125     }
126 }
127 // usage example:
128 int main() {
129     int n = 5;
130     graph g(n);
131     auto add = [&](int u, int v) {
132         g.adj[u].push_back(v);
133         g.adj[v].push_back(u);
134     };
135 }

```

```

136 // sample graph
137 add(0, 1);
138 add(0, 2);
139 add(1, 2); // triangle (0,1,2)
140 add(2, 3);
141 add(3, 4);
142
143 int match_size = g.Edmonds();
144 cout << "Maximum matching size: " <<
145     match_size << "\n";
146 for (int i = 0; i < n; ++i)
147     if (g.m[i] != -1 && i < g.m[i])
148         cout << i << " - " << g.m[i] <<
149             "\n";
150 }

```

6.12 topological sort

```

1 // topological sort 1
2 optional<vector<int>>> top_sort(vector<vector
3     <int>>& adj) {
4     vector<int> res{};
5     int n{static_cast<int>(adj.size())};
6     vector<int> cnt(n, 0); // predecessor
7     // count
8     for (int u = 0; u < n; ++u)
9         for (auto& v : adj[u]) ++cnt[v];
10
11     queue<int> qu{};
12     for (int u = 0; u < n; ++u) if (!cnt[u])
13         qu.push(u);
14     while (!qu.empty()) {
15         auto u = qu.front();
16         qu.pop();
17         res.push_back(u);
18
19         for (auto& v : adj[u])
20             if (--cnt[v] == 0) qu.push(v);
21     }
22     if (res.size() != adj.size()) return
23         nullopt;
24     return res;
25 }

```

6.13 Floyd's algorithm

```

1 // Floyd's Cycle-Finding Algorithm
2 // Given a function succ(x) that returns the
3 // successor of x,
4 a = succ(x);
5 b = succ(succ(x));
6 while (a != b) {
7     a = succ(a);
8     b = succ(succ(b));
9 }
10 // Find the first element in the cycle
11 a = x;
12 while (a != b) {

```



```

13     a = succ(a);
14     b = succ(b);
15 }
16 first = a;
17
18 // Find the length of the cycle
19 length = 1;
20 b = succ(a);
21 while (a != b) {
22     b = succ(b);
23     length = length + 1;
24 }
25
26 // Finding the starting point of the cycle
27 // from any node
28 int n;
29 vector<int> adj, res;
30 vector<vector<int>> radj;
31
32 void fill_radj(int u) {
33     for (int v : radj[u]) {
34         if (res[v] == -1) {
35             res[v] = res[u];
36             fill_radj(v);
37         }
38     }
39 }
40
41 void floyd(int x) {
42     int y = x;
43     do { // find a cycle using Floyd's
44         // algorithm
45         x = adj[x];
46         y = adj[adj[y]];
47     } while (y != x);
48     do { // set res[x] = x for all x along
49         // cycle
50         res[x] = x;
51         x = adj[x];
52     } while (y != x);
53     do { // set res'es for all x not along
54         // cycle
55         fill_radj(x);
56         x = adj[x];
57     } while (y != x);
58 }
59
60 void solve() {
61     cin >> n;
62     adj.resize(n);
63     radj.assign(n, {});
64     res.assign(n, -1);
65
66     for (auto& e : adj) {
67         cin >> e;
68         e--;
69     }
70
71     for (int i = 0; i < n; ++i) {
72         radj[adj[i]].push_back(i);
73     }
74
75     for (int i = 0; i < n; ++i) {
76         if (res[i] == -1) {
77             floyd(i);
78         }
79     }
80 }

```

6.14 stable matching

```

1 // Stable Marriage Problem (stable matching)
2 // Gale-Shapley algorithm
3 class bipartiteGraph {
4 public:
5     int n;
6     vector<vector<int>> px, py;
7     vector<int> mx, my;
8
9     bipartiteGraph(int _n) : n(_n), px(n,
10         vector<int>(n)), py(n, vector<int>(n)),
11         mx(n, -1), my(n, -1) {}
12
13     void Gale_Shapley() {
14         queue<int> qu{};
15         vector<priority_queue<pair<int, int>
16             >>> pq(n);
17         for (int x{0}; x < n; ++x) {
18             qu.push(x);
19             for (int y{0}; y < n; ++y) pq[x]
20                 .emplace(px[x][y], y);
21         }
22
23         while (!qu.empty()) {
24             auto x{qu.front()}; qu.pop();
25
26             int y;
27             do {
28                 y = pq[x].top().second; pq[x]
29                     .pop();
30             } while (my[y] != -1 && py[y][x]
31                 <= py[y][my[y]]);
32
33             if (my[y] != -1) mx[my[y]] = -1,
34                 qu.push(my[y]); // y
35                 // prefers x to my[y]
36             mx[x] = y, my[y] = x;
37         }
38     }
39 };
40
41 int main() {
42     int n = 3;
43     bipartiteGraph g(n);
44
45     // Men preferences (from most to Least
46     // preferred)
47     vector<vector<int>> men = {
48         {0, 1, 2},
49         {1, 2, 0},
50         {1, 0, 2}
51     };
52
53     // Women preferences (from most to Least
54     // preferred)

```

```

44     vector<vector<int>> women = {
45         {1, 0, 2},
46         {2, 1, 0},
47         {0, 1, 2}
48     };
49
50     // Convert orders to scores (higher is
51     // better)
52     for (int x = 0; x < n; ++x)
53         for (int pos = 0; pos < n; ++pos)
54             g.px[x][men[x][pos]] = n - pos;
55
56     for (int y = 0; y < n; ++y)
57         for (int pos = 0; pos < n; ++pos)
58             g.py[y][women[y][pos]] = n - pos;
59
60     g.Gale_Shapley();
61
62     // Output matches: man x is matched to
63     // woman g.mx[x]
64     for (int x = 0; x < n; ++x)
65         cout << x << " " << g.mx[x] << '\n';
66
67     return 0;
68 }

```

6.15 Bellman-Ford

```

1 // Bellman-Ford algorithm
2 template<typename T>
3 optional<vector<optional<T>>> Bellman_Ford(
4     const vector<vector<pair<int, T>>>& adj,
5     int s) {
6     const auto& n{adj.size()};
7     vector<optional<T>> d(n, nullopt);
8     d[s] = 0;
9
10    queue<int> qu{};
11    vector<bool> in(n, false), in2(n, false);
12
13    qu.push(s); in[s] = true;
14    for (int i{0}; i < n; ++i) { // at most
15        // n-1 edges
16        while (!qu.empty()) {
17            int u{qu.front()};
18            qu.pop(); in[u] = false;
19            for (auto& [v, w] : adj[u])
20                if (!d[v] || d[v] > d[u].
21                    value() + w) { // relax
22                    d[v] = d[u].value() + w;
23                    if (!in2[v]) qu2.push(v);
24                    in2[v] = true;
25                }
26            qu.swap(qu2); in.swap(in2);
27        }
28    }
29
30    if (qu.empty()) return d;
31    return nullopt; // if negative cycle
32 }

```

7 Language

7.1 CNF

```

1 #define MAXN 55
2 struct CNF{
3     int s,x,y;//s->xy | s->x, if y==1
4     int cost;
5     CNF(){}
6     CNF(int s,int x,int y,int c):s(s),x(x),y(y),cost(c){}
7 };
8 int state;//規則數量
9 map<char,int> rule;//每個字元對應到的規則。
10 //小寫字母為終端字符
11 vector<CNF> cnf;
12 void init(){
13     state=0;
14     rule.clear();
15     cnf.clear();
16 }
17 void add_to_cnf(char s,const string &p,int cost){
18     //加入一個s -> <p>的文法。代價為cost
19     if(rule.find(s)==rule.end())rule[s]=state++;
20     for(auto c:p)if(rule.find(c)==rule.end())rule[c]=state++;
21     if(p.size()==1){
22         cnf.push_back(CNF(rule[s],rule[p[0]],-1,cost));
23     }else{
24         int left=rule[s];
25         int sz=p.size();
26         for(int i=0;i<sz-2;++i){
27             cnf.push_back(CNF(left,rule[p[i]],state,0));
28             left=state++;
29         }
30         cnf.push_back(CNF(left,rule[p[sz-2]],rule[p[sz-1]],cost));
31     }
32 }
33 vector<long long> dp[MAXN][MAXN];
34 vector<bool> neg_INF[MAXN][MAXN];//如果花費
35 //是負的可能會有無限小的情形
36 void relax(int l,int r,const CNF &c,long long cost,bool neg_c=0){
37     if(!neg_INF[l][r][c.s]&&(neg_INF[l][r][c.x]||cost<dp[l][r][c.s])){
38         if(neg_c||neg_INF[l][r][c.x]){
39             dp[l][r][c.s]=0;
40             neg_INF[l][r][c.s]=true;
41         }else dp[l][r][c.s]=cost;
42     }
43 }
44 void bellman(int l,int r,int n){
45     for(int k=1;k<=state;++k)
46         for(auto c:cnf)
47             if(c.y==1)relax(l,r,c,dp[l][r][c.x]+c.cost,k=n);
48 }

```

```

47 void cyk(const vector<int> &tok){
48     for(int i=0;i<(int)tok.size();++i){
49         for(int j=0;j<(int)tok.size();++j){
50             dp[i][j]=vector<long long>(state+1,
51                 INT_MAX);
52             neg_INF[i][j]=vector<bool>(state+1,
53                 false);
54         }
55         dp[i][i][tok[i]]=0;
56         bellman(i,i,tok.size());
57     }
58     for(int r=1;r<(int)tok.size();++r){
59         for(int l=r-1;l>=0;--l){
60             for(int k=l;k<r;++k)
61                 for(auto c:cnf)
62                     if(~c.y)relax(l,r,c,dp[l][k][c.x]+
63                         dp[k+1][r][c.y]+c.cost);
64             bellman(l,r,tok.size());
65         }
66     }
67 }

```

8 Number Theory

8.1 Linear Sieve

```

1 // Calculate the smallest divisor of
2 // integers in [2, maxn) in O(maxn)
3 vector<int> min_div[[]] {
4     constexpr int maxn = 400000 + 10;
5     vector<int> v(maxn), p;
6
7     for (int i = 2; i < maxn; ++i) {
8         if (!v[i]) {
9             v[i] = i;
10            p.push_back(i);
11        }
12        for (int j = 0; p[j] * i < maxn; ++j) {
13            v[p[j] * i] = p[j];
14            if (p[j] == v[i]) break;
15        }
16    }
17    return v;
18 }();

```

8.2 C(n,m)

```

1 ll Cnm(ll n, ll m) {
2     if (m > n / 2) m = n - m;
3     ll r[1];
4     for (ll i{1}, j{n}; i <= m; ++i, --j) r
5         *= j, r /= i;
6     return r;
7 }

```

8.3 Euler Totient 1 to n

```

1 void phi_1_to_n(int n) {
2     vector<int> phi(n + 1);
3     for (int i = 0; i <= n; ++i)
4         phi[i] = i;
5
6     for (int i = 2; i <= n; ++i) {
7         if (phi[i] == i) {
8             for (int j = i; j <= n; j += i)
9                 phi[j] -= phi[j] / i;
10        }
11    }
12 }

```

8.4 derangement (Principle of Inclusion-Exclusion)

```

1 // 1. Principle of Inclusion-Exclusion
2 // n! = n! * Σ (from k=0 to n) [((-1)^k) / (k!)]
3
4 mint c = 1;
5 for (int i = 1; i <= n; i++) {
6     c = (c * i) + (i % 2 == 1 ? -1 : 1);
7     cout << c.val() << ' ';
8 }

```

8.5 matrix template (with fast power)

```

1 template<class T> struct Matrix {
2     T **mat; int a, b;
3
4     Matrix() : a(0), b(0) {}
5     Matrix(int a_, int b_) : a(a_), b(b_) {}
6     int i, j;
7     mat = new T*[a];
8     for (i = 0; i < a; ++i) {
9         mat[i] = new T[b];
10        for (j = 0; j < b; ++j) {
11            mat[i][j] = 0;
12        }
13    }
14 }
15 Matrix(const vector<vector<T>>& vt) {
16     int i, j;
17
18     *this = Matrix((int)vt.size(), (int)
19         vt[0].size());
20     for (i = 0; i < a; ++i) {
21         for (j = 0; j < b; ++j) {
22             mat[i][j] = vt[i][j];
23         }
24     }
25 }
26 Matrix operator*(const Matrix& m) {

```

```

26     int i, j, k;
27
28     assert(b == m.a);
29     Matrix r(a, m.b);
30     for (i = 0; i < a; ++i) {
31         for (j = 0; j < m.b; ++j) {
32             for (k = 0; k < b; ++k) {
33                 r.mat[i][j] += mul(mat[i]
34                     ][k], m.mat[k][j]);
35                 r.mat[i][j] %= MOD;
36             }
37         }
38     }
39     return r;
40 }
41 Matrix& operator*=(const Matrix& m) {
42     return *this = (*this) * m;
43 }
44 friend Matrix power(Matrix m, long long
45     p) {
46     int i;
47
48     assert(m.a == m.b);
49     Matrix r(m.a, m.b);
50     for (i = 0; i < m.a; ++i) {
51         r.mat[i][i] = 1;
52     }
53     for (; p > 0; p >= 1, m *= m) {
54         if (p & 1) {
55             r *= m;
56         }
57     }
58     return r;
59 }
60 int main() {
61     Matrix<int> mat(adj);
62     mat = power(mat, k); // mat^k
63     cout << mat.mat[i][j];
64 }

```

8.6 Sieve of Eratosthenes (with big num)

```

1 const int MX = 100000;
2 bool np[MX + 1];
3 vector<int> prime_numbers;
4
5 int init = []() {
6     np[0] = np[1] = true;
7     for (int i = 2; i <= MX; i++) {
8         if (!np[i]) {
9             prime_numbers.push_back(i);
10            for (int j = i; j <= MX / i; j
11                ++){ // 避免溢出的写法
12                np[i * j] = true;
13            }
14        }
15    }
16    return 0;
17 }();

```

```

17 bool is_prime(long long n) {
18     if (n <= MX) {
19         return !np[n];
20     }
21     for (long long p : prime_numbers) {
22         if (p * p > n) {
23             break;
24         }
25         if (n % p == 0) {
26             return false;
27         }
28     }
29     return true;
30 }

```

8.7 mod inv

```

1 // Modular inverse when mod is prime
2 long long mod_inverse(long long a, long long
3     mod) {
4     return power_mod(a, mod - 2, mod); //
5         Fermat's Little Theorem
6 }

```

8.8 derangement (DP)

```

1 // 2. DP
2 // !n = (n - 1) * (!n - 1) + !n - 2),
3 // with !0 = 1, !1 = 0
4
5 mint a = 1, b = 0;
6 cout << 0 << ' ';
7
8 for (int i = 2; i <= n; i++) {
9     mint c = (i - 1) * (a + b);
10    cout << c.val() << ' ';
11    a = b;
12    b = c;
13 }

```

8.9 Euler Totient

```

1 int phi(int n) {
2     int result = n;
3     for (int i = 2; i * i <= n; i++) {
4         if (n % i == 0) {
5             while (n % i == 0)
6                 n /= i;
7             result -= result / i;
8         }
9     }
10    if (n > 1)
11        result -= result / n;
12    return result;
13 }

```

8.10 fast power

```
1 /* Iterative Function to calculate (a^b) %
   mod in O(Log b) */
2 long long power_mod(long long a, long long b
   , long long mod) {
3     long long res{1};
4     while (b > 0) {
5         if (b & 1) res = res * a % mod;
6         b >>= 1;
7         a = (a * a) % mod;
8     }
9     return res;
10 }
```

8.11 first and second mex

```
1 // Calculate first and second MEX
2 pair<int, int> calculate_mexes(vector<int>&
   nums) {
3     int n = nums.size();
4     vector<bool> seen(n + 2, false);
5
6     for (int num : nums) {
7         if (num >= 0 && num < seen.size()) {
8             seen[num] = true;
9         }
10    }
11
12    int first_mex = -1;
13    int second_mex = -1;
14
15    for (int i = 0; i < seen.size(); ++i) {
16        if (!seen[i]) {
17            if (first_mex == -1) {
18                first_mex = i;
19            } else {
20                second_mex = i;
21                break;
22            }
23        }
24    }
25
26    return {first_mex, second_mex};
27 }
```

8.12 Catalan Number

```
1 // Catalan Number
2 // C(n) = 1/(n+1) * (2n choose n) = (2n)! /
   ((n+1)! * n!)
3
4 ll catalan(int n) {
5     return fact[2 * n] * invf[n + 1] % mod *
6         invf[n] % mod;
7 }
```

8.13 Chinese Remainder Theorem

```
1 // coprime (p^k)
2 pair<ll, ll> CRT(const vector<pair<ll, ll>&
   congruences) {
3     ll M = 1, sol = 0;
4     for (auto& [m, a] : congruences) M *= m;
5     for (auto& [m, a] : congruences) {
6         ll x = M / m, y = MI(x, m);
7         sol = MA(sol, MM(MM(a, x, M), y, M),
8             M);
9     }
10    return {M, sol};
11
12    // example usage
13    int main() {
14        // x ≡ 2 (mod 3)
15        // x ≡ 3 (mod 5)
16        // x ≡ 2 (mod 7)
17        vector<pair<ll, ll>> congruences = {{3,
18            2}, {5, 3}, {7, 2}};
19        auto [M, sol] = CRT(congruences);
20        cout << "M: " << M << ", x: " << sol <<
21            endl; // M: 105, x: 23
22    }
```

8.14 mod inv (not prime)

```
1 /* exists when a and mod are coprime */
2 /* but mod is not prime */
3 long long MI(long long a, long long mod) {
4     return power_mod(a, euler_phi(mod) - 1,
5         mod);
6 }
```

8.15 mod inv (not coprime)

```
1 /* a and mod are not coprime */
2 long long MI(long long a, long long mod) {
3     long long d, x, y;
4     extEcu(a, mod, d, x, y);
5     return d == 1 ? (x + mod) % mod : -1;
6 }
```

8.16 Mint

```
1 using ll = long long;
2 const ll PM{1000000007};
3 ll MC(ll a) { return (a % PM + PM) % PM; }
4 // for negative a
5 ll MA(ll a, ll b) { return (a + b) % PM; }
6 // (a + b) % PM
7 ll MS(ll a, ll b) { return (a - b + PM) % PM
8     ; } // (a - b) % PM
```

```
6 ll MM(ll a, ll b) { return (a * b) % PM; }
7 // (a * b) % PM
8 ll MP(ll a, ll b) { // (a ^ b) % PM
9     ll res{1};
10    do {
11        if (b & 1) res = MM(res, a);
12        while (a = MM(a, a), b >>= 1);
13        return res;
14    }
15    MI(ll a) { return MP(a, PM - 2); } //
16    modular inverse of a % PM
17 ll MD(ll a, ll b) { return MM(a, MI(b)); }
18 // (a / b) % PM
```

8.17 inclusion-exclusion principle

```
1 /*
2 inclusion-exclusion principle:  $U A = \sum |A_i| - \sum |A_i \cap A_j| + \sum |A_i \cap A_j \cap A_k| - \dots$ 
3 where  $A_i$  is the set of cows that like flavor i
4
5 if  $f(n) = \sum g(d)$  for all  $d|n$ 
6 then  $g(n) = \sum \mu(d) * f(n/d)$  for all  $d|n$ 
7 where  $\mu(d)$  is the mobius function defined as
8 :
9  $|U A| = \sum \mu(d) * |A_d|$ 
10 where  $A_d = \{x \in U : d|x\}$ 
11  $\mu(d) = 1$  if  $d$  is a product of even number of
12 distinct primes
13  $\mu(d) = -1$  if  $d$  is a product of odd number of
14 distinct primes
15  $\mu(d) = 0$  if  $d$  has a squared prime factor
16 */
17 problem: In number theory we call an integer
18 square-free if it is not divisible by a
19 perfect square, except 1. You have to
20 count them!
21 square-free number: An integer n is square-
22 free if no  $p^2$  divides n.
23
24 /*
25 const int VALMAX = 1e7;
26
27 int mobius[VALMAX];
28
29 int main() {
30     int test_num;
31     cin >> test_num;
32
33     mobius[1] = -1;
34     for (int i = 1; i < VALMAX; i++) {
35         if (mobius[i]) {
36             mobius[i] = -mobius[i];
37             for (int j = 2 * i; j < VALMAX; j += i)
38                 mobius[j] += mobius[i];
39         }
40     }
41
42     for (int t = 0; t < test_num; t++) {
43         long long n;
44         cin >> n;
45         long long ans = 0;
46         for (int i = 1; i <= n; i++) {
47             ans += mobius[i] * n / i;
48         }
49         cout << ans << '\n';
50     }
51 }
```

```
38 long long ans = 0;
39 cin >> n;
40 for (int i = 1; i <= n; i++) {
41     ans += mobius[i] * n / i;
42 }
43 cout << ans << '\n';
44 }
45
46 /*
47 problem: Given N cows (2 ≤ N ≤ 50,000),
48 each listing five favorite ice cream
49 flavors,
50 where two cows are compatible if they share
51 at least one flavor,
52 determine the number of cow pairs that are
53 not compatible.
54
55 ans = n * (n - 1) / 2 -  $\sum |A_i| + \sum |A_i \cap A_j| - \sum |A_i \cap A_j \cap A_k| + \dots$ 
56 where  $A_i$  is the set of cows that like
57 flavor i
58
59 void solve() {
60     int n;
61     cin >> n;
62     vector<array<int, 5>> vc(n);
63
64     map<vector<int>, int> subsets;
65     for (int i = 0; i < n; ++i) {
66         for (int j = 0; j < 5; ++j) {
67             cin >> vc[i][j];
68         }
69
70         sort(vc[i].begin(), vc[i].end());
71         for (int mask = 1; mask < (1<<5); ++
72             mask) {
73             vector<int> subset;
74             for (int b = 0; b < 5; ++b) {
75                 if (mask & (1<<b)) {
76                     subset.push_back(vc[i][b]);
77                 }
78             }
79             subsets[subset]++;
80         }
81     }
82
83     int ans = n * (n - 1) / 2;
84     for (const auto& [subset, freq] :
85         subsets) {
86         ans -= (subset.size() % 2 == 1 ? 1 :
87             -1) * freq * (freq - 1) / 2;
88     }
89     cout << ans << '\n';
90 }
```

8.18 C(n,k) mod inverse

```

1 fac[0] = 1;
2 for (int i = 1; i <= n; ++i) {
3     fac[i] = fac[i - 1] * i % MOD;
4 }
5
6 inv_fac[n] = power_mod(fac[n], MOD - 2, MOD)
7 ;
8 for (int i = n - 1; i >= 0; --i) {
9     inv_fac[i] = inv_fac[i + 1] * (i + 1) %
10     MOD;
11 }
12 // C(n, k) = fac[n] * inv_fac[k] * inv_fac[n
13 - k];

```

8.19 Linear Diophantine 丟番圖

```

1 // ax + by = c
2 // x0 = x * (c / gcd(a, b))
3 // y0 = y * (c / gcd(a, b))
4 // x = x0 + k * (b / gcd(a, b))
5 // y = y0 - k * (a / gcd(a, b))
6 // k is any integer
7
8 t11 extendedEuclid(ll a, ll b) {
9     if (b == 0) return {a, 1, 0};
10    ll d, x1, y1;
11    tie(d, x1, y1) = extendedEuclid(b, a % b
12    );
13    return {d, y1, x1 - (a / b) * y1};
14 }
15
16 p11 linearDiophantine(ll a, ll b, ll c) {
17     ll d, x, y;
18     tie(d, x, y) = extendedEuclid(a, b);
19     if (c % d != 0) return {-1, -1}; // no
20     solution
21     x *= c / d;
22     y *= c / d;
23     return {x, y};
24 }
25 // Example usage
26 int main() {
27     ll a = 15, b = 10, c = 35;
28     p11 res = linearDiophantine(a, b, c);
29     if (res.first == -1 && res.second == -1)
30     {
31         cout << "No solution exists" << endl
32         ;
33     } else {
34         cout << "One solution is x = " <<
35         res.first << ", y = "
36         << res.second << endl;
37     }
38     return 0;
39 }
40 ax + by = c

```

8.20 擴展歐基里德

```

1 /* solve x, y for ax + by = gcd(a, b) = g */
2 template<typename T>
3 void extEcu(T a, T b, T &g, T &x, T &y) {
4     if (b) extEcu(b, a % b, g, y, x), y -= x
5     * (a / b);
6     else g = a, x = 1, y = 0;
7 }

```

8.21 Sieve of Eratosthenes

```

1 void sieve(vector<int>& primes) {
2     vector<int> is_prime(INF + 1, 0);
3     is_prime[0] = 1;
4     is_prime[1] = 1;
5
6     int sq = sqrt(INF);
7
8     for (int i = 2; i <= sq; ++i) {
9         if (!is_prime[i]) {
10            primes.push_back(i);
11            for (int j = i * i; j <= INF; j
12            += i) {
13                is_prime[j] = 1;
14            }
15        }
16    }
17 }

```

8.22 builtin

```

1 // __builtin_ctz
2 // 這個函數作用是返回輸入數二進制表示從最低
3 位開始 ( 右起 ) 的連續的0的個數 ; 如果傳入0
4 則行為未定義。
5
6 int __builtin_ctz (unsigned int x)
7 Returns the number of trailing 0-bits in x,
8 starting at the least significant bit
9 position. If x is 0, the result is
10 undefined.
11
12 int __builtin_ctzl (unsigned long)
13 Similar to __builtin_ctz, except the
14 argument type is unsigned long.
15
16 int __builtin_ctzll (unsigned long long)
17 Similar to __builtin_ctz, except the
18 argument type is unsigned long long.
19
20 // __builtin_clz
21 // 這個函數作用是返回輸入數二進制表示從最高
22 位開始 ( 左起 ) 的連續的0的個數 ; 如果傳入0
23 則行為未定義。
24
25 int __builtin_clz (unsigned int x)

```

18 Returns the number of leading 0-bits in x,
starting at the most significant bit
position. If x is 0, the result is
undefined.

```

19
20 int __builtin_clzl (unsigned long)
21 Similar to __builtin_clz, except the
22 argument type is unsigned long.
23
24 int __builtin_clzll (unsigned long long)
25 Similar to __builtin_clz, except the
26 argument type is unsigned long long.
27
28 // __builtin_ffs
29 // 這個函數作用是返回輸入數二進制表示的最低
30 非0位的下標。下標從1開始計數 ; 如果傳入0
31 則返回0
32
33 int __builtin_ffs (unsigned int x)
34 Returns one plus the index of the least
35 significant 1-bit of x, or if x is zero,
36 returns zero.
37
38 int __builtin_ffsl (unsigned long)
39 Similar to __builtin_ffs, except the
40 argument type is unsigned long.
41
42 int __builtin_ffsll (unsigned long long)
43 Similar to __builtin_ffs, except the
44 argument type is unsigned long long.
45
46 // __builtin_popcount
47 // 這個函數作用是返回輸入數二進制表示中1的個
48 數。
49
50 int __builtin_popcount (unsigned int x)
51 Returns the number of 1-bits in x.
52
53 int __builtin_popcountl (unsigned long)
54 Similar to __builtin_popcount, except the
55 argument type is unsigned long.
56
57 int __builtin_popcountll (unsigned long long)
58 Similar to __builtin_popcount, except the
59 argument type is unsigned long long.
60
61 // __builtin_parity
62 // 這個函數作用是返回輸入數二進制表示中1的個
63 數的奇偶性。1表示奇數個1。0表示偶數個1。
64
65 int __builtin_parity (unsigned int x)
66 Returns the parity of x, i.e. the number of
67 1-bits in x modulo 2.
68
69 int __builtin_parityl (unsigned long)
70 Similar to __builtin_parity, except the
71 argument type is unsigned long.
72
73 int __builtin_parityll (unsigned long long)
74 Similar to __builtin_parity, except the
75 argument type is unsigned long long.

```

8.23 C(n,k) DP

```

1 long long binomial(long long n, long long k,
2 long long p) {
3     // dp[i][j] = iCj
4     vector<vector<long long>> dp(n + 1,
5     vector<long long>(k + 1, 0));
6
7     for (int i = 0; i <= n; ++i) {
8         dp[i][0] = 1;
9         if (i <= k) dp[i][i] = 1;
10    }
11
12    for (int i = 0; i <= n; ++i) {
13        for (int j = 1; j <= min(i, k); ++j) {
14            if (i != j) {
15                dp[i][j] = (dp[i - 1][j - 1]
16                + dp[i - 1][j]) % p;
17            }
18        }
19    }
20
21    return dp[n][k];
22 }

```

9 Range Queries

9.1 subarray maximum sum queries

```

1 // CSES: Subarray Sum Queries
2
3 const int maxn = 2e5;
4 int n = 0;
5 int t[maxn << 1];
6
7 void build() {
8     for (int i = 0; i < n; ++i) {
9         t[i + n] = 0;
10    }
11
12    for (int i = n - 1; i >= 1; --i) {
13        t[i] = t[i << 1] + t[i << 1 | 1];
14    }
15 }
16
17 void update(int idx, int val) {
18     idx += n;
19     if (t[idx] == val) return;
20     t[idx] = val;
21     for (idx >= 1; idx >= 1; idx >= 1) {
22         t[idx] = t[idx << 1] + t[idx << 1 |
23         1];
24     }
25 }
26
27 int query(int l, int r) {
28     int ans = 0;
29     for (l += n, r += n; l < r; l >= 1, r
30     >= 1) {
31         if (l & 1) ans += t[l++];
32     }
33 }

```

```

29     if (r & 1) ans += t[--r];
30 }
31 return ans;
32 }
33
34 void solve() {
35     int q;
36     cin >> q;
37     n = 0;
38     vector<pii> op(q);
39     map<int, int> mp;
40     vector<int> to_val;
41
42     memset(t, 0, sizeof(t));
43
44     for (int i = 0; i < q; ++i) {
45         char c;
46         cin >> c >> op[i].second;
47         if (c == 'I') {
48             op[i].first = 0;
49             mp[op[i].second] = 1;
50         }
51         else if (c == 'D') {
52             op[i].first = 1;
53             mp[op[i].second] = 1;
54         }
55         else if (c == 'K') {
56             op[i].first = 2;
57         }
58         else {
59             op[i].first = 3;
60         }
61     }
62
63     for (auto& x : mp) {
64         x.second = n;
65         to_val.push_back(x.first);
66         n++;
67     }
68
69     build();
70
71     for (pii o : op) {
72         if (o.first == 0) {
73             int idx = mp[o.second];
74             update(idx, 1);
75         }
76         if (o.first == 1) {
77             int idx = mp[o.second];
78             update(idx, 0);
79         }
80         if (o.first == 2) {
81             int k = o.second;
82             int ans = -1;
83             int l = 0, r = n - 1;
84             while (l <= r) {
85                 int m = l + (r - l) / 2;
86                 int rk = query(0, m + 1);
87                 if (rk >= k) {
88                     ans = m;
89                     r = m - 1;
90                 }
91                 else {
92                     l = m + 1;
93                 }
94             }
95         }
96     }

```

```

95     if (ans == -1) cout << "invalid\n";
96     else cout << to_val[ans] << "\n";
97 }
98
99 if (o.first == 3) {
100     auto it = mp.lower_bound(o.second);
101     if (it == mp.begin() && (it == mp.end() || o.second <= it->first)) {
102         cout << "0\n";
103     }
104     else if (it == mp.end()) {
105         cout << query(0, n) << "\n";
106     }
107     else {
108         cout << query(0, it->second) << "\n";
109     }
110 }
111 }
112 }

```

9.2 find first equal or greater

```

1 // CSE: Hotel Queries
2
3 int n, m;
4 int sz = 1;
5 const int maxn = 2e6;
6 int t[maxn << 1];
7
8 void build(const vector<int>& a) {
9     for (int i = 0; i < n; ++i) {
10         t[i + sz] = a[i];
11     }
12     for (int i = sz - 1; i >= 1; --i) {
13         t[i] = max(t[i << 1], t[i << 1 | 1]);
14     }
15 }
16
17 void update(int i, int val) {
18     i += sz;
19     t[i] = val;
20     for (i >= 1; i >= 1; i >= 1) {
21         t[i] = max(t[i << 1], t[i << 1 | 1]);
22     }
23 }
24
25 int query(int r) {
26     if (t[1] < r) return 0;
27
28     int i = 1;
29     while (i < sz) {
30         if (t[i << 1] >= r) i = i << 1;
31         else i = i << 1 | 1;
32     }
33     return i - sz + 1;
34 }

```

```

35 void solve() {
36     cin >> n >> m;
37     vector<int> a(n);
38
39     for (int i = 0; i < n; ++i) {
40         cin >> a[i];
41     }
42
43     while (sz < n) sz <= 1;
44     memset(t, 0, sizeof(t));
45     build(a);
46
47     int r;
48     while (m--) {
49         cin >> r;
50         int idx = query(r);
51         cout << idx << " ";
52         if (idx) {
53             a[idx - 1] -= r;
54             update(idx - 1, a[idx - 1]);
55         }
56     }
57     cout << "\n";
58 }
59 }

```

9.3 find k-th and count less than

```

1 /*
2 SPOJ: ORDERSET - Order statistic set
3
4 In this problem, you have to maintain a
5 dynamic set of numbers which support the
6 two fundamental operations
7
8 INSERT(S,x): if x is not in S, insert x into S
9 DELETE(S,x): if x is in S, delete x from S
10 and the two type of queries
11
12 K-TH(S) : return the k-th smallest element of S
13 COUNT(S,x): return the number of elements of S smaller than x
14 */
15
16 const int maxn = 2e5;
17 int n = 0;
18 int t[maxn << 1];
19
20 void build() {
21     for (int i = 0; i < n; ++i) {
22         t[i + n] = 0;
23     }
24     for (int i = n - 1; i >= 1; --i) {
25         t[i] = t[i << 1] + t[i << 1 | 1];
26     }
27 }
28
29 void update(int idx, int val) {
30     idx += n;
31     if (t[idx] == val) return;
32     t[idx] = val;
33 }

```

```

31 for (idx >= 1; idx >= 1; idx >= 1) {
32     t[idx] = t[idx << 1] + t[idx << 1 | 1];
33 }
34 }
35
36 int query(int l, int r) {
37     int ans = 0;
38     for (l += n, r += n; l < r; l >= 1, r >= 1) {
39         if (l & 1) ans += t[l++];
40         if (r & 1) ans += t[--r];
41     }
42     return ans;
43 }
44
45 void solve() {
46     int q;
47     cin >> q;
48     n = 0;
49     vector<pii> op(q);
50     map<int, int> mp;
51     vector<int> to_val;
52
53     memset(t, 0, sizeof(t));
54
55     for (int i = 0; i < q; ++i) {
56         char c;
57         cin >> c >> op[i].second;
58         if (c == 'I') {
59             op[i].first = 0;
60             mp[op[i].second] = 1;
61         }
62         else if (c == 'D') {
63             op[i].first = 1;
64             mp[op[i].second] = 1;
65         }
66         else if (c == 'K') {
67             op[i].first = 2;
68         }
69         else {
70             op[i].first = 3;
71         }
72     }
73
74     for (auto& x : mp) {
75         x.second = n;
76         to_val.push_back(x.first);
77         n++;
78     }
79
80     build();
81
82     for (pii o : op) {
83         if (o.first == 0) {
84             int idx = mp[o.second];
85             update(idx, 1);
86         }
87         if (o.first == 1) {
88             int idx = mp[o.second];
89             update(idx, 0);
90         }
91         if (o.first == 2) {
92             int k = o.second;
93             int ans = -1;
94             int l = 0, r = n - 1;

```



```

95 while (l <= r) {
96     int m = l + (r - l) / 2;
97     int rk = query(0, m + 1);
98     //printf("(m, rk): %lld, %
100     //lld\n", m, rk);
101     if (rk >= k) {
102         ans = m;
103         r = m - 1;
104     }
105     else {
106         l = m + 1;
107     }
108     if (ans == -1) cout << "invalid\
109     n";
110     else cout << to_val[ans] << "\n"
111     ;
112     if (o.first == 3) {
113         auto it = mp.lower_bound(o.
114         second);
115         if (it == mp.begin() && (it ==
116         mp.end() || o.second <= it->
117         first)) {
118             cout << "0\n";
119         }
120         else if (it == mp.end()) {
121             cout << query(0, n) << "\n";
122         }
123         else {
124             cout << query(0, it->second)
125             << "\n";
126         }
127     }
128 }

```

10 String

10.1 Z

```

1 // 計算并返回 z 数组 · 其中 z[i] = |LCP(s[i
2 :j], s)|
3 vector<int> calc_z(const string& s) {
4     int n = s.size();
5     vector<int> z(n);
6     int box_l = 0, box_r = 0;
7     for (int i = 1; i < n; i++) {
8         if (i <= box_r) {
9             z[i] = min(z[i - box_l], box_r -
10             i + 1);
11         }
12         while (i + z[i] < n && s[z[i]] == s[
13             i + z[i]]) {
14             box_l = i;
15             box_r = i + z[i];
16             z[i]++;
17         }
18     }
19     z[0] = n;

```

```

17 return z;
18 }

```

10.2 manacher

```

1 class Solution {
2 public:
3     int countSubstrings(string s) {
4         int l1 = s.size(), l2 = l1 * 2 + 1;
5         string ch = "#";
6         for(char c: s) {
7             ch = ch + c + "#";
8         }
9
10        int c = 0, r = 0, cnt = 0;
11        vector<int> p(l2);
12        for(int i = 0; i < l2; i++) {
13            p[i] = (i < r)? min(p[2 * c - i
14            ], r - i): 1;
15            while (i + p[i] < l2 && i - p[i]
16            >= 0 && ch[i + p[i]] == ch[i
17            - p[i]]) p[i]++;
18            if(i + p[i] > r) {
19                r = i + p[i];
20                c = i;
21            }
22            int l = p[i] - 1;
23            if(l % 2 == 0) cnt += 1 / 2;
24            else cnt += 1 / 2 + 1;
25        }
26        return cnt;
27    }
28 };

```

10.3 AC 自動機

```

1 template<char L='a',char R='z'>
2 class ac_automaton{
3     struct joe{
4         int next[R-L+1],fail,efl,ed,cnt_dp,vis;
5         joe():ed(0),cnt_dp(0),vis(0){
6             for(int i=0;i<=R-L;++i)next[i]=0;
7         }
8     };
9     public:
10        std::vector<joe> S;
11        std::vector<int> q;
12        int qs,qe,vt;
13        ac_automaton():S(1),qs(0),qe(0),vt(0){
14            void clear(){
15                q.clear();
16                S.resize(1);
17                for(int i=0;i<=R-L;++i)S[0].next[i]=0;
18                S[0].cnt_dp=S[0].vis=qs=qe=vt=0;
19            }
20            void insert(const char *s){
21                int o=0;
22                for(int i=0,id;s[i];++i){
23                    id=s[i]-L;
24                    if(!S[o].next[id]){

```

```

25        S.push_back(joe());
26        S[o].next[id]=S.size()-1;
27    }
28    o=S[o].next[id];
29    }
30    ++S[o].ed;
31    }
32    void build_fail(){
33        S[0].fail=S[0].efl=-1;
34        q.clear();
35        q.push_back(0);
36        ++qe;
37        while(qs!=qe){
38            int pa=q[qs++],id,t;
39            for(int i=0;i<=R-L;++i){
40                t=S[pa].next[i];
41                if(!t)continue;
42                id=S[pa].fail;
43                while(~id&&!S[id].next[i])id=S[id].
44                fail;
45                S[t].fail=~id?S[id].next[i]:0;
46                S[t].efl=S[S[t].fail].ed?S[t].fail:S
47                [S[t].fail].efl;
48                q.push_back(t);
49                ++qe;
50            }
51        }
52        /*DP 出每個前綴在字串s出現的次數並傳回所有
53        字串被s匹配成功的次數O(N*M)*/
54        int match_0(const char *s){
55            int ans=0,id,p=0,i;
56            for(i=0;s[i];++i){
57                id=s[i]-L;
58                while(!S[p].next[id]&&p=S[p].fail;
59                if(!S[p].next[id])continue;
60                p=S[p].next[id];
61                ++S[p].cnt_dp; /*匹配成功則它所有後綴都
62                可以被匹配(DP計算)*/
63            }
64            for(i=qe-1;i>=0;--i){
65                ans+=S[q[i]].cnt_dp*S[q[i]].ed;
66                if(~S[q[i]].fail)S[S[q[i]].fail].
67                cnt_dp+=S[q[i]].cnt_dp;
68            }
69            return ans;
70        }
71        int match_1(const char *s)const{
72            int ans=0,id,p=0,t;
73            for(int i=0;s[i];++i){
74                id=s[i]-L;
75                while(!S[p].next[id]&&p=S[p].fail;
76                if(!S[p].next[id])continue;
77                p=S[p].next[id];
78                if(S[p].ed)ans+=S[p].ed;
79                for(t=S[p].efl;~t;t=S[t].efl){
80                    ans+=S[t].ed; /*因為都走efl邊所以保證
81                    匹配成功*/
82                }
83            }
84            return ans;
85        }
86    };

```

```

82 /*枚舉(s的子串nA)的所有相異字串各恰一次
83 並傳回次數O(N*M^(1/3))*/
84 int match_2(const char *s){
85     int ans=0,id,p=0,t;
86     ++vt;
87     /*把戳記vt+=1 · 只要vt沒溢位 · 所有S[p].
88     vis==vt就會變成false
89     這種利用vt的方法可以O(1)歸零vis陣列*/
90     for(int i=0;s[i];++i){
91         id=s[i]-L;
92         while(!S[p].next[id]&&p=S[p].fail;
93         if(!S[p].next[id])continue;
94         p=S[p].next[id];
95         if(S[p].ed&&S[p].vis!=vt){
96             S[p].vis=vt;
97             ans+=S[p].ed;
98         }
99         for(t=S[p].efl;~t&&S[t].vis!=vt;t=S[t].
100         efl){
101             S[t].vis=vt;
102             ans+=S[t].ed; /*因為都走efl邊所以保證
103             匹配成功*/
104         }
105     }
106     return ans;
107 }
108 /*把AC自動機變成真的自動機*/
109 void evolution(){
110     for(qs=1;qs!=qe;){
111         int p=q[qs++];
112         for(int i=0;i<=R-L;++i)
113             if(S[p].next[i]==0)S[p].next[i]=S[S[
114             p].fail].next[i];
115     }
116 }

```

10.4 KMP

```

1 // 在文本串 text 中查找模式串 pattern · 返回
2 所有成功匹配的位置 (pattern[0] 在 text
3 中的下標)
4 vector<int> kmp(const string& text, const
5 string& pattern) {
6     int m = pattern.size();
7     vector<int> pi(m);
8     int cnt = 0;
9     for (int i = 1; i < m; i++) {
10         char b = pattern[i];
11         while (cnt && pattern[cnt] != b) {
12             cnt = pi[cnt - 1];
13         }
14         if (pattern[cnt] == b) {
15             cnt++;
16         }
17         pi[i] = cnt;
18     }
19     vector<int> pos;
20     cnt = 0;
21     for (int i = 0; i < text.size(); i++) {

```

```

20 char b = text[i];
21 while (cnt && pattern[cnt] != b) {
22     cnt = pi[cnt - 1];
23 }
24 if (pattern[cnt] == b) {
25     cnt++;
26 }
27 if (cnt == m) {
28     pos.push_back(i - m + 1);
29     cnt = pi[cnt - 1];
30 }
31 }
32 return pos;
33 }

```

10.5 suffix array lcp

```

1 #define radix_sort(x,y){\
2     for(i=0;i<A;++i)c[i]=0;\
3     for(i=0;i<n;++i)c[x[y[i]]]++;\
4     for(i=1;i<A;++i)c[i]+=c[i-1];\
5     for(i=n-1;~i;--i)sa[--c[x[y[i]]]]=y[i];\
6 }
7 #define AC(r,a,b)\
8     r[a]!=(r[b]|(a+k)>n)?r[a+k]:r[b+k]
9 void suffix_array(const char *s,int n,int *
10 sa,int *rank,int *tmp,int *c){
11     int A='z'+1,i,k,id=0;
12     for(i=0;i<n;++i)rank[tmp[i]=i]=s[i];
13     radix_sort(rank,tmp);
14     for(k=1;id<n-1;k<=1){
15         for(id=0,i=n-k;i<n;++i)tmp[id++]=i;
16         for(i=0;i<n;++i)
17             if(sa[i]>k)tmp[id++]=sa[i]-k;
18         radix_sort(rank,tmp);
19         swap(rank,tmp);
20         for(rank[sa[0]]=id=0,i=1;i<n;++i)
21             rank[sa[i]]=id+=AC(tmp,sa[i-1],sa[i]);
22         A=id+1;
23     }
24 }
25 //h: 高度數組 sa: 後綴數組 rank: 排名
26 void suffix_array_lcp(const char *s,int len,
27 int *h,int *sa,int *rank){
28     for(int i=0;i<len;++i)rank[sa[i]]=i;
29     for(int i=0,k=0;i<len;++i){
30         if(rank[i]==0)continue;
31         if(k)--k;
32         while(s[i+k]==s[sa[rank[i]-1]+k])++k;
33         h[rank[i]]=k;
34     }
35 }

```

10.6 hash

```

1 #define MAXN 1000000
2 #define mod 1073676287
3 /*mod 必須要是質數*/

```

```

4 typedef long long T;
5 char s[MAXN+5];
6 T h[MAXN+5];/*hash陣列*/
7 T h_base[MAXN+5];/*h_base[n]=(prime^n)%mod*/
8 void hash_init(int len,T prime){
9     h_base[0]=1;
10    for(int i=1;i<=len;++i){
11        h[i]=(h[i-1]*prime+s[i-1])%mod;
12        h_base[i]=(h_base[i-1]*prime)%mod;
13    }
14 }
15 T get_hash(int l,int r){/*閉區間寫法·設編號
16    為0 ~ len-1*/
17    return (h[r+1]-(h[l]*h_base[r-l+1])%mod+
18        mod)%mod;
19 }

```

10.7 minimal string rotation

```

1 int min_string_rotation(const string &s){
2     int n=s.size(),i=0,j=1,k=0;
3     while(i<n&&j<n&&k<n){
4         int t=s[(i+k)%n]-s[(j+k)%n];
5         ++k;
6         if(t>0)i+=k;
7         else j+=k;
8         if(i==j)++j;
9         k=0;
10    }
11    return min(i,j);/*最小循環表示法起始位置
12 }

```

10.8 reverseBWT

```

1 const int MAXN = 305, MAXC = 'Z';
2 int ranks[MAXN], tots[MAXC], first[MAXN];
3 void rankBWT(const string &bw){
4     memset(ranks,0,sizeof(int)*bw.size());
5     memset(tots,0,sizeof(tots));
6     for(size_t i=0;i<bw.size();++i)
7         ranks[i] = tots[int(bw[i])]++;
8 }
9 void firstCol(){
10    memset(first,0,sizeof(first));
11    int totc = 0;
12    for(int c='A';c<='Z';++c){
13        if(!tots[c]) continue;
14        first[c] = totc;
15        totc += tots[c];
16    }
17 }
18 string reverseBwt(string bw,int begin){
19     rankBWT(bw), firstCol();
20     int i = begin; /*原字串最後一個元素的位置
21     string res;
22     do{
23         char c = bw[i];

```

```

24     res = c + res;
25     i = first[int(c)] + ranks[i];
26 }while (i != begin );
27 return res;
28 }

```

11 Tree

11.1 Euler tour+RMQ

```

1 // Euler Tour Technique
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d, first, euler{}, log2{};
6     vector<vector<int>> st{};
7     void dfs(int u, int w = -1, int dep = 0)
8     {
9         d[u] = dep;
10        first[u] = euler.size();
11        euler.push_back(u);
12        for (auto& v : adj[u]) {
13            if (v == w) continue;
14            dfs(v, u, dep + 1);
15            euler.push_back(u);
16        }
17    public:
18        LCA(const vector<vector<int>>& _adj, int
19            root) : adj{_adj}, n{adj.size()}, d
20            (n), first(n) {
21            dfs(root);
22
23            int tn{euler.size()};
24            log2.resize(tn + 1);
25            log2[1] = 0;
26            for (int i{2}; i <= tn; ++i) log2[i]
27                = log2[i / 2] + 1;
28
29            st.assign(tn, vector<int>(log2[tn] +
30                1));
31            for (int i{tn - 1}; i >= 0; --i) {
32                st[i][0] = euler[i];
33                for (int j{1}; i + (1 << j) <=
34                    tn; ++j) {
35                    auto& x{st[i][j - 1]};
36                    auto& y{st[i + (1 << (j - 1)
37                        )][j - 1]};
38                    st[i][j] = d[x] <= d[y] ? x
39                        : y;
40                }
41            }
42            int operator()(int u, int v) {
43                int l{first[u]}, r{first[v]};
44                if (l > r) swap(l, r);
45                ++r; /* make the interval left
46                    closed right open
47
48                int j{log2[r - l]};
49                auto& x{st[l][j]};

```

```

43         auto& y{st[r - (1 << j)][j]};
44         return d[x] <= d[y] ? x : y;
45     }
46 };
47
48 int main() {
49     int n, q;
50     cin >> n >> q;
51     vector<vector<int>> adj(n);
52
53     for (int i = 1; i < n; ++i) {
54         int u;
55         cin >> u;
56         u--;
57         adj[u].pb(i);
58         adj[i].pb(u);
59     }
60
61     LCA lca(adj, 0);
62
63     while (q--) {
64         int u, v;
65         cin >> u >> v;
66         u--, v--;
67         cout << lca(u, v) + 1 << "\\n";
68     }
69     return 0;
70 }

```

11.2 HLD template

```

1 struct HLD {
2     int n, cur = 0;
3     vector<int> sz, top, dep, par, tin, tout
4         , seq;
5     vector<vector<int>> adj;
6     HLD(int n = 0) {
7         init(n);
8     }
9     void init(int n_) {
10        n = n_;
11        sz.assign(n, 1);
12        top.assign(n, -1);
13        dep.assign(n, 0);
14        par.assign(n, -1);
15        tin.assign(n, 0);
16        tout.assign(n, 0);
17        seq.assign(n, 0);
18        adj.resize(n, {});
19    }
20    void add_edge(int u, int v) { adj[u].
21        push_back(v), adj[v].push_back(u); }
22    void build(int root = 0) {
23        top[root] = root, dep[root] = 0, par
24            [root] = -1;
25        dfs1(root), dfs2(root);
26    }
27    void dfs1(int u) {
28        if (auto it = find(adj[u].begin(),
29            adj[u].end(), par[u]); it != adj
30            [u].end()) {
31            adj[u].erase(it);
32        }
33    }

```

```

28     for (auto &v : adj[u]) {
29         par[v] = u;
30         dep[v] = dep[u] + 1;
31         dfs1(v);
32         sz[u] += sz[v];
33         if (sz[v] > sz[adj[u][0]]) {
34             swap(v, adj[u][0]);
35         }
36     }
37 void dfs2(int u) {
38     tin[u] = cur++;
39     seq[tin[u]] = u;
40     for (auto v : adj[u]) {
41         top[v] = v == adj[u][0] ? top[u]
42             : v;
43         dfs2(v);
44     }
45     tout[u] = cur;
46 }
47 int lca(int u, int v) {
48     while (top[u] != top[v]) {
49         if (dep[top[u]] > dep[top[v]]) {
50             u = par[top[u]];
51         } else {
52             v = par[top[v]];
53         }
54     }
55     return dep[u] < dep[v] ? u : v;
56 }
57 int dist(int u, int v) { return dep[u] +
58     dep[v] - 2 * dep[lca(u, v)]; }
59 int jump(int u, int k) {
60     if (dep[u] < k) { return -1; }
61     int d = dep[u] - k;
62     while (dep[top[u]] > d) { u = par[
63         top[u]]; }
64     return seq[tin[u] - dep[u] + d];
65 }
66 // u is v's ancestor
67 bool is_ancestor(int u, int v) { return
68     tin[u] <= tin[v] && tin[v] < tout[u]
69     ]; }
70 // root's parent is itself
71 int rooted_parent(int r, int u) {
72     if (r == u) { return u; }
73     if (is_ancestor(r, u)) { return par[
74         u]; }
75     auto it = upper_bound(adj[u].begin(),
76         adj[u].end(), r, [&](int x,
77             int y) {
78         return tin[x] < tin[y];
79     }) - 1;
80     return *it;
81 }
82 // rooted at u, v's subtree size
83 int rooted_size(int r, int u) {
84     if (r == u) { return n; }
85     if (is_ancestor(u, r)) { return sz[u]
86         ]; }
87     return n - sz[rooted_parent(r, u)];
88 }
89 int rooted_lca(int r, int a, int b) {
90     return lca(a, b) ^ lca(a, r) ^ lca(b
91         , r); }
92 };

```

```

82 int main() {
83     ios::sync_with_stdio(false);
84     cin.tie(nullptr);
85
86     int n = 7;
87     HLD hld(n);
88
89     // build this tree:
90     //      0
91     //    / \
92     //   1  2
93     //  / \  \
94     // 3  4  5
95     //  //  \
96     // 6
97     hld.add_edge(0, 1);
98     hld.add_edge(0, 2);
99     hld.add_edge(1, 3);
100    hld.add_edge(1, 4);
101    hld.add_edge(2, 5);
102    hld.add_edge(5, 6);
103
104    hld.build(0); // root at node 0
105
106    cout << "LCA(3,4) = " << hld.lca(3,4) <<
107        "\n"; // expected 1
108    cout << "LCA(3,6) = " << hld.lca(3,6) <<
109        "\n"; // expected 0
110    cout << "Distance(3,4) = " << hld.dist
111        (3,4) << "\n"; // expected 2
112    cout << "Distance(3,6) = " << hld.dist
113        (3,6) << "\n"; // expected 4
114    cout << "Is 0 ancestor of 6? " << (hld.
115        is_ancestor(0,6) ? "yes" : "no") <<
116        "\n"; // yes
117
118    return 0;
119 }

```

11.3 dist between u-v in tree

```

1 int dist(int a, int b) {
2     int c = lca(a, b);
3     return depth[a] + depth[b] - 2 * depth[c]
4     ];

```

11.4 all longest path dfs

```

1 // all longest path (generalization of the
2   tree diameter problem)
3 vector<tuple<int, int, int>> dp{};
4 // [mx1, x, mx2] the path of mx1 goes
5   through x
6 int dfs1(int u, int p = -1) {
7     int mx = 0;
8     for (int v : adj[u]) {
9         if (v != p) {
10             int len = 1 + dfs1(v, u);

```

```

10     mx = max(mx, len);
11
12     auto& [mx1, x, mx2] = dp[u];
13     if (len >= mx1) {
14         mx2 = mx1;
15         mx1 = len;
16         x = v;
17     }
18     else if (len > mx2) {
19         mx2 = len;
20     }
21 }
22 return mx;
23 }
24
25 void dfs2(int u, int p = -1) {
26     if (p != -1) {
27         int tmx;
28         { // calculate the longest path
29             through parent
30             auto& [mx1, x, mx2] = dp[p];
31             if (x != u) tmx = mx1 + 1;
32             else tmx = mx2 + 1;
33         }
34         { // update the path
35             auto& [mx1, x, mx2] = dp[u];
36             if (tmx >= mx1) {
37                 mx2 = mx1;
38                 mx1 = tmx;
39                 x = p;
40             }
41             else if (tmx > mx2) {
42                 mx2 = tmx;
43             }
44         }
45     }
46     for (int v : adj[u]) {
47         if (v != p) {
48             dfs2(v, u);
49         }
50     }
51 }
52
53 void all_longest_path() {
54     dp.resize(n);
55     dfs1(0), dfs2(0);
56 }

```

11.5 all longest path top sort

```

1 // all longest path in DAG
2 // 1. topological sort
3 vector<int> in(n, 0);
4 for (int i = 0; i < m; ++i) {
5     int a, b, w;
6     cin >> a >> b >> w;
7     adj[a].emplace_back(b, w);
8     in[b]++;
9 }
10
11 vector<int> topo; // sequence of top sort
12 queue<int> q;

```

```

13 for (int i = 0; i < n; ++i) {
14     if (in[i] == 0) {
15         q.push(i);
16     }
17 }
18 while (!q.empty()) {
19     int pa = q.front();
20     q.pop();
21     topo.push_back(pa);
22     for (auto& [child, w] : adj[pa]) {
23         in[child]--;
24         if (in[child] == 0) {
25             q.push(child);
26         }
27     }
28 }
29
30 // all Longest path
31 vector<int> dist(n, INT_MIN);
32 vector<vector<int>> parents(n);
33 dist[0] = process[0];
34
35 for (int u : topo) {
36     for (auto& [v, w] : adj[u]) {
37         if (dist[v] < dist[u] + process[v] +
38             w) {
39             dist[v] = dist[u] + process[v] +
40                 w;
41             parents[v] = {u};
42             else if (dist[v] == dist[u] +
43                 process[v] + w) {
44                 parents[v].push_back(u);
45             }
46         }
47     }
48 }
49 cout << dist[n - 1];

```

11.6 Heavy-light decomposition (HLD)

```

1 const int maxn = 2e5;
2 int n, q;
3 int val[maxn];
4 int id[maxn];
5 int sz[maxn];
6 int parent[maxn];
7 int depth[maxn];
8 int tp[maxn];
9 vector<int> adj[maxn];
10 int timer = 0;
11
12 int dfs_sz(int u, int p) {
13     sz[u] = 1;
14     parent[u] = p;
15
16     // calculate size of each subtree
17     for (int v : adj[u]) {
18         if (v != p) {
19             depth[v] = depth[u] + 1;
20             sz[u] += dfs_sz(v, u);

```

```

21     }
22 }
23 return sz[u];
24 }
25
26 void dfs_hld(int u, int p, int top, SGT<int,
27 MergeMax>& tree) {
28     id[u] = timer++;
29     tp[u] = top;
30     tree.update(id[u], val[u]);
31
32     // find the heavy child
33     int h_v = -1, h_sz = -1;
34     for (int v : adj[u]) {
35         if (v != p) {
36             if (h_sz < sz[v]) {
37                 h_sz = sz[v];
38                 h_v = v;
39             }
40         }
41     }
42     if (h_v == -1) {
43         return;
44     }
45     // continue the chain
46     dfs_hld(h_v, u, top, tree);
47     // start a new chain for each light
48     // child
49     for (int v : adj[u]) {
50         if (v != p && v != h_v) {
51             dfs_hld(v, u, v, tree);
52         }
53     }
54 }

```

```

55 int path(int x, int y, SGT<int, MergeMax>&
56 tree) {
57     int ans = 0;
58     // move x and y to their top nodes until
59     // they are in the same chain
60     while (tp[x] != tp[y]) {
61         if (depth[tp[x]] < depth[tp[y]])
62             swap(x, y);
63         ans = max(ans, tree.query(id[tp[x]],
64             id[x] + 1));
65         x = parent[tp[x]];
66     }
67     if (depth[x] > depth[y]) swap(x, y);
68     ans = max(ans, tree.query(id[x], id[y] +
69         1));
70     return ans;
71 }
72
73 void solve() {
74     cin >> n >> q;
75     SGT<int, MergeMax> tree(n, -1);
76
77     for (int i = 0; i < n; ++i) {
78         cin >> val[i];
79     }
80     for (int i = 0; i < n - 1; ++i) {
81         int a, b;
82         cin >> a >> b;
83         a--, b--;
84         adj[a].push_back(b);
85     }
86 }

```

```

87     adj[b].push_back(a);
88 }
89
90 dfs_sz(0, 0);
91 dfs_hld(0, 0, 0, tree);
92
93 cerr << "tp: ";
94 for (int i = 0; i < n; ++i) {
95     cerr << tp[i] << " ";
96 }
97 cerr << "\n";
98
99 while (q--) {
100     int mode;
101     cin >> mode;
102     if (mode == 1) {
103         int s, x;
104         cin >> s >> x;
105         s--;
106         val[s] = x;
107         tree.update(id[s], val[s]);
108     }
109     else {
110         int a, b;
111         cin >> a >> b;
112         cout << path(a - 1, b - 1, tree)
113             << "\n";
114     }
115 }

```

11.7 binary lifting

```

1 // binary lifting
2 class LCA {
3     const vector<vector<int>>& adj;
4     int n;
5     vector<int> d;
6     vector<int> log2;
7     vector<vector<int>> an{};
8     void dfs(int u, int w = -1, int dep = 0) {
9         d[u] = dep;
10        an[u][0] = w;
11        for (int i{1}; i <= log2[n - 1] &&
12            an[u][i - 1] != -1; ++i)
13            an[u][i] = an[an[u][i - 1]][i -
14                1]; // 走 2^(i-1) 再走 2^(i-1) 步
15
16        // 因為計算 an 會用到祖先的資訊，所以
17        // 先計算再繼續往下
18        for (auto& v : adj[u]) {
19            if (v == w) continue; // parent
20            dfs(v, u, dep + 1);
21        }
22    public:
23        LCA(const vector<vector<int>>& _adj, int
24            root)
25            : adj{_adj}, n{adj.size()}, d(n), log2(n)
26            ) {}

```

```

27     log2[1] = 0;
28     for (int i{2}; i < log2.size(); ++i)
29         log2[i] = log2[i / 2] + 1;
30     an.assign(n, vector<int>(log2[n - 1]
31         + 1, -1));
32     dfs(root);
33 }
34
35 int operator()(int u, int v) {
36     if (d[u] > d[v]) swap(u, v);
37
38     for (int i{log2[d[v] - d[u]]}; i >=
39         0; --i)
40         if (d[v] - d[u] >= (1 << i)) v =
41             an[v][i];
42     // v 先走到跟 u 同高度
43     if (u == v) return u;
44
45     for (int i{log2[d[u]]}; i >= 0; --i)
46         if (an[u][i] != an[v][i]) u = an
47             [u][i], v = an[v][i];
48     // u, v 一起走到 Lca(u, v) 的下方
49
50     return an[u][0];
51     // 回傳 Lca(u, v)
52 }
53
54 int main() {
55     int n, q;
56     cin >> n >> q;
57     vector<vector<int>> adj(n);
58
59     for (int i = 1; i < n; ++i) {
60         int u;
61         cin >> u;
62         u--;
63         adj[u].pb(i);
64         adj[i].pb(u);
65     }
66
67     // adj, root
68     LCA lca(adj, 0);
69
70     while (q--) {
71         int u, v;
72         cin >> u >> v;
73         u--, v--;
74         cout << lca(u, v) + 1 << "\n";
75     }
76     return 0;
77 }

```

11.8 get kth ancestor

```

1 void dfs(int u, int p = -1) {
2     up[0][u] = (p == -1 ? u : p);
3     for (int v : adj[u]) {
4         if (v != p) {
5             dfs(v, u);
6         }
7     }
8 }

```

```

9 int get_kth_ancestor(int x, int k) {
10     for (int i = 0; i <= LOG; ++i) {
11         if (k & (1LL << i)) {
12             x = up[i][x];
13         }
14     }
15     return (x == 0 ? -1 : x);
16 }
17
18 void solve() {
19     int n, q;
20     cin >> n >> q;
21
22     adj[0].push_back(1);
23     for (int i = 2; i <= n; ++i) {
24         int u;
25         cin >> u;
26         adj[u].push_back(i);
27     }
28     dfs(0, -1);
29
30     for (int k = 1; k <= LOG; ++k) {
31         for (int v = 1; v <= n; ++v) {
32             up[k][v] = up[k - 1][up[k - 1][v]
33                 ];
34         }
35     }
36
37     while (q--) {
38         int x, k;
39         cin >> x >> k;
40         cout << get_kth_ancestor(x, k) << "\n";
41     }
42 }
43 }

```

11.9 check z on path u-v

```

1 bool on_path(int z, int u, int v) {
2     return dist(u, z) + dist(z, v) == dist(u,
3         v);
4 }

```

11.10 path maximum edge between u, v in tree

```

1 int depth[maxn];
2 int up[LOG][maxn];
3 int maxEdge[LOG][maxn];
4
5 void dfs(int u, int p = -1, int w = 0) {
6     up[0][u] = (p == -1 ? u : p);
7     maxEdge[0][u] = (p == -1 ? 0 : w);
8     for (auto &[v, rw] : adj[u]) {
9         if (v != p) {
10             depth[v] = depth[u] + 1;
11             dfs(v, u, rw);
12         }
13     }
14 }

```

```

12     }
13 }
14 }
15
16 int lca_weight(int a, int b) {
17     if (depth[a] < depth[b]) swap(a, b);
18     int diff = depth[a] - depth[b];
19     int ans = 0;
20
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) {
23             ans = max(ans, maxEdge[k][a]);
24             a = up[k][a];
25         }
26         diff >>= 1;
27     }
28     if (a == b) {
29         return ans;
30     }
31     for (int k = LOG - 1; k >= 0; --k) {
32         if (up[k][a] != up[k][b]) {
33             ans = max(ans, maxEdge[k][a]);
34             ans = max(ans, maxEdge[k][b]);
35             a = up[k][a];
36             b = up[k][b];
37         }
38     }
39
40     ans = max(ans, maxEdge[0][a]);
41     ans = max(ans, maxEdge[0][b]);
42     return ans;
43 }
44
45 void solve() {
46     dfs(0, -1, 0);
47     for (int k = 1; k < LOG; ++k) {
48         for (int v = 0; v < n; ++v) {
49             up[k][v] = up[k - 1][up[k - 1][v]];
50             maxEdge[k][v] = max(maxEdge[k - 1][v], maxEdge[k - 1][up[k - 1][v]]);
51         }
52     }
53     int path_max = lca_weight(0, 1);
54 }

```

11.11 binary jumping

```

1 const int LOG = 32;
2 const int maxn = 2e5 + 2;
3
4 int depth[maxn];
5 int up[LOG][maxn];
6 vector<int> adj[maxn];
7
8 void dfs(int u, int p = -1) {
9     up[0][u] = (p == -1 ? u : p);
10    for (int v : adj[u]) {
11        if (v != p) {
12            depth[v] = depth[u] + 1;
13            dfs(v, u);
14        }
15    }
16 }

```

```

15     }
16 }
17
18 int lca(int a, int b) {
19     if (depth[a] < depth[b]) swap(a, b);
20     int diff = depth[a] - depth[b];
21     for (int k = 0; diff; ++k) {
22         if (diff & 1) a = up[k][a];
23         diff >>= 1;
24     }
25     if (a == b) return a;
26     for (int k = LOG - 1; k >= 0; --k) {
27         if (up[k][a] != up[k][b]) {
28             a = up[k][a];
29             b = up[k][b];
30         }
31     }
32     return up[0][a];
33 }
34
35 void solve() {
36     ios::sync_with_stdio(false);
37     cin.tie(nullptr);
38
39     int n, q;
40     cin >> n >> q;
41
42     for (int i = 2; i <= n; ++i) {
43         int u;
44         cin >> u;
45         adj[u].push_back(i);
46     }
47
48     dfs(1, -1); // root = 1
49
50     for (int k = 1; k < LOG; ++k) {
51         for (int v = 1; v <= n; ++v) {
52             up[k][v] = up[k - 1][up[k - 1][v]];
53         }
54     }
55
56     while (q--) {
57         int a, b;
58         cin >> a >> b;
59         cout << lca(a, b) << "\n";
60     }
61 }

```

11.12 tree diameter

```

1 int diam = 0;
2
3 int dfs(int u, int p = -1) {
4     int mx = 0;
5     for (int v : adj[u]) {
6         if (v != p) {
7             int len = 1 + dfs(v, u);
8             diam = max(diam, mx + len);
9             mx = max(mx, len);
10        }
11    }
12    return mx;
13 }

```

```

13 }

```

11.13 all longest path

```

1 int fir[maxn]; // Length of the longest
2                 downward path from u into its subtree.
3 int sec[maxn]; // Length of the second
4                 longest downward path from u
5
6 void dfs1(int u, int p) {
7     for (int v : adj[u]) {
8         if (v != p) {
9             dfs1(v, u);
10            if (fir[v] + 1 > fir[u]) {
11                sec[u] = fir[u];
12                fir[u] = fir[v] + 1;
13            }
14            else if (fir[v] + 1 > sec[u]) {
15                sec[u] = fir[v] + 1;
16            }
17        }
18    }
19
20    // to_p: the best path length that comes
21    // from the parent's side
22    void dfs2(int u, int p, int to_p) {
23        res[u] = max(to_p, fir[u]);
24
25        for (int v : adj[u]) {
26            if (v != p) {
27                if (fir[v] + 1 == fir[u]) {
28                    dfs2(v, u, max(to_p, sec[u]) + 1);
29                }
30                else {
31                    dfs2(v, u, res[u] + 1);
32                }
33            }
34        }
35    }
36
37    // usage
38    dfs1(1, 0);
39    dfs2(1, 0, 0);
40    // Now res[i] is the maximum distance from
41    // node i to any other node
42    for (int i = 1; i <= n; ++i) {
43        cout << res[i] << " ";
44    }
45 }

```

11.14 tree diameter (len,end)

```

1 array<int, 2> dfs(int u, int w = -1) {
2     array<int, 2> mx{0, u}; // {length,
3     farthest leaf}
4     for (auto& v : adj[u]) {
5         if (v == w) continue;
6         auto [len, leaf] = dfs(v, u);
7     }
8     mx[0] = max(mx[0], len);
9     mx[1] = leaf;
10    return mx;
11 }

```

```

6         mx = max(mx, {len + 1, leaf});
7     }
8     return mx;
9 }
10
11 array<int, 3> tree_diameter(int a = 0) {
12     auto b{dfs(a)[1]}; // farthest node
13     // from 'a'
14     auto [1, c]{dfs(b)}; // farthest node
15     // from 'b'
16     return {1, b, c}; // {diameter
17     // length, endpoint1, endpoint2}
18 }

```

11.15 minimum distance in functional graph

```

1 // Given a functional graph (each node has
2 // exactly one outgoing edge), answer
3 // queries about the minimum distance
4 // between two nodes, or determine that
5 // it's impossible to get from one to the
6 // other.
7 // There are three cases:
8 // 1. Both in Tree
9 // 2. Both in Cycle
10 // 3. One in Tree, one in Cycle
11
12 int main() {
13     int planet_num;
14     int query_num;
15     std::cin >> planet_num >> query_num;
16     vector<int> next(planet_num);
17     vector<vector<int>> before(planet_num);
18     for (int p = 0; p < planet_num; p++) {
19         std::cin >> next[p];
20         next[p]--;
21         before[next[p]].push_back(p);
22     }
23
24     /*
25     * -2 = We haven't even got to processing
26     * this planet yet.
27     * -1 = This node is part of a tree.
28     * >= 0: the ID of the cycle the planet
29     * belongs to.
30     */
31     vector<int> cycle_id(planet_num, -2);
32     // Each map, given a planet #, returns the
33     // index of that planet in the
34     // cycle.
35     vector<std::map<int, int>> cycles;
36     for (int p = 0; p < planet_num; p++) {
37         if (cycle_id[p] != -2) { continue; }
38         vector<int> path{p};
39         int at = p;
40         while (cycle_id[next[at]] == -2) {
41             at = next[at];
42             cycle_id[at] = -3; // Leave
43             // breadcrumbs for this iteration
44             path.push_back(at);
45         }
46         // cycle found, process it
47         int cycle_start = at;
48         int cycle_end = at;
49         while (cycle_id[cycle_end] != -3) {
50             cycle_end = next[cycle_end];
51             cycle_id[cycle_end] = -3;
52             path.push_back(cycle_end);
53         }
54         cycles.push_back({});
55         for (int i = cycle_start; i <= cycle_end; i++) {
56             cycles.back()[i] = path[i - cycle_start];
57         }
58     }
59 }

```



```

39 std::map<int, int> cycle;
40 bool in_cycle = false;
41 for (int i : path) {
42     in_cycle = in_cycle || i == next[at];
43     if (in_cycle) { cycle[i] = cycle.size(); }
44     cycle_id[i] = in_cycle ? cycles.size() : -1;
45 }
46 cycles.push_back(cycle);
47 }
48 /*
49  * Precalculate the distance from each planet to its cycle with BFS.
50  * (cyc_dist[p] = 0) if p is part of a cycle.
51  */
52 vector<int> cyc_dist(planet_num);
53 for (int p = 0; p < planet_num; p++) {
54     // Check if this planet is part of a cycle.
55     if (cycle_id[next[p]] == -1 || cycle_id[p] != -1) { continue; }
56     cyc_dist[p] = 1;
57     vector<int> stack(before[p]);
58     while (!stack.empty()) {
59         int curr = stack.back();
60         stack.pop_back();
61         cyc_dist[curr] = cyc_dist[next[curr]] + 1;
62         stack.insert(stack.end(), before[curr].begin(), before[curr].end());
63     }
64 }
65 // Initialize the binary jumping arrays.
66 int log2 = std::ceil(std::log2(planet_num));
67 vector<vector<int>> pow2_ends(planet_num, vector<int>(log2 + 1));
68 for (int p = 0; p < planet_num; p++) {
69     pow2_ends[p][0] = next[p];
70 }
71 for (int i = 1; i <= log2; i++) {
72     for (int p = 0; p < planet_num; p++) {
73         pow2_ends[p][i] = pow2_ends[pow2_ends[p][i-1]][i-1];
74     }
75 }
76 /*
77  * Given a starting planet & dist, returns the planet we end up at
78  * if we use the teleporter dist times.
79  */
80 auto advance = [&](int pos, int dist) {
81     for (int pow = log2; pow >= 0; pow--) {
82         if ((dist & (1 << pow)) != 0) { pos = pow2_ends[pos][pow]; }
83     }
84     return pos;
85 };
86 for (int q = 0; q < query_num; q++) {
87     int u, v; // going from u to v
88     std::cin >> u >> v;

```

```

91 u--;
92 v--;
93 if (cycle_id[pow2_ends[u][log2]] != cycle_id[pow2_ends[v][log2]]) {
94     cout << -1 << '\n';
95     continue;
96 }
97 if (cycle_id[u] != -1 || cycle_id[v] != -1) {
98     if (cycle_id[v] == -1 && cycle_id[u] != -1) {
99         cout << -1 << '\n';
100         continue;
101     }
102     // Handle the 2nd & 3rd cases at the same time.
103     int dist = cyc_dist[u];
104     int u_cyc = advance(u, cyc_dist[u]);
105     // The root of u's tree
106     std::map<int, int> &cyc = cycles[cycle_id[u_cyc]]; // u and v's cycle
107     int u_ind = cyc[u_cyc];
108     int v_ind = cyc[v];
109     int diff = u_ind <= v_ind ? v_ind - u_ind : cyc.size() - (u_ind - v_ind);
110     cout << dist + diff << '\n';
111 } else {
112     if (cyc_dist[v] > cyc_dist[u]) {
113         cout << -1 << '\n';
114         continue;
115     }
116     int diff = cyc_dist[u] - cyc_dist[v];
117     cout << (advance(u, diff) == v ? diff : -1) << '\n';
118 }
119 }
120 }

```

11.16 union length of two tree paths

```

1 /*
2 Problem: Given a tree and queries (a,b,c,d), find the size of the union of nodes on paths a↔b and c↔d.
3
4 Let P1 be the nodes on the path a↔b and P2 the nodes on c↔d.
5 We want |P1 ∪ P2| = |P1| + |P2| - |P1 ∩ P2|.
6 |P1| = dist(a,b) + 1, |P2| = dist(c,d) + 1. So the only hard part is |P1 ∩ P2|.
7
8 Key facts for trees:
9
10 The intersection of two simple paths is itself either empty or a simple path.
11
12 Any endpoint of that intersection must be one of the LCAs among endpoint pairs of the two paths.
13

```

```

14 Concretely the candidate nodes are
15 Lca(a,b), Lca(c,d), Lca(a,c), Lca(a,d), Lca(b,c), Lca(b,d)
16
17 A node z lies on path u↔v iff dist(u,z) + dist(z,v) == dist(u,v).
18
19 So for each query:
20 compute the 6 candidate LCAs,
21 keep those candidates that lie on both paths (test with distances),
22 if none → intersection size = 0; if one → intersection size = 1;
23 otherwise take the two kept candidates with greatest depth (they are the endpoints of the intersection)
24 and intersection size = dist(p,q) + 1.
25
26 */
27 const int LOG = 32;
28 const int maxn = 1e5;
29 int depth[maxn];
30 vector<int> adj[maxn];
31 int up[LOG + 1][maxn];
32 int n, q;
33
34 void dfs(int u, int p = -1) {
35     up[0][u] = (p == -1 ? u : p);
36     for (int v : adj[u]) {
37         if (v != p) {
38             depth[v] = depth[u] + 1;
39             dfs(v, u);
40         }
41     }
42 }
43
44 void solve() {
45     cin >> n >> q;
46     for (int i = 0; i < n - 1; ++i) {
47         int a, b;
48         cin >> a >> b;
49         a--, b--;
50         adj[a].push_back(b);
51         adj[b].push_back(a);
52     }
53     memset(depth, 0, sizeof(depth));
54     dfs(0, -1);
55
56     for (int k = 1; k < LOG; ++k) {
57         for (int v = 0; v < n; ++v) {
58             up[k][v] = up[k-1][up[k-1][v]];
59         }
60     }
61
62     while (q--) {
63         int a, b, c, d;
64         cin >> a >> b >> c >> d;
65         a--, b--, c--, d--;
66         int len1 = dist(a, b) + 1;
67         int len2 = dist(c, d) + 1;

```

```

73 array<int, 6> cand = {
74     lca(a, b), lca(c, d),
75     lca(a, c), lca(a, d),
76     lca(b, c), lca(b, d)
77 };
78
79 vector<int> good;
80 for (int z : cand) {
81     if (on_path(z, a, b) && on_path(z, c, d)) {
82         good.push_back(z);
83     }
84 }
85
86 int inter = 0;
87 if (good.size() == 1) {
88     inter = 1;
89 }
90 else if (good.size() > 1) {
91     sort(good.begin(), good.end(), [&](int x, int y) {
92         return depth[x] > depth[y];
93     });
94     int p = good[0], qn = good[1];
95     inter = dist(p, qn) + 1;
96 }
97 cout << (len1 + len2 - inter) << "\n";
98 }
99 }
100 }

```

11.17 Centroid decomposition

```

1 // Centroid Decomposition
2 // Problem: Given a tree with N nodes (1-indexed) and M queries. Each query is one of the following two types:
3 // 1 v: Paint node v red.
4 // 2 v: Print the minimal distance between node v and any red node.
5 // Initially, only node 1 is red.
6 // Constraints: 1 ≤ N, M ≤ 10^5
7 // Time complexity: O((N + M) log N)
8
9 vector<vector<int>> adj;
10 vector<int> subtree_size;
11 // min_dist[v] = the minimal distance between v and a red node
12 vector<int> min_dist;
13 vector<bool> is_removed;
14 vector<vector<pii>> ancestors;
15
16 int get_subtree_size(int u, int p = -1) {
17     subtree_size[u] = 1;
18     for (int v : adj[u]) {
19         if (v == p || is_removed[v]) continue;
20         subtree_size[u] += get_subtree_size(v, u);
21     }
22     return subtree_size[u];
23 }

```

```

24 int get_centroid(int u, int tree_size, int p
25 = -1) {
26     for (int v : adj[u]) {
27         if (v == p || is_removed[v])
28             continue;
29         if (subtree_size[v] * 2 > tree_size)
30             return get_centroid(v, tree_size
31             , u);
32     }
33     return u;
34 }
35 // Calculate the distance between current `u`
36 // and the `centroid` it belongs.
37 void get_dist(int u, int centroid, int p =
38 -1, int cur_dist = 1) {
39     for (int v : adj[u]) {
40         if (v == p || is_removed[v])
41             continue;
42         cur_dist++;
43         get_dist(v, centroid, u, cur_dist);
44         cur_dist--;
45     }
46     ancestors[u].emplace_back(centroid,
47     cur_dist);
48 }
49 void build_centroid_decomp(int u = 0) {
50     int centroid = get_centroid(u,
51     get_subtree_size(u));
52     // For all nodes in the subtree rooted
53     // at `centroid`, calculate their
54     // distance to the centroid
55     for (int v : adj[centroid]) {
56         if (is_removed[v]) continue;
57         get_dist(v, centroid, centroid);
58     }
59     is_removed[centroid] = true;
60     for (int v : adj[centroid]) {
61         if (is_removed[v]) continue;
62         build_centroid_decomp(v);
63     }
64 }
65 // Paint `node` red by updating all of its
66 // ancestors' minimal distances to a red
67 // node
68 void paint(int u) {
69     for (auto &[ancestor, dist] : ancestors[
70     u]) {
71         min_dist[ancestor] = min(min_dist[
72     ancestor], dist);
73     }
74     min_dist[u] = 0;
75 }
76 // Print the minimal distance between `u` to
77 // a red node.
78 void query(int u) {
79     int ans = min_dist[u];

```

```

73 for (auto &[ancestor, dist] : ancestors[
74     u]) {
75     if (!dist) continue;
76     ans = min(ans, dist + min_dist[
77     ancestor]);
78 }
79 cout << ans << "\n";
80 }
81 void solve() {
82     int N, M;
83     cin >> N >> M;
84     adj.assign(N, vector<int>());
85     for (int i = 0; i < N - 1; ++i) {
86         int a, b;
87         cin >> a >> b;
88         a--, b--;
89         adj[a].push_back(b);
90         adj[b].push_back(a);
91     }
92     subtree_size.assign(N, 0);
93     ancestors.assign(N, vector<pii>());
94     is_removed.assign(N, false);
95     build_centroid_decomp();
96     min_dist.assign(N, INF);
97     paint(0);
98     for (int i = 0; i < M; ++i) {
99         int t, v;
100         cin >> t >> v;
101         v--;
102         if (t == 1) {
103             paint(v);
104         }
105         else {
106             query(v);
107         }
108     }
109 }
110 }
111 }

```

12 default

12.1 debug

```

1 #ifndef DEBUG
2 #define dbg(...) {\
3     fprintf(stderr, "%s - %d : (%s) = ",
4     __PRETTY_FUNCTION__, __LINE__, #
5     __VA_ARGS__); \
6     _DO(__VA_ARGS__); \
7 }
8 #define dbg(...)
9 #endif

```

12.2 template

```

1 // alias g++='g++ -std=c++14 -fsanitize=
2 // undefined -Wall -Wextra -Wshadow -D
3 // LOCAL'
4 #include <bits/stdc++.h>
5 using namespace std;
6 #ifdef LOCAL
7 #include "../debug.h"
8 #else
9 #pragma GCC optimize("O3,unroll-loops")
10 #pragma GCC target("avx2,bmi,bmi2,lzcnt,
11     popcnt")
12 #define debug(...) ((void)0)
13 #define orange(...) ((void)0)
14 #endif
15 #define int long long
16 #define pii pair<int, int>
17 #define ff first
18 #define ss second
19 #define pb push_back
20 #define SPEEDY ios_base::sync_with_stdio(
21     false); cin.tie(0); cout.tie(0);
22 void solve() {
23 }
24 }
25 signed main() {
26     SPEEDY;
27     return 0;
28 }

```

12.3 vimrc

```

1 set nu
2 set ts=4
3 set shiftwidth=4
4 set autoindent smartindent
5 set cindent
6
7 imap jk <ESC>
8 inoremap {<CR> {<CR>}<ESC>ko

```

13 other

13.1 Nim game

```

1 a1^a2^a3^...^an != 0 ? A win : B win

```

13.2 找小於 n 所有出現的 1 數量

```

1 current == 0 higher * factor
2 current == 1 higher * factor + lower + 1
3 other current (higher + 1) * factor

```

14 other language

14.1 python heap

```

1 import heapq
2
3 heap = [7,1,2,2]
4 heapq.heapify(heap)
5 print(heap) # [1, 2, 2, 7]
6 heapq.heappush(heap, 5)
7 print(heap) # [1, 2, 2, 7, 5]
8 print(heapq.heappop(heap)) # 1
9 print(heap) # [2, 2, 5, 7]

```

14.2 java

14.2.1 文件操作

```

1 import java.io.*;
2 import java.util.*;
3 import java.math.*;
4 import java.text.*;
5
6 public class Main{
7
8     public static void main(String args[]){
9         throws FileNotFoundException,
10         IOException
11         Scanner sc = new Scanner(new FileReader(
12         "a.in"));
13         PrintWriter pw = new PrintWriter(new
14         FileWriter("a.out"));
15         int n,m;
16         n=sc.nextInt();//讀入下一個INT
17         m=sc.nextInt();
18
19         for(ci=1; ci<=c; ++ci){
20             pw.println("Case #"+ci+": easy for
21             output");
22         }
23
24         pw.close();//关闭流并释放。这个很重要。
25         否则是没有输出的
26         sc.close();//关闭流并释放
27     }
28 }

```

14.2.2 优先队列

```
1 PriorityQueue queue = new PriorityQueue( 1,
2     new Comparator(){
3         public int compare( Point a, Point b ){
4             if( a.x < b.x || a.x == b.x && a.y < b.y )
5                 return -1;
6             else if( a.x == b.x && a.y == b.y )
7                 return 0;
8             else return 1;
9         }
10    });
```

14.2.3 Map

```
1 Map map = new HashMap();
2 map.put("sa","dd");
3 String str = map.get("sa").toString();
4
5 for(Object obj : map.keySet()){
6     Object value = map.get(obj );
7 }
```

14.2.4 sort

```
1 static class cmp implements Comparator{
2     public int compare(Object o1,Object o2){
3         BigInteger b1=(BigInteger)o1;
4         BigInteger b2=(BigInteger)o2;
5         return b1.compareTo(b2);
6     }
7 }
8 public static void main(String[] args)
9     throws IOException{
10     Scanner cin = new Scanner(System.in);
11     int n;
12     n=cin.nextInt();
13     BigInteger[] seg = new BigInteger[n];
14     for (int i=0;i<n;i++)
15         seg[i]=cin.nextBigInteger();
16     Arrays.sort(seg,new cmp());
17 }
```

14.3 python output

```
1 hello = 'Hello'
2 world = 7122
3 print(f'{hello} {world}') # Hello 7122
4
5 import math
6 print(f'PI is approximately {math.pi:.3f}.')
7 # PI is approximately 3.142.
8
9 print('AAA {} BBB "{}!"'.format('Jin', 'KeLa
10 # AAA Jin BBB "KeLa!"
```

```
11 hello = 'hello, world\n'
12 hellos = repr(hello)
13 print(hellos) # 'hello, world\n'
14
15 x = 32.5
16 y = 40000
17 print(repr((x, y, ('spam', 'eggs'))))
18 # "(32.5, 40000, ('spam', 'eggs'))"
19
20 x = 7
21 print(eval('3 * x')) # 21
```

14.4 python 大數因數分解

```
1 # 大數因數分解 (使用 Pollard's Rho 與 Miller
2     -Rabin)
3 import sys, random
4 from math import gcd
5
6 # Miller-Rabin 檢定 (機率性質數判定)
7 def is_probable_prime(n, k=12):
8     if n < 2:
9         return False
10    # 先檢查一些小質數
11    small_primes =
12        [2,3,5,7,11,13,17,19,23,29]
13    for p in small_primes:
14        if n % p == 0:
15            return n == p
16    # 把 n-1 寫成 d * 2^s
17    d = n - 1
18    s = 0
19    while d % 2 == 0:
20        d //= 2
21        s += 1
22    # 重複 k 次隨機測試
23    for _ in range(k):
24        a = random.randrange(2, n - 1) # 隨
25        # 機挑一個測試基數
26        x = pow(a, d, n)
27        if x == 1 or x == n - 1:
28            continue
29        composite = True
30        for __ in range(s - 1):
31            x = pow(x, 2, n)
32            if x == n - 1:
33                composite = False
34                break
35        if composite:
36            return False
37        return True
38
39 # Pollard's Rho 演算法 (找非平凡因數)
40 def pollards_rho(n):
41     if n % 2 == 0:
42         return 2
43     if n % 3 == 0:
44         return 3
45     # 隨機多項式 (x^2 + c) mod n
46     while True:
```

```
47     c = random.randrange(1, n-1) #
48     # 隨機挑選常數 c
49     x = random.randrange(2, n-1) #
50     # 起始點
51     y = x
52     d = 1
53     while d == 1:
54         x = (pow(x, 2, n) + c) % n # x
55         # -> f(x)
56         y = (pow(y, 2, n) + c) % n # y
57         # -> f(f(y)) · 走兩步
58         d = gcd(abs(x - y), n) #
59         # 計算兩者差的 gcd
60         if d == n:
61             # 失敗就重試
62             break
63         if d > 1 and d < n:
64             # 找到非平凡因數
65             return d
66
67 # 遞迴分解
68 def factor(n, out):
69     if n == 1:
70         return
71     if is_probable_prime(n):
72         out.append(n)
73     else:
74         d = pollards_rho(n)
75         while d is None or d == n: # 偶爾失
76             # 敗就重試
77             d = pollards_rho(n)
78         factor(d, out)
79         factor(n // d, out)
80
81 def main():
82     data = sys.stdin.read().strip().split()
83     if not data:
84         return
85     # 每個 token 當作一個數字
86     for token in data:
87         try:
88             n = int(token)
89         except:
90             continue
91         if n <= 1:
92             print(n)
93             continue
94         facs = []
95         factor(n, facs)
96         facs.sort()
97         # 輸出因數
98         print(" ".join(str(x) for x in facs))
99
100 if __name__ == "__main__":
101     random.seed() # 使用系統時間作為隨機種子
102     main()
```

14.5 decimal

```
1 # 使用 decimal 模組來處理高精度小數運算
2 from decimal import *
3 setcontext(Context(prec=MAX_PREC, Emax=
4     MAX_EMAX, rounding=ROUND_FLOOR))
5 print(Decimal(input()) * Decimal(input()))
6
7 # 將小數轉成分數 · 方便做近似或理論分析 · 且可
8     以限制分母大小 ·
9 from fractions import Fraction
10 Fraction('3.14159').limit_denominator(10).
11     numerator # 22
12
13 # 設定精確度
14 from decimal import Decimal, getcontext
15 getcontext().prec = 70
16
17 n = 100
18 # 指定 n 為高精確度的物件
19 n = Decimal(n)
20 n /= 7
21 print(n)
22
23 # 將小數轉成分數
24 from fractions import Fraction
25 n = 1.5654
26 # 建立一個轉換物件
27 n = Fraction(n)
28 print(n)
```

14.6 python 大數排序

```
1 # 大數排序
2 # Line one n : 多少數字
3 # next Line : 依序輸入每行一個
4 # sort : sort + lambda
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 i = 0
10
11 while(i < len(data)):
12     T = int(data[i].strip())
13     i += 1
14     temp = [int(data[i + index].strip()) for
15         index in range(T)]
16     i += T
17     temp = sorted(temp, key = lambda x : (x))
18     for ele in temp:
19         print(ele)
```

14.7 python 大數計算 2

```

1 # 單行輸入
2 # format : n1, operation, n2
3 from sys import stdin
4
5 data = stdin.read().splitlines()
6
7 limit = len(data)
8 i = 0
9
10 while(i < limit):
11     a, operation, b = map(str, data[i].split())
12     a, b = int(a), int(b)
13     i += 1
14     if(operation == '+'):
15         print(int(a + b))
16     elif(operation == '-'):
17         print(int(a - b))
18     elif(operation == '*'):
19         print(int(a * b))
20     else:
21         print(int(a // b))

```

14.8 python input

```

1 ans = sum(map(float, input().split()))
2 # input: 1.1 2.2 3.3 4.4 5.5
3 print(ans) # 16.5
4
5 (n, m) = map(int, input().split()) # 300 200
6 print(n * m) # 60000
7
8 Arr = list(map(int, input().split()))
9 # input: 1 2 3 4 5
10 print(Arr) # [1, 2, 3, 4, 5]

```

14.9 python 大數計算

```

1 # 讀取多行輸入
2 # line one first number
3 # line two operation
4 # line three second number
5 from sys import stdin
6
7 data = stdin.read().splitlines()
8
9 limit = len(data)
10 i = 0
11 while(i < limit):
12     a = int(data[i].strip())
13     i += 1
14     operation = data[i].strip()
15     i += 1
16     b = int(data[i].strip())
17     i += 1
18     if(operation == '*'):
19         print(int(a * b))

```

```

20 else:
21     print(int(a / b))

```

15 zformula

15.1 formula

15.1.1 Pick 公式

給定頂點坐標均是整點的簡單多邊形。面積 = 內部格點數 + 邊上格點數/2-1

15.1.2 圖論

- 對於平面圖 $\cdot F = E - V + C + 1$ $\cdot C$ 是連通分量數
- 對於平面圖 $\cdot E \leq 3V - 6$
- 對於連通圖 G $\cdot$ 最大獨立點集的大小設為 $I(G)$ $\cdot$ 最大匹配大小設為 $M(G)$ $\cdot$ 最小點覆蓋設為 $Cv(G)$ $\cdot$ 最小邊覆蓋設為 $Ce(G)$ $\cdot$ 對於任意連通圖：

$$(a) \quad I(G) + Cv(G) = |V|$$

$$(b) \quad M(G) + Ce(G) = |V|$$

- 對於連通二分圖：

$$(a) \quad I(G) = Cv(G)$$

$$(b) \quad M(G) = Ce(G)$$

- 最大權閉合圖：

$$(a) \quad C(u, v) = \infty, (u, v) \in E$$

$$(b) \quad C(S, v) = W_v, W_v > 0$$

$$(c) \quad C(v, T) = -W_v, W_v < 0$$

$$(d) \quad \text{ans} = \sum_{W_v > 0} W_v - \text{flow}(S, T)$$

- 最大密度子圖：

$$(a) \quad \text{求 } \max \left(\frac{W_e + W_v}{|V|} \right), e \in E', v \in V'$$

$$(b) \quad U = \sum_{v \in V} 2W_v + \sum_{e \in E} W_e$$

$$(c) \quad C(u, v) = W_{(u, v)}, (u, v) \in E \cdot \text{雙向邊}$$

$$(d) \quad C(S, v) = U, v \in V$$

$$(e) \quad D_u = \sum_{(u, v) \in E} W_{(u, v)}$$

$$(f) \quad C(v, T) = U + 2g - D_v - 2W_v, v \in V$$

$$(g) \quad \text{二分搜 } g:$$

$$l = 0, r = U, \text{eps} = 1/n^2$$

$$\text{if}((U \times |V| - \text{flow}(S, T))/2 > 0) \text{ then } l = \text{mid}$$

$$\text{else } r = \text{mid}$$

$$(h) \quad \text{ans} = \min_cut(S, T)$$

$$(i) \quad |E| = 0 \text{ 要特殊判斷}$$

- 弦圖：

- 點數大於 3 的環都要有一條弦
- 完美消除序列從後往前依次給每個點染色。給每個點染上可以染的最小顏色
- 最大團大小 = 色數
- 最大獨立集: 完美消除序列從前往後能選就選
- 最小團覆蓋: 最大獨立集的點和他延伸的邊構成
- 區間圖是弦圖
- 區間圖的完美消除序列: 將區間按造又端點由小到大排序
- 區間圖染色: 用線段樹做

15.1.3 dinic 特殊圖複雜度

- 單位流: $O\left(\min\left(V^{3/2}, E^{1/2}\right)E\right)$
- 二分圖: $O\left(V^{1/2}E\right)$

15.1.4 0-1 分數規劃

$x_i = \{0, 1\} \cdot x_i$ 可能會有其他限制 $\cdot$ 求 $\max\left(\frac{\sum B_i x_i}{\sum C_i x_i}\right)$

- $D(i, g) = B_i - g \times C_i$
- $f(g) = \sum D(i, g) x_i$
- $f(g) = 0$ 時 g 為最佳解 $\cdot f(g) < 0$ 沒有意義
- 因為 $f(g)$ 單調可以二分搜 g
- 或用 Dinkelbach 通常比較快

```

1 binary_search(){
2     while(r-l>eps){
3         g=(l+r)/2;
4         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
5         找出一組合法x[i]使f(g)最大;
6         if(f(g)>0) l=g;
7         else r=g;
8     }
9     Ans = r;
10 }
11 Dinkelbach(){
12     g=任意狀態(通常設為0);
13     do{
14         Ans=g;
15         for(i:所有元素)D[i]=B[i]-g*C[i]; //D(i,g)
16         找出一組合法x[i]使f(g)最大;
17         p=0, q=0;
18         for(i:所有元素)
19             if(x[i])p+=B[i], q+=C[i];
20         g=p/q; //更新解 · 注意q=0的情況
21     }while(abs(Ans-g)>EPS);
22     return Ans;
23 }

```

15.1.5 學長公式

- $\sum_{d|n} \phi(n) = n$
- $g(n) = \sum_{d|n} f(d) \Rightarrow f(n) = \sum_{d|n} \mu(d) \times g(n/d)$
- Harmonic series $H_n = \ln(n) + \gamma + 1/(2n) - 1/(12n^2) + 1/(120n^4)$
- $\gamma = 0.57721566490153286060651209008240243104215$
- 格雷碼 $= n \oplus (n >> 1)$
- $SG(A + B) = SG(A) \oplus SG(B)$
- 選轉矩陣 $M(\theta) = \begin{pmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{pmatrix}$

15.1.6 基本數論

- $\sum_{d|n} \mu(n) = [n == 1]$
- $g(m) = \sum_{d|m} f(d) \Leftrightarrow f(m) = \sum_{d|m} \mu(d) \times g(m/d)$
- $\sum_{i=1}^n \sum_{j=1}^m \text{互質數量} = \sum \mu(d) \lfloor \frac{n}{d} \rfloor \lfloor \frac{m}{d} \rfloor$
- $\sum_{i=1}^n \sum_{j=1}^n \text{lcm}(i, j) = n \sum_{d|n} d \times \phi(d)$

15.1.7 排組公式

- 卡卡特蘭 $\frac{C_n^{kn}}{n(k-1)+1} \cdot C_m^n = \frac{n!}{m!(n-m)!}$
- $H(n, m) \cong x_1 + x_2 + \dots + x_n = k, \text{num} = C_{n+k-1}^k$
- Stirling number of $2^{n^d}, n$ 人分 k 組方法數目
 - $S(0, 0) = S(n, n) = 1$
 - $S(n, 0) = 0$
 - $S(n, k) = kS(n-1, k) + S(n-1, k-1)$
- Bell number, n 人分任意多組方法數目
 - $B_0 = 1$
 - $B_n = \sum_{i=0}^n S(n, i)$
 - $B_{n+1} = \sum_{k=0}^n C_k^n B_k$
 - $B_{p+n} \equiv B_n + B_{n+1} \pmod{p}$, p is prime
 - $B_{p^m+n} \equiv mB_n + B_{n+1} \pmod{p}$, p is prime
 - From $B_0 : 1, 1, 2, 5, 15, 52, 203, 877, 4140, 21147, 115975$
- Derangement, 錯排, 沒有人在自己位置上
 - $D_n = n!(1 - \frac{1}{1!} + \frac{1}{2!} - \frac{1}{3!} + \dots + (-1)^n \frac{1}{n!})$
 - $D_n = (n-1)(D_{n-1} + D_{n-2}), D_0 = 1, D_1 = 0$
 - From $D_0 : 1, 0, 1, 2, 9, 44, 265, 1854, 14833, 133496$

6. Binomial Equality

- $\sum_k \binom{r}{m+k} \binom{s}{n-k} = \binom{r+s}{m+n}$
- $\sum_k \binom{l}{m+k} \binom{s}{n-k} = \binom{l+s}{l-m+n}$
- $\sum_k \binom{l}{m+k} \binom{s+k}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m}{n-l}$
- $\sum_{k \leq l} \binom{l-k}{m} \binom{s}{k-n} (-1)^k = (-1)^{l+m} \binom{s-m-1}{l-n-m}$
- $\sum_{0 \leq k \leq l} \binom{l-k}{m} \binom{q+k}{n} = \binom{l+q+1}{m+n+1}$
- $\binom{r}{k} = (-1)^k \binom{k-r-1}{k}$
- $\binom{r}{m} \binom{m}{k} = \binom{r}{n} \binom{r-k}{m-k}$
- $\sum_{k \leq n} \binom{r+k}{k} = \binom{r+n+1}{m+1}$
- $\sum_{0 \leq k \leq n} \binom{k}{m} = \binom{n+1}{m+1}$
- $\sum_{k \leq m} \binom{m+r}{k} x^k y^k = \sum_{k \leq m} \binom{m+r}{k} (-x)^k (x+y)^{m-k}$

15.1.8 冪次, 冪次和

1. $a^{b \% a} P = a^{b \% \varphi(p) + \varphi(p)}, b \geq \varphi(p)$
2. $1^3 + 2^3 + 3^3 + \dots + n^3 = \frac{n^4}{4} + \frac{n^3}{2} + \frac{n^2}{4}$
3. $1^4 + 2^4 + 3^4 + \dots + n^4 = \frac{n^5}{5} + \frac{n^4}{2} + \frac{n^3}{3} - \frac{n}{30}$
4. $1^5 + 2^5 + 3^5 + \dots + n^5 = \frac{n^6}{6} + \frac{n^5}{2} + \frac{5n^4}{12} - \frac{n^2}{12}$
5. $0^k + 1^k + 2^k + \dots + n^k = P(k), P(k) = \frac{(n+1)^{k+1} - \sum_{i=0}^{k-1} C_i^{k+1} P(i)}{k+1}, P(0) = n+1$
6. $\sum_{k=0}^{m-1} k^n = \frac{1}{n+1} \sum_{k=0}^n C_k^{n+1} B_k m^{n+1-k}$
7. $\sum_{j=0}^m C_j^{m+1} B_j = 0, B_0 = 1$
8. 除了 $B_1 = -1/2$ · 剩下的奇數項都是 0
9. $B_2 = 1/6, B_4 = -1/30, B_6 = 1/42, B_8 = -1/30, B_{10} = 5/66, B_{12} = -691/2730, B_{14} = 7/6, B_{16} = -3617/510, B_{18} = 43867/798, B_{20} = -174611/330,$

15.1.9 Burnside’s lemma

1. $|X/G| = \frac{1}{|G|} \sum_{g \in G} |X^g|$
2. $X^g = t^{c(g)}$
3. G 表示有幾種轉法 · X^g 表示在那種轉法下 · 有幾種是會保持對稱的 · t 是顏色數 · $c(g)$ 是循環節不動的面數 ·
4. 正立方體塗三顏色 · 轉 0 有 3^6 個元素不變 · 轉 90 有 6 種 · 每種有 3^3 不變 · 180 有 $3 \times 3^4 \cdot 120(\text{角})$ 有 $8 \times 3^2 \cdot 180(\text{邊})$ 有 $6 \times 3^3 \cdot$ 全部 $\frac{1}{57} (3^6 + 6 \times 3^3 + 3 \times 3^4 + 8 \times 3^2 + 6 \times 3^3) =$

15.1.10 Count on a tree

1. Rooted tree: $s_{n+1} = \frac{1}{n} \sum_{i=1}^n (i \times a_i \times \sum_{j=1}^{\lfloor n/i \rfloor} a_{n+1-i \times j})$
2. Unrooted tree:

(a) Odd: $a_n - \sum_{i=1}^{n/2} a_i a_{n-i}$

(b) Even: $Odd + \frac{1}{2} a_{n/2} (a_{n/2} + 1)$
3. Spanning Tree

(a) 完全圖 $n^n - 2$

(b) 一般圖 (Kirchhoff’s theorem) $M[i][i] = \text{degree}(V_i), M[i][j] = -1, \text{if have } E(i, j), 0 \text{ if no edge. delete any one row and col in } A, \text{ans} = \det(A)$

15.1.11 循環小數轉分數

1. 若 $x = 0.\overline{a} \cdot$ 則

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

其中 a 為循環節 · k 為循環節的位數 ·

2. 例子：

$$0.\overline{37} = \frac{37}{99}$$
$$0.\overline{5} = \frac{5}{9}$$

15.1.12 循環小數轉分數

1. 純循環小數：若 $x = 0.\overline{a}$ · 其中 a 為循環節、長度為 k ·

$$x = \frac{a}{\underbrace{99 \dots 9}_{k \text{ digits}}}$$

例：

$$0.\overline{37} = \frac{37}{99}, \quad 0.\overline{5} = \frac{5}{9}$$

2. 混循環小數：若 $x = 0.b\overline{a}$ · 其中 b 為前綴、長度 m · a 為循環節、長度 k ·

$$x = \frac{(b \cdot 10^k + a) - b}{10^m (10^k - 1)}$$

例：

$$0.12\overline{3} = \frac{(12 \cdot 10^1 + 3) - 12}{10^2 (10^1 - 1)} = \frac{123 - 12}{100 \cdot 9} = \frac{111}{900} = \frac{37}{300}$$

15.1.13 常見級數與組合公式

1. 平方和公式：

$$1^2 + 2^2 + \dots + n^2 = \frac{n(n+1)(2n+1)}{6}$$

$$4^2 + 6^2 + \dots + (2n)^2 = \frac{(2n)(n+1)(2n+1)}{3}$$

$$1^2 + 3^2 + \dots + (2n+1)^2 = \frac{n(2n-1)(2n+1)}{3}$$

2. 立方和公式：

$$1^3 + 2^3 + \dots + n^3 = \frac{n^4 + 2n^3 + n^2}{4}$$

3. 四次方和公式：

$$1^4 + 2^4 + \dots + n^4 = \frac{6n^5 + 15n^4 + 10n^3 - n}{30}$$

4. 五次方和公式：

$$1^5 + 2^5 + \dots + n^5 = \frac{2n^6 + 6n^5 + 5n^4 - n^2}{12}$$

5. 六次方和公式：

$$1^6 + 2^6 + \dots + n^6 = \frac{6n^7 + 21n^6 + 21n^5 - 7n^3 + n}{42}$$

6. 七次方和公式：

$$1^7 + 2^7 + \dots + n^7 = \frac{3n^8 + 12n^7 + 14n^6 - 7n^4 + 2n^2}{24}$$

7. 八次方和公式：

$$\frac{1^8 + 2^8 + \dots + n^8}{= \frac{10n^9 + 45n^8 + 60n^7 - 42n^5 + 20n^3 - 3n}{90}}$$

8. 九次方和公式：

$$\frac{1^9 + 2^9 + \dots + n^9}{= \frac{2n^{10} + 10n^9 + 15n^8 - 14n^6 + 10n^4 - 3n^2}{20}}$$

9. 十次方和公式：

$$\frac{1^{10} + 2^{10} + \dots + n^{10}}{= \frac{6n^{11} + 33n^{10} + 55n^9 - 66n^7 + 66n^5 - 33n^3 + 5n}{66}}$$

10. 等比級數：

$$S = a \cdot \frac{r^n - 1}{r - 1}$$

11. 二項式係數恆等式：

$$\binom{n}{0} + \binom{n}{1} + \dots + \binom{n}{n} = 2^n$$

$$\binom{n}{1} + \binom{n}{2} + \dots + \binom{n}{n} = 2^n - 1$$

$$\binom{n}{1} + \binom{n}{3} + \dots + \binom{n}{n-k} = 2^{n-1}$$

12. 分配問題 (玩具分給小孩)：

(a) n 個玩具 · k 位小孩 · 可以有人沒拿到：

$$\binom{n+k-1}{n} = \binom{n+k-1}{k-1}$$

(b) n 個玩具 · k 位小孩 · 每個人至少一個：

$$\binom{n-1}{k-1}$$

15.1.14 位元運算

(a) 位元條件：

$$(x+k) \& (y+k) = 0$$

(b) 加法恆等式 (利用 XOR 與 AND)：

$$a + b = (a \oplus b) + 2 \cdot (a \& b)$$

(c) OR 與 AND 的關係：

$$a \mid b = a + b - (a \& b)$$

(d) 交換兩數：

$$a = a \oplus b, \quad b = a \oplus b, \quad a = a \oplus b$$

(e) 取得最低位的 1：

$$x \& (-x)$$

(f) 清除最低位的 1：

$$x \& (x - 1)$$

15.1.15 除數與倍數

(a) LCM:

$$\text{lcm}(a, b) = \frac{a \times b}{\text{gcd}(a, b)}$$

(b) LCM (prime factorization):

$$\text{lcm}(a, b) = \prod_{p \in \text{primes}} p^{\max(e_a, e_b)}$$

(c) GCD (prime factorization):

$$\text{gcd}(a, b) = \prod_{p \in \text{primes}} p^{\min(e_a, e_b)}$$

(d) Counting divisors:

$$\text{If } n = \prod_{i=1}^k p_i^{e_i}, \text{ cnt is } \prod_{i=1}^k (e_i + 1)$$

15.1.16 數論公式

13. Bezout's identity :

$ax + by = \gcd(a, b)$ (必定存在整數解 x, y)

14. 模指數運算 (幂的幂):

$a^{b^c} \equiv a^{\text{power_mod}(b, c, \text{MOD}-1)} \pmod{\text{MOD}}$

Pythagorean theorem :

$(n^2 + m^2, 2nm, n^2 - m^2), \quad n > m > 0$

Wilson's theorem :

$(p - 1)! \pmod p = p - 1 \quad (p \text{ 為質數})$

Catalan number :

$C_n = \frac{1}{n + 1} \binom{2n}{n} = \frac{(2n)!}{(n + 1)!n!}$

海龍公式 :

$\text{Area} = \sqrt{s(s - a)(s - b)(s - c)}, \quad s = \frac{a + b + c}{2}$

兩圓相交弦長 :

$\text{Length} = 2 \cdot \sqrt{\frac{(s - a)(s - b)}{s}}, \quad s = \frac{r_1 + r_2 + d}{2}$

三角形內切圓半徑 :

$r = \frac{2 \cdot \text{Area}}{a + b + c}$

三角形外接圓半徑 :

$R = \frac{abc}{4 \cdot \text{Area}}$

三角形重心 :

$G = \left(\frac{x_1 + x_2 + x_3}{3}, \frac{y_1 + y_2 + y_3}{3} \right)$

三角形垂心 :

$H = \left(\frac{x_1 \tan A + x_2 \tan B + x_3 \tan C}{\tan A + \tan B + \tan C}, \frac{y_1 \tan A + y_2 \tan B + y_3 \tan C}{\tan A + \tan B + \tan C} \right)$

三角形內心 :

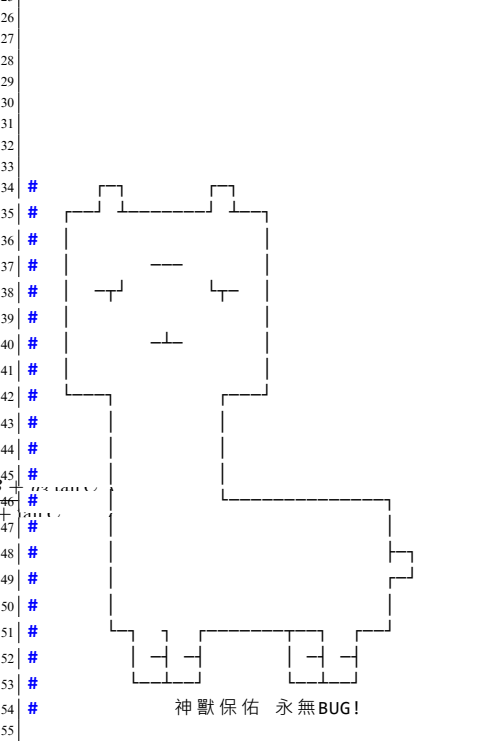
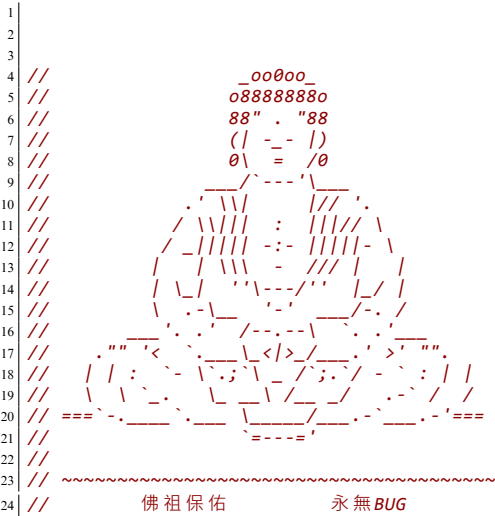
$I = \left(\frac{ax_1 + bx_2 + cx_3}{a + b + c}, \frac{ay_1 + by_2 + cy_3}{a + b + c} \right)$

奇 (偶) 長度陣列子數量 :

$n = \frac{(i + 1) * (n - i) + 1}{2}$

16 Интернационал

16.1 保佑



ACM ICPC Team Reference - BogoSort

Contents

1 Algorithm	1	4 Data Structure	5	6.14 stable matching	14	10.7 minimal string rotation	20
1.1 LIS	1	4.1 undo disjoint set	5	6.15 Bellman-Ford	14	10.8 reverseBWT	20
1.2 LCS	1	4.2 segment tree range update (lazy propagation)	5	7 Language	14	11 Tree	20
2 Basic	1	4.3 lazy propagation	6	7.1 CNF	14	11.1 Euler tour+RMQ	20
2.1 mod helper function	1	4.4 Lazy heap	6	8 Number Theory	15	11.2 HLD template	20
2.2 self-defined-pq-operator . . .	1	4.5 segment tree prefix sum lower bound	6	8.1 Linear Sieve	15	11.3 dist between u-v in tree	21
2.3 generating all subsets	1	4.6 Fenwick tree (BIT)	7	8.2 C(n,m)	15	11.4 all longest path dfs	21
2.4 tuple	1	4.7 BIT - range update + range prefix sum	7	8.3 Euler Totient 1 to n	15	11.5 all longest path top sort . . .	21
2.5 memset	1	4.8 Order Statistics Tree	7	8.4 derangement (Principle of Inclusion-Exclusion)	15	11.6 Heavy-light decomposition (HLD)	21
2.6 submask enumeration	1	4.9 Trie (Prefix tree)	7	8.5 matrix template (with fast power)	15	11.7 binary lifting	22
2.7 custom-hash	1	4.10 segment tree	7	8.6 Sieve of Eratosthenes (with big num)	15	11.8 get kth ancestor	22
2.8 stringstream split by comma .	1	4.11 BIT range update point query	8	8.7 mod inv	15	11.9 check z on path u-v	22
3 DP	1	4.12 DSU remove node find prev next one	8	8.8 derangement (DP)	15	11.10 path maximum edge between u, v in tree	22
3.1 deque	1	4.13 DSU	8	8.9 Euler Totient	15	11.11 binary jumping	23
3.2 walk	1	5 Geometry	8	8.10 fast power	16	11.12 tree diameter	23
3.3 grouping	2	5.1 cross. product	8	8.11 first and second mex	16	11.13 all longest path	23
3.4 matching	2	5.2 Check Point in Polygon . . .	8	8.12 Catalan Number	16	11.14 tree diameter (len,end)	23
3.5 projects	2	5.3 Point	9	8.13 Chinese Remainder Theorem	16	11.15 minimum distance in func- tional graph	23
3.6 stones	2	5.4 Polygon Lattice Points	9	8.14 mod inv (not prime)	16	11.16 union length of two tree paths	24
3.7 coins	2	5.5 line intersect	9	8.15 mod inv (not coprime)	16	11.17 Centroid decomposition . . .	24
3.8 elevator rides	3	5.6 Polygon area	9	8.16 Mint	16	12 default	25
3.9 slimes	3	5.7 using std::complex	9	8.17 inclusion-exclusion principle .	16	12.1 debug	25
3.10 digit sum	3	5.8 point distance from a line . .	10	8.18 C(n,k) mod inverse	17	12.2 template	25
3.11 sushi	3	6 Graph	10	8.19 Linear Diophantine 丢番圖 .	17	12.3 vimrc	25
3.12 candies	3	6.1 Prim	10	8.20 擴展歐基里德	17	13 other	25
3.13 permutation	4	6.2 Eulerian cycle	10	8.21 Sieve of Eratosthenes	17	13.1 Nim game	25
3.14 Knasack2	4	6.3 maximum weight bipartite matching	10	8.22 builtin	17	13.2 找小於 n 所有出現的 1 數量 .	25
3.15 flowers	4	6.4 maximum flow	11	8.23 C(n,k) DP	17		
3.16 independent set	4	6.5 maximum cardinality bipar- tite matching	12	9 Range Queries	17		
3.17 couting tower	5	6.6 Floyd-Warshall	12	9.1 subarray maximum sum queries	17		
3.18 couting numbers	5	6.7 MST	12	9.2 find first equal or greater . . .	18		
		6.8 Hopcroft-Karp	12	9.3 find k-th and count less than .	18		
		6.9 Dijkstra	12	10 String	19		
		6.10 Dinic	12	10.1 Z	19		
		6.11 maximum cardinality matching	13	10.2 manacher	19		
		6.12 topological sort	13	10.3 AC 自動機	19		
		6.13 Flyod's algorithm	13	10.4 KMP	19		
				10.5 suffix array lcp	20		
				10.6 hash	20		

14 other language	25	14.5 decimal	26	15.1.2 圖論	27	15.1.11 循環小數轉分數 . . .	28
14.1 python heap	25	14.6 python 大數排序	26	15.1.3 dinic 特殊圖複雜度 . .	27	15.1.12 循環小數轉分數 . . .	28
14.2 java	25	14.7 python 大數計算 2	27	15.1.4 0-1 分數規劃	27	15.1.13 常見級數與組合公式 .	28
14.2.1 文件操作	25	14.8 python input	27	15.1.5 學長公式	27	15.1.14 位元運算	28
14.2.2 优先队列	26	14.9 python 大數計算	27	15.1.6 基本數論	27	15.1.15 除數與倍數	28
14.2.3 Map	26	15 zformula	27	15.1.7 排組公式	27	15.1.16 數論公式	29
14.2.4 sort	26	15.1 formula	27	15.1.8 冪次, 冪次和	28	16 Интернационал	29
14.3 python output	26	15.1.1 Pick 公式	27	15.1.9 Burnside's lemma . . .	28	16.1 保佑	29
14.4 python 大數因數分解	26			15.1.10 Count on a tree	28		

ACM ICPC Judge Test - BogoSort

C++ Resource Test

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 namespace system_test {
5
6     const size_t KB = 1024;
7     const size_t MB = KB * 1024;
8     const size_t GB = MB * 1024;
9
10    size_t block_size, bound;
11    void stack_size_dfs(size_t depth = 1) {

```

```

12        if (depth >= bound)
13            return;
14        int8_t ptr[block_size]; // 若無法編譯將
15            block_size 改成常數
16        memset(ptr, 'a', block_size);
17        cout << depth << endl;
18        stack_size_dfs(depth + 1);
19    }
20    void stack_size_and_runtime_error(size_t
21        block_size, size_t bound = 1024) {
22        system_test::block_size = block_size;
23        system_test::bound = bound;
24        stack_size_dfs();
25    }
26    double speed(int iter_num) {
27        const int block_size = 1024;
28        volatile int A[block_size];
29        auto begin = chrono::high_resolution_clock
30            ::now();
31        while (iter_num--)
32            for (int j = 0; j < block_size; ++j)
33                A[j] += j;
34        auto end = chrono::high_resolution_clock::
35            now();
36        chrono::duration<double> diff = end -
37            begin;

```

```

38        return diff.count();
39    }
40    void runtime_error_1() {
41        // Segmentation fault
42        int *ptr = nullptr;
43        *(ptr + 7122) = 7122;
44    }
45    void runtime_error_2() {
46        // Segmentation fault
47        int *ptr = (int *)memset;
48        *ptr = 7122;
49    }
50    void runtime_error_3() {
51        // munmap_chunk(): invalid pointer
52        int *ptr = (int *)memset;
53        delete ptr;
54    }
55    void runtime_error_4() {
56        // free(): invalid pointer
57        int *ptr = new int[7122];
58        ptr += 1;
59        delete[] ptr;
60    }
61    }
62

```

```

63    void runtime_error_5() {
64        // maybe illegal instruction
65        int a = 7122, b = 0;
66        cout << (a / b) << endl;
67    }
68    void runtime_error_6() {
69        // floating point exception
70        volatile int a = 7122, b = 0;
71        cout << (a / b) << endl;
72    }
73    void runtime_error_7() {
74        // call to abort.
75        assert(false);
76    }
77    } // namespace system_test
78
79    #include <sys/resource.h>
80    void print_stack_limit() { // only work in
81        Linux
82        struct rlimit l;
83        getrlimit(RLIMIT_STACK, &l);
84        cout << "stack_size = " << l.rlim_cur << "
85            byte" << endl;
86    }
87

```